

UAM OAuth and OIDC OpenID Provider Implementation Guide - 6.0.1-GA

Table of Contents

1. Preface - UAM OAuth and OIDC OpenID Provider Implementation Guide...	4
2. Introduction to UAM OAuth and OIDC OpenID Provider Implementation.....	5
3. UAM OAuth 2.1 and OIDC Flows.....	11
4. UAM as OIDC OpenID Provider.....	21
4.1. OIDC Scopes.....	22
4.2. Consent Management.....	35
4.3. Consent Page.....	38
4.4. Login Page.....	46
4.5. OIDC Tokens.....	52
4.6. Cross-Origin Resource Sharing (CORS) for OIDC Endpoints.....	67
4.7. Identity Mapping.....	71
5. UAM OAuth and OIDC OP Implementation.....	76
5.1. UAM OAuth and OIDC Integration.....	79
5.2. Implementation Planning.....	82
6. OAuth Client.....	89
6.1. Secure Authorization Request.....	90
6.2. Recommended Flows.....	92
6.3. OAuth Client Registration.....	93
6.3.1. Admin Console Client Registration.....	94
6.3.1.1. Admin Console Client Registration for Recommended Flows...	97
6.3.1.2. Admin Console Client Registration for Deprecated Flows.....	127
6.3.2. Dynamic Client Registration.....	168
6.4. Access Control.....	207
6.5. Deprecated Flows.....	209
7. OAuth Client Authentication.....	210
7.1. Private Key JWT.....	211
7.2. Client Secret.....	212
7.3. PKCE (only for token exchange).....	214

8. Refresh Token.....	215
9. Common Login Page.....	218
10. Custom Login / Consent Page and OAuth Proxy Integration.....	219
11. UAM OAuth Proxy.....	224
11.1. UAM OAuth Proxy Deployment.....	225
11.2. UAM OAuth Proxy Configuration.....	226
12. OpenID Connect Endpoints.....	231
13. Advanced Features.....	260
14. Customizations.....	262
15. Appendices.....	268

1. Preface - UAM OAuth and OIDC OpenID Provider Implementation Guide

This document provides the concept, implementation steps and API integration details of OAuth 2.1 and OpenID Connect (OIDC) with AccessMatrix Universal Access Management (UAM).

Intended Reader

This guide is intended as a reference for the security administrators, i-Sprint's Professional Service personnel, and developers to carry out the following operations:

- Identify the appropriate OIDC and OAuth 2.1 flow to use
- Register and configure OAuth clients
- Customize login page, consent page or OAuth /auth endpoint

Document Scope

This document provides the necessary steps to configure OIDC Endpoints and OAuth 2.1 Flows with details on its concept and API implementation.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [AccessMatrix Common Administration Guide](#)
 - [AccessMatrix Common Login Page Deployment Guide](#)
 - [AccessMatrix REST API Specification](#)
-

2. Introduction to UAM OAuth and OIDC OpenID Provider Implementation

This section provides a basic overview of OIDC, OAuth 2.0, and OAuth 2.1.

OAuth 2.0

OAuth 2.0 is a standard used by applications to grant access to application data securely. It is an open standard that controls authorization to access a protected resource (e.g. web app, native app, or API service) via delegated authorization. Delegated authorization is a system where the resource owner can delegate access to some of these resources to a designated client application.

OAuth defines four roles:

- **Authorization Server**
 - The server issuing access tokens to the client after successfully authenticating the resource owner and obtaining authorization. In this implementation, the AccessMatrix UAM server acts as the authorization server.
 - The authorization server is at the core of both OAuth 2.0 and its OpenID Connect extension. It is simply an OAuth 2.0 token minting engine which has a unique issuer URI and its own signing key for tokens, in order to keep a proper boundary between security domains. In the context of this guide, the AccessMatrix UAM server is the authorization server back end.
 - The authorization server also acts as an OpenID Connect Provider, which means you can request ID tokens in addition to access tokens from the authorization server endpoints.
 - **Resource Owner**
 - Generally, this refers to the application's end user that grants it permission to access the resource server with an access token.
 - **Client**
 - The application that requests the access token from AccessMatrix UAM server and then passes it to the resource server.
 - **Resource Server**
 - The server that accepts the access token and verifies if the access token is valid or not.
-

OAuth 2.0 Grant Flow

In this flow, the client requests authorization from the resource owner (usually the user). If the user gives authorization, the client passes the authorization grant to the authorization server (in this case, AccessMatrix UAM server). Grant is the authorization given or granted to the client by the user. If the grant is valid, the authorization server returns an access token, possibly alongside a refresh and/or ID token. The client then uses this access token to access the resource server.

To view OAuth 2.0 specifications, refer to the [OAuth 2.0 Authorization Framework](#).

OpenID Connect

OpenID Connect (OIDC) is an open standard protocol built on top of the OAuth 2.0 protocol which conveys information about the authenticated users. It adds an additional token called an ID token which is issued by the OpenID Connect Provider and contains information about the end user in the form of claims. As ID tokens are a standard format to contain the user identity and information, OIDC is normally used to achieve Single Sign-On or Federated Authentication.

Although OpenID Connect is built on top of OAuth 2.0, the specification uses slightly different terms for the roles in the flows:

- **OpenID Connect Provider**

The authorization server that issues the ID token. In this implementation, the AccessMatrix UAM server acts as the OpenID Connect provider.

- **End User**

The person whose information is included in the ID token.

- **Relying Party**

The client application that requests the ID token from AccessMatrix UAM server.

OpenID Connect Flow

- The OIDC process flow is the same as the OAuth 2.1 authorization process flow but with the following additions:
 - An ID token is returned by the authorization server in addition to the access token.
 - /userinfo endpoint for getting more user information (if the ID token is not sufficient).
 - openid is passed as a parameter in the scope during the initial call to the authorization server.
-

-
- If an Authorization server is also set up for OpenID Connect, then you can also retrieve the ID token which can be used for user authentication in addition to exchanging the authorization code for an access token.
 - The ID token is returned in JSON Web Token (JWT) format.
 - The ID token is the added piece in OpenID Connect which allows the OAuth 2.1 flow to be used for federated authentication. Federated Authentication is the ability to login to an App (e.g. BoPeep) using your Google login. In this case, BoPeep federates the ability to identify the user to Google.

To view OpenID Connect specifications, refer to the [OpenID Connect](#) site.

OAuth 2.1

OAuth 2.1 is an incremental update to OAuth 2.0 that aims to consolidate best practices and improve security by incorporating lessons learned from the deployment of OAuth 2.0.

The key differences between OAuth 2.0 and OAuth 2.1 are outlined below:

Key Differences

1. **Removal of Implicit Grant and Resource Owner Password Credential Grant**

- OAuth 2.0: Includes:
 - the Implicit grant flow, which was intended for clients that cannot securely store client secrets (e.g., single-page applications).
 - the Resource Owner Password Credentials Flow when
 - If the resource owner (such as device operating system or a highly privileged application) owns both the client application and the resource that it is accessing
 - the application can be trusted to store resource owner login credential
 - other flows or other redirect-based flows are not feasible
 - Does not need to realize SSO and require multi-step authentications
 - OAuth 2.1: Deprecates and removes:
 - Implicit grant due to security concerns, such as access token leakage in URLs and the lack of refresh tokens.
-

-
- Resource Owner Password Credentials Flow, as it insecurely exposes the credentials of the resource owner to the client. Even if the client is benign, it results in an increased attack surface (credentials can leak in more places than just the OAuth Authorization Server) and users are trained to enter their credentials in places other than the OAuth Authorization Server. Furthermore, adapting the resource owner password credentials grant to two-factor authentication, authentication with cryptographic credentials and authentication processes that require multiple steps can be hard or impossible.

2. PKCE for All Clients

- OAuth 2.0: PKCE (Proof Key for Code Exchange) is recommended primarily for public clients, such as mobile and native apps.
- OAuth 2.1: Requires PKCE for all clients, including confidential clients, to mitigate authorization code interception attacks

3. Authorization Code Flow Only

- OAuth 2.0: Supports multiple grant types, including authorization code, implicit, resource owner password credentials, and client credentials.
- OAuth 2.1: Recommends the authorization code flow with PKCE as the primary grant type, removing the implicit and resource owner password credentials flows due to their security vulnerabilities.

4. Refresh Token Usage

- OAuth 2.0: Refresh tokens are optional and their usage is not consistently defined across implementations.
- OAuth 2.1: Encourages the use of refresh tokens with additional security measures, such as binding them to the client and using refresh token rotation to mitigate the risk of token theft.

5. Enhanced Security Recommendations

- OAuth 2.0: Provides general security recommendations but leaves implementation details up to developers.
 - OAuth 2.1: Incorporates more stringent and explicit security recommendations, including using PKCE, enforcing HTTPS, and implementing best practices for token handling and storage.
-

Practical Impact

- **Developers:** Need to update existing OAuth 2.0 implementations to align with the new security best practices outlined in OAuth 2.1.
- **Clients:** Applications must support PKCE and switch to authorization code flow if they were previously using implicit or resource owner password credentials flows.
- **Security:** Overall security is improved, reducing the likelihood of common attacks such as token interception, leakage, and misuse.

OAuth 2.1 represents an evolution of OAuth 2.0 by focusing on security enhancements and best practices. By deprecating less secure grant types, mandating PKCE for all clients, and emphasizing secure token handling, OAuth 2.1 aims to provide a more robust and secure framework for authorization in modern applications.

To view OAuth 2.1 specifications, refer to the [OAuth 2.1 Authorization Framework](#).

OIDC Tokens

Before the intended OAuth flow will be implemented for the OAuth client application, it is better to understand first the usage of various tokens and be able to know on how to work with the tokens granted.

- **Access Token**
 - Access token, which is commonly referred to as `access_token`, is a credential that can be used by an OAuth client application to obtain a protected resource. The resource server would then validate the access token and ensure that it is not expired and that its scope covers the requested resource.
 - This token is issued by the authorization server (i.e. AccessMatrix UAM server) in exchange for the grant. And it can be used by an application to access an API.
 - Access token is usually opaque to the client. Opaque tokens are in a proprietary format that typically contains some information about identity with the use of some identifier. These tokens cannot be parsed by the recipient. To validate an opaque token, the recipient of the token needs to call the server that issued the token.
 - **ID Token**
-

-
- An ID token is a JSON Web Token (JWT) that contains the authenticated user's information in the form of claims. It also contains information about the authentication event.
 - JSON Web Token (JWT) are tokens that conform to the JSON Web Token standard and contain information about identity in the form of claims. It is self-contained and transparent to the recipient of the tokens but these tokens are signed, thus no one could simply edit the token's contents and claims as they wanted. These tokens do not require the recipient to call the issuing server for validation.

- **Refresh Token**

- Refresh token is an optional token that is exchanged for a new access token if the access token has expired.
 - It is a special token that is used to renew the existing access tokens or ID tokens. This allows you to have a short-lived access token without having to collect credentials every time a single token expires. You can request this token alongside the access and/or ID tokens as part of the user's initial authentication flow.
 - A refresh token is a string representing the authorization granted to the client by the resource owner. The string is usually opaque to the client.
-

3. UAM OAuth 2.1 and OIDC Flows

This section covers multiple flows of OAuth 2.1 and OIDC. It can be divided into two sections:

- [Recommended Flows](#)
- [Deprecated Flows](#)

Recommended Flows

Authorization Code Flow with PKCE

The Authorization Code flow with PKCE is a secure way of obtaining access tokens from an authorization server. This allows for access to protected resources (such as web APIs), without the sharing of user credentials. The PKCE challenge is used with Authorization Code grant to protect the authorization code. This prevents attackers from hijacking and gaining unauthorized access to tokens.

i **Note:** You may refer to [Appendices](#) for more details about PKCE.

The Authorization Code flow with PKCE can be used for both:

- [Confidential clients](#)
- [Public clients](#)

Confidential Clients

The Authorization Code flow with PKCE is best used by confidential web applications (that has end users) that are executed on a server. In such a scenario, the server side of the web application acts as the OIDC Relying Party. In addition, the server is stateful for each user session.

This flow relies on the server to execute a request that exchanges an authorization code using client secret or private key JWT authentication methods to obtain an access token from the AccessMatrix UAM server. The application then interacts with the end user's web browser, where it redirects the user for login (optionally, for consent as well) and then receives the authorization code.

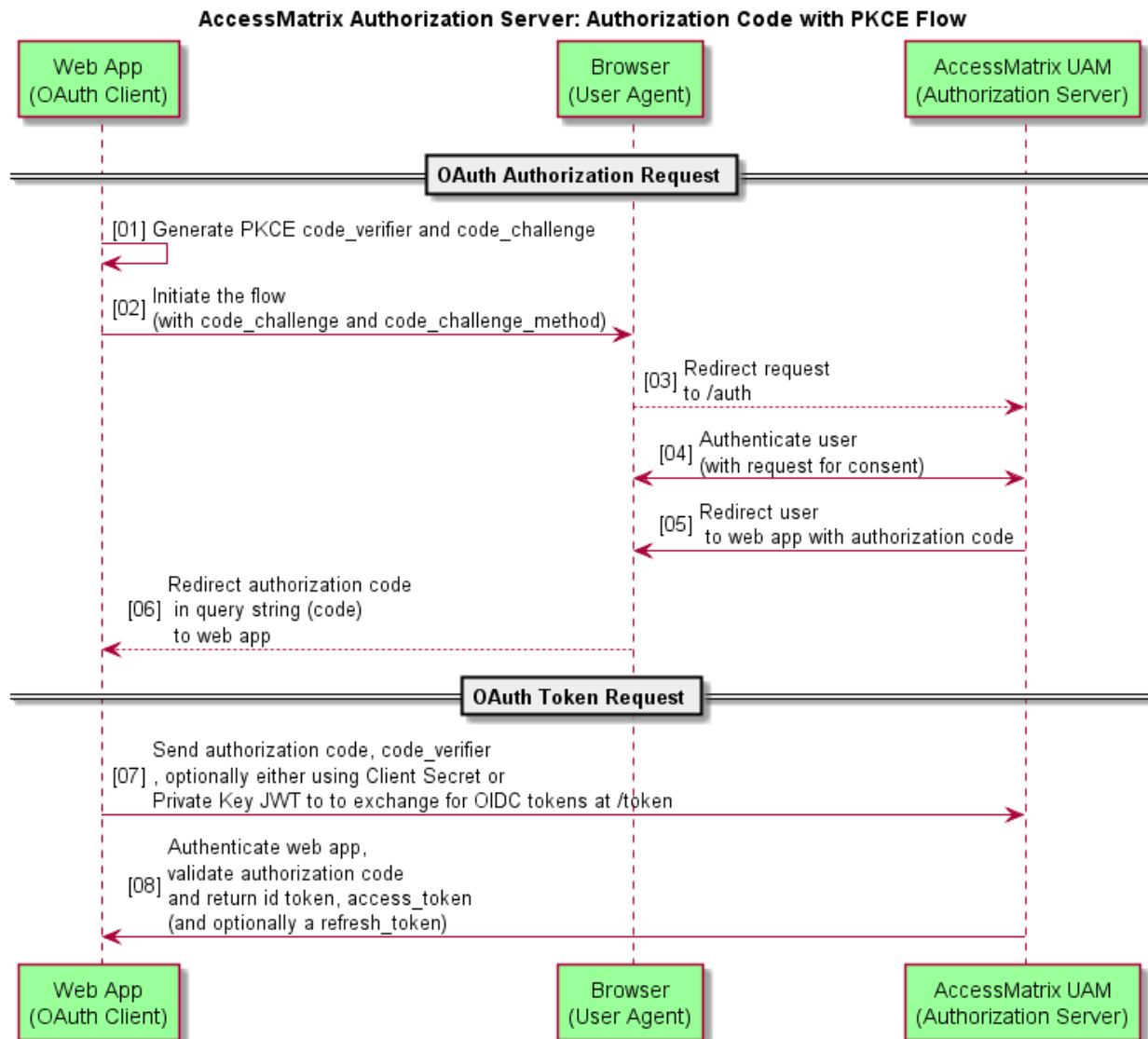
The PKCE extension provides added security to mitigate the authorization code interception attack.

Public Clients

The Authorization Code flow with PKCE can be used by public clients such as mobile native applications, .

For public clients, the OAuth client application's client secret could be exposed when using the Authorization Code flow. The PKCE extension helps to overcome the vulnerability of the exposed client secret and mitigate authorization code interception attacks.

Sequence Diagram



Click [here](#) to learn more about the configuration for this flow.

Client Credentials Flow

The Client Credentials flow can be used as an authorization grant when the authorization scope is limited to the protected resources under the control of the client, or to protected resources previously arranged with the authorization server. Client credentials are used as an authorization grant typically

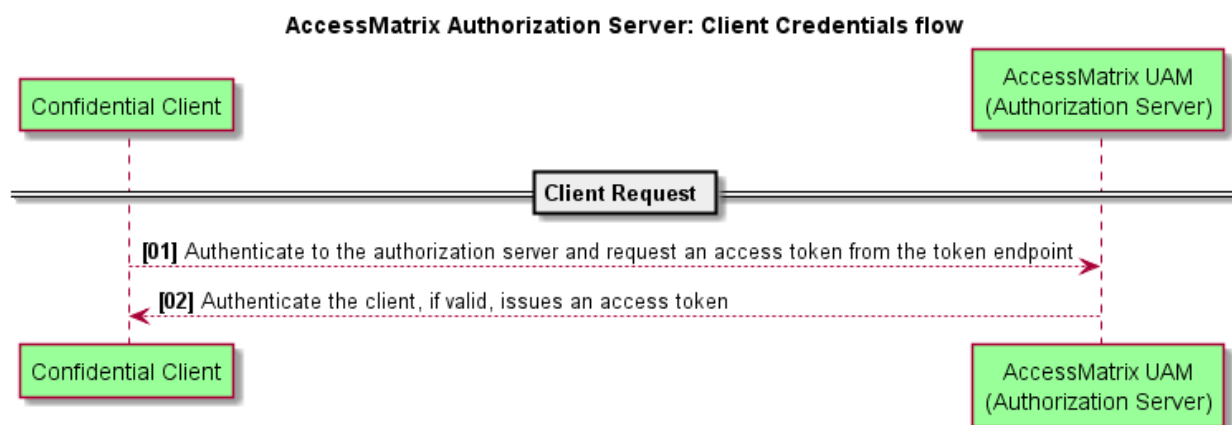
when the client is acting on its own behalf (the client is also the resource owner) or is requesting access to protected resources based on an authorization previously arranged with the authorization server.

In this flow, the client can request an access token using only its client credentials (or other supported means of authentication) when the client is requesting access to the protected resources under its control, or those of another resource owner that has been previously arranged with the authorization server.



Note: The client credentials grant type should only be used by confidential clients.

Sequence Diagram



Click [here](#) to learn more about the configuration for this flow.

Hybrid Flow with PKCE

The Hybrid flow with PKCE is an OpenID Connect flow that incorporates characteristics of both the Implicit flow and the Authorization Code flow with PKCE. Refer to [Appendices - D. About PKCE](#) for more details about PKCE.

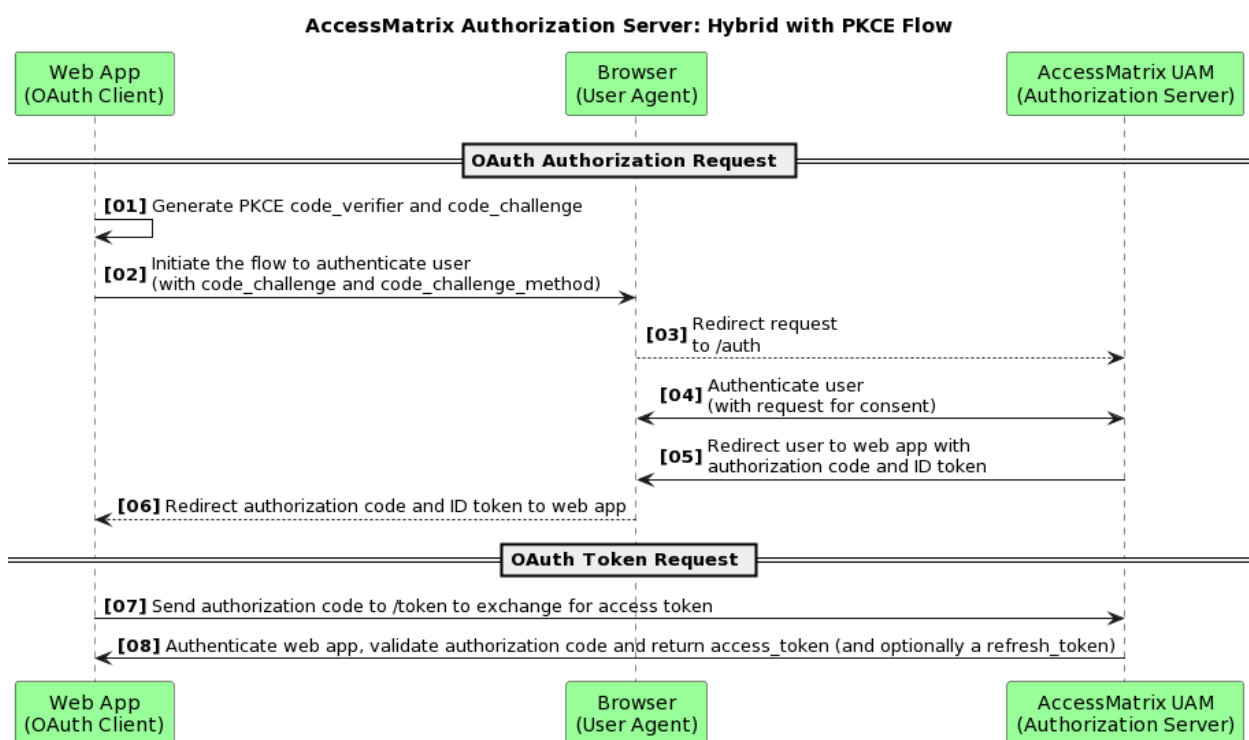
Confidential clients are recommended for this flow, as opposed to public clients. Confidential clients are OAuth clients that can securely store client credentials, which include client secrets and other user-identifiable information. Public clients are OAuth clients that are unable to and should avoid storing client credentials. This refers to any client that can be compromised if the application is running on an insecure device.

Confidential clients are recommended because the flow requires the client to authenticate to the authorization server using its credentials (such as the client secret or Private Key JWT).

The Hybrid flow with PKCE enables clients to immediately access user info using an ID token. However, they can still obtain secure, long-lived access tokens and refresh tokens to access protected resources in the later steps of the flow by using the authorization code.

i Note: This flow does not allow "code id_token token" to be used, so that the OP server only returns the authorization code and ID token to the scripting-based front-end application, not the access token. Access token is typically used to access resources (e.g. API), it should be securely obtained (by the client application backend to exchange authorization code for the access token) and stored prior to use.

Sequence Diagram



Click [here](#) to learn more about the configuration for this flow.

Deprecated Flows

Authorization Code Flow

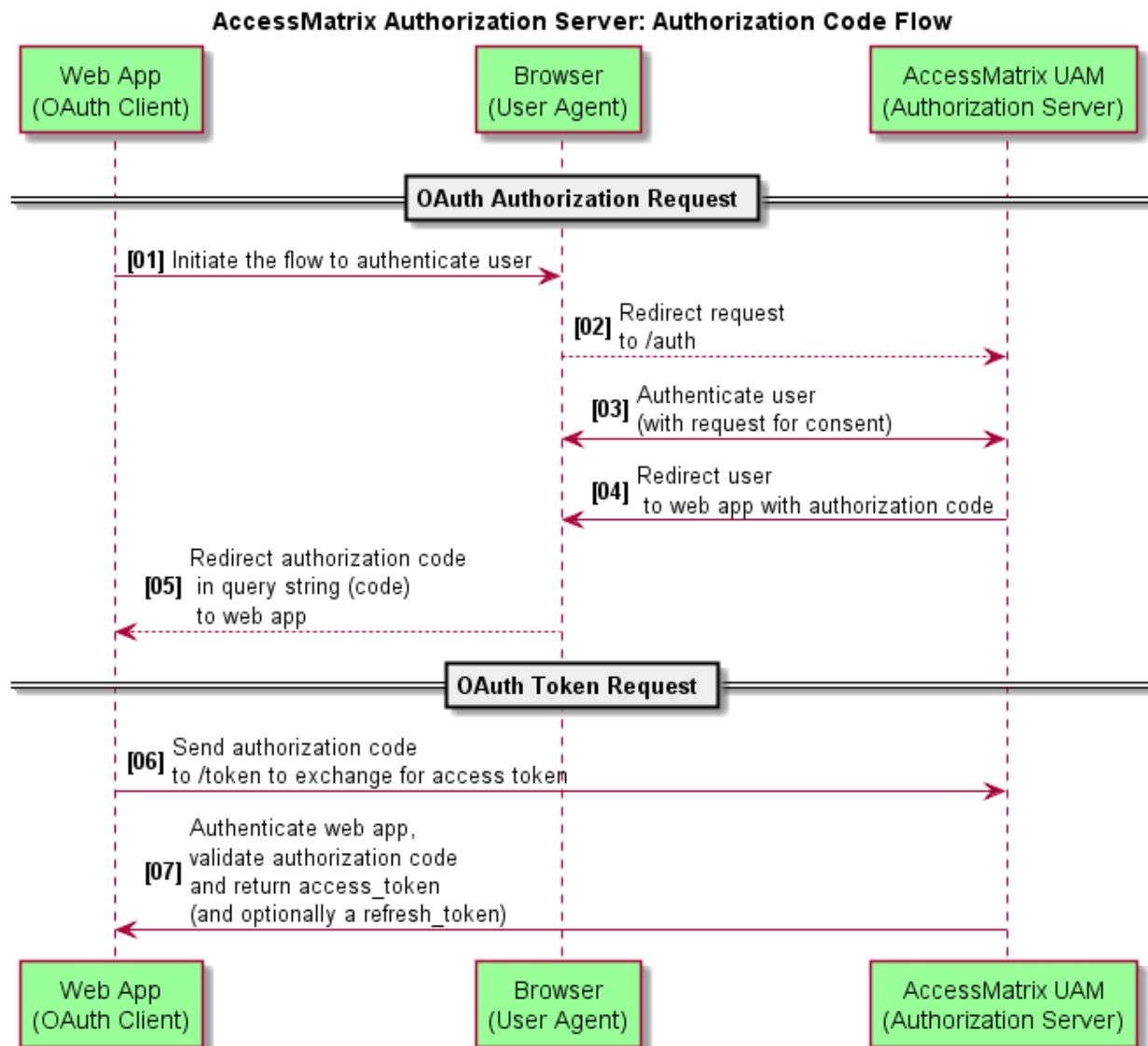


Note: Based on the new OAuth best practice guidelines and the latest OAuth 2.1 draft specification, the PKCE challenge must be used with the Authorization Code grant whenever possible. It is recommended that you use the Authorization Code with PKCE flow instead.

The Authorization Code flow is best used by a confidential web application that has end users where its execution is on a server (the server is stateful on the user's session, and the server-side of the web application is the OIDC Relying Party). It relies on the server to execute a request that exchanges authorization code using the client secret to obtain an access token from the AccessMatrix UAM server. The application interacts with the end user's web browser where it redirects the user for login (optionally, for consent as well) and then receives the authorization code.

Usually, the web application has its user session (web session or other forms of session ID).

Sequence Diagram



Click [here](#) learn more about the configuration for this flow.

Implicit Flow

This flow is deprecated, Authorization Code with PKCE flow should be used instead. Previously this flow is recommended if an application is not capable to secure the confidentiality of the client secret - which is used as a form of client authentication to exchange an authorization code for OIDC tokens. Such OAuth client application is a single-page application (SPA) (that is a scripting-based front-end web application running in a browser).



The single-page application (SPA) may also have microservices as its back end. Its architecture is normally stateless, where it breaks down the server to multiple services to isolate changes and to allow faster deployment. API Gateway is normally deployed in front of the microservices for the routing of the APIs and also for verifying the user's session. In this case, the ID token can be used as the user's session or as a token to identify the user when accessing the microservices. The SPA usually communicates to microservices via web service API (normally REST API). Single Sign-On is important in SPA with microservices back end architecture to allow developers to focus on the business logic in the microservices.

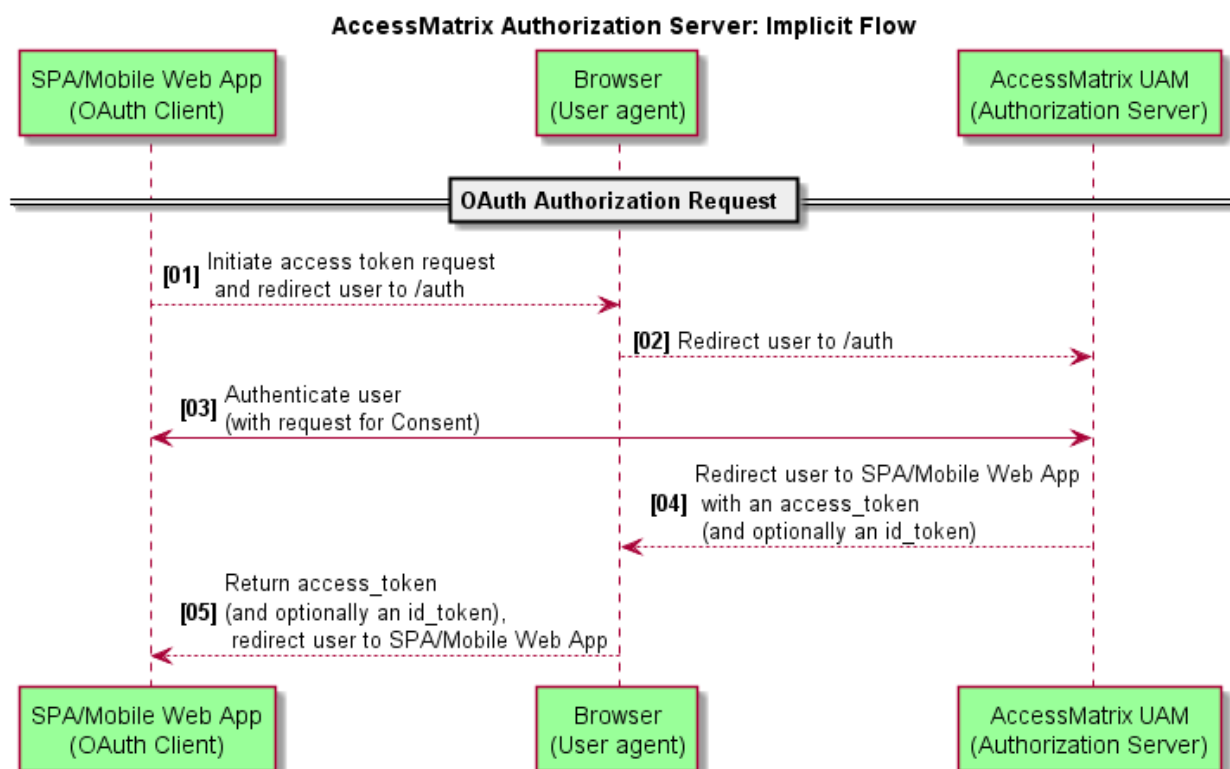
formation text here.

SPA will interact with the web browser, the resource owner's user-agent, and also to be redirected from the authorization server to receive incoming request and response where the OIDC tokens can be retrieved straight away. However, as part of the security consideration, it is important to note that the access token is exposed on the client side in this flow,.



Note: Implicit flow does not return a refresh token because the browser cannot keep it private.

Sequence Diagram



Click [here](#) learn more about the configuration for this flow.

Resource Owner Password Credentials Flow

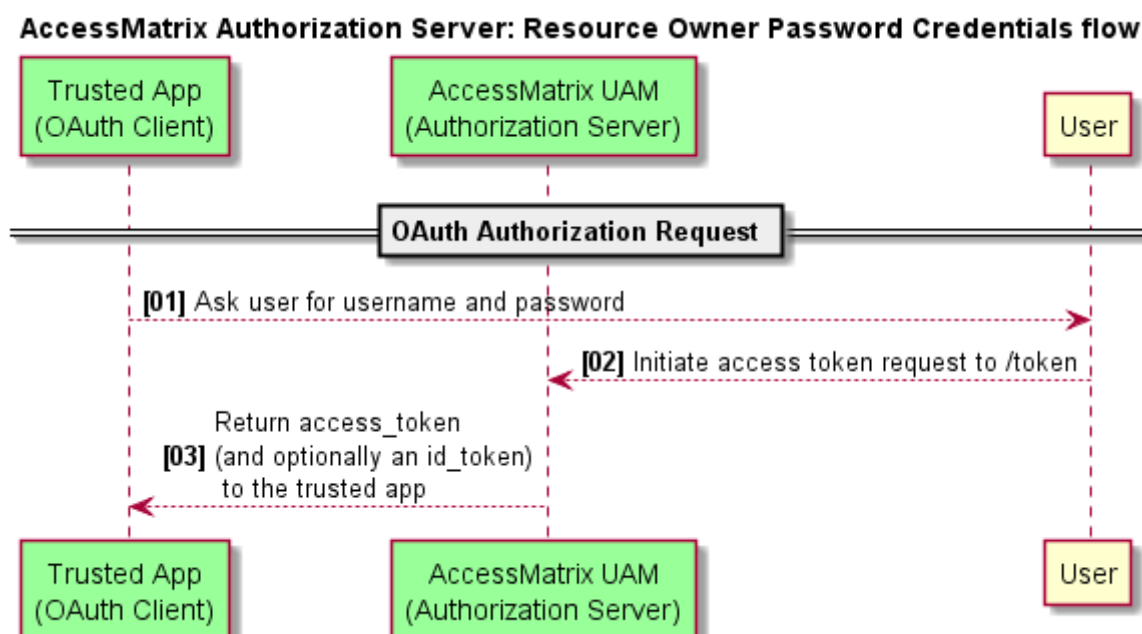
i Note: Based on the new OAuth best practice guidelines and the latest OAuth 2.1 draft specification, the Resource Owner Password Credentials flow is deemed highly insecure and should not be used for new applications. It is recommended that you use the Authorization Code with PKCE flow instead.

Resource owner password credentials flow is used if the resource owner is trusted to have a control on both the client application and the resource it is interacting with, and the redirect-based flow (via a web browser) is not feasible.

In this flow, the end-user will be asked to fill in the credentials (username/password) typically using an interactive form. This information will be sent to the back end and then to the authorization server. This flow is suitable for the application that is capable of obtaining the resource owner's credentials (username and password, typically using an interactive form) and is absolutely trusted with the information.

You should minimize the use of this application type whenever possible as due to the nature that user password is exposed to the application and there is a risk of unintentional disclosure of password (e.g., via log files or other records kept by the application). This application cannot realize SSO and also cannot support multi-step authentications.

Sequence Diagram



Click [here](#) learn more about the configuration for this flow.

Hybrid Flow

i Note: Based on the new OAuth best practice guidelines and the latest OAuth 2.1 draft specification, the PKCE challenge must be used with the Authorization Code grant whenever possible. It is recommended that you use the Hybrid with PKCE flow instead.

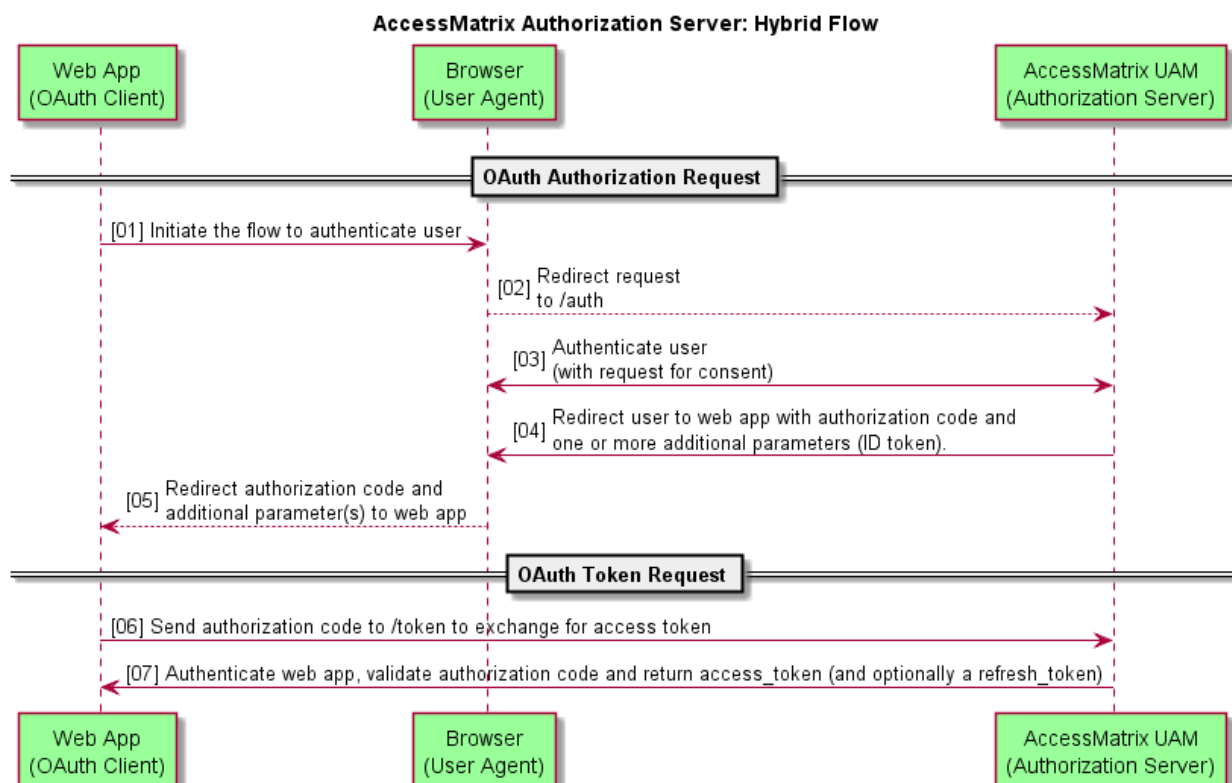
The Hybrid flow is an OpenID Connect flow that incorporates characteristics of both the Implicit flow and the Authorization Code flow.

For applications that can securely store their client secrets, this flow enables them to obtain immediate access to an ID token for information about the user, while also enabling them to retrieve access and refresh tokens to gain access to protected resources for an extended period of time.

The response from the authorization server is determined based on the combination of values in the `response_type` request parameter. For example:

- If "code id_token" is used, then the server returns the authorization code and an ID token.

Sequence Diagram



Click [here](#) to learn more about the configuration for this flow.

4. UAM as OIDC OpenID Provider

The following pages provide additional details on OAuth 2.1 and OIDC features supported by UAM:

- [OIDC Scopes](#)
- [Consent Management](#)
- [Consent Page](#)
- [Login Page](#)
- [Token Format and Cryptography Management](#)
- [Cross-Origin Resources Sharing \(CORS\) for OIDC Endpoints](#)
- [Identity Mapping](#)

Implementation Checklist

1. Read [OIDC Scopes](#) and [Consent Management](#) to determine if scopes (apart from "openid") and user consent prompts are required for the implementation.
2. AccessMatrix UAM provides OAuth Proxy as the OIDC endpoints component which can be used together with the built-in authentication front end. This can be the AccessMatrix Common Login Page (CLP) or your custom login page. Refer to the following tasks:

Options	Tasks
Standard (CLP)	Configure Standard Login Page
	Configure Standard Consent Page (Optional)
Custom	Custom Login Page and OAuth Proxy Integration

3. Set up the OIDC token signing key for the OAuth client application in the AccessMatrix OAuth authorization server. Refer to [Configure OIDC Token Signing Key](#) for details.
 4. Configure the OIDC endpoints URL for discovery use. Refer to [OpenID Connect Endpoints URL](#) for details.
-

4.1. OIDC Scopes

In the context of OAuth/OIDC, scopes are used to authorize access to a user's protected resources.

Scopes can be used in two ways:



Note: For more details about scopes and user consent, refer to [Consent Management](#).

- Scope values can be used in the application to specify what access privileges are being requested for access tokens. The scopes associated with access tokens determine what resources will be available when they are used to access OAuth 2.0 protected endpoints. Protected resource endpoints may perform different actions and return different information based on the scope values and other parameters used when requesting with the presented access token.
- For OpenID Connect, scopes can be used to request that specific sets of information be made available as claim values.

During the OIDC authentication flow ([/auth](#) is called), if a scope is required by an application and user is expected to authorize access for the user info (scope requires consent), a consent screen that describes the user information would be prompted. For example, when a user logs in for the first time to an application, they might be asked to give the application access to their email address and basic profile information. In a typical use case however, the application requests access to this information at the authorization endpoint using the scope parameter.



Note: For more details on returning additional user info to the OAuth client without the use of scopes, refer to [Customizations - Returning User Info](#).

To [configure scopes in OAuth Client](#), they have to first be defined under OP. There are two ways to configure scopes in OP:

- [Using Admin Console](#)
- [Using REST API](#)

Configure OP Scopes via Admin Console

User Info can requested using scope parameter via [/auth](#) or [/token](#) endpoints. If your OpenID Provider / authorization server has scopes that allow application access, you may define the supported scopes and configure how you intend the requested user info to be returned (via scope)

Below is the steps of configuration to achieve that:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Authz Server**.
2. Click the **Edit** button and click on the **Scope Management** tab.
3. Click the **Edit** button in the **Scopes** attribute to launch the **Scopes Configuration** dialogue.
4. To add a custom scope, click the **Add** button.

- a. Specify the **Scope Name**.
- b. Select any value for **User Response as Scope Value**.



Note: This field can be left empty. For more details on this field, refer to [Customize User Response](#).

- c. Select the **Consent Rule** value. Set "REQUIRED" if it requires user consent, otherwise, set to "NOT_REQUIRED".
 - d. Check the corresponding checkbox to return claims in **ID Token, User Info, or Access Token (JWT/JWE)**.
 - e. Click the **Add** button if you wish to add more custom scopes. Otherwise, click **OK** to add the custom scope to the list of Scopes.
5. To add a predefined scope, click the **Add OpenID Connect Predefined Scope** button.
 - a. Select the **Scope Name** from the predefined list. A value will automatically be set to the **User Response as Scope Value** field.
 - b. Select the **Consent Rule** value. Set "REQUIRED" if it requires user consent, otherwise, set to "NOT_REQUIRED".
 - c. Check the corresponding checkbox to return claims in **ID Token, User Info, or Access Token (JWT/JWE)**.
 - d. Click the **Add OpenID Connect Predefined Scope** button if you wish to add more predefined scopes. Otherwise, click **OK** to add the predefined scope to the list of scopes.
 6. Click the **Save** button to save the changes.

Viewing Granted Scopes

Administrators can view the scopes that a user has granted to an OAuth client. This can be done using the Admin Console:

-
1. Navigate to **Operation Dashboard > User & Segment > User**.
 2. Specify the relevant search criteria under **Search User**.
 3. Click the **Search** button.
 4. Select the relevant user under the **User Id** column to display the **View User** page.
 5. Select the **OAuth Consented Scopes** tab to view a list of all the scopes the user has granted to OAuth clients.

i **Note:** Only OAuth clients with scopes granted by a consenting user will be displayed. Individual scopes are separated by spaces in the list.

Configure OP Scopes via REST API

This section provides detailed information on managing scopes in the OAuth authorization server via REST API.

i **Notes:**

- There are various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs:
 - Admin session token
 - Dynamic agent session
 - Access token
- The following examples below use the dynamic agent session authentication method. Refer to [Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs](#) for more information on each of these methods.

Create Scopes

- Use the `authz/oauth/scope/create` API to create scopes (i.e. define the supported scopes) in the OAuth authorization server.
- Use the **HTTP POST** method for this API call.
- Refer to the [AccessMatrix REST API Specification](#) for the details of this API.

The following parameters can be posted as an application/json media type to the API:

Request Parameters:

Parameter	Data Type	Description
dynamicAgentSession	Boolean	Specifies if a dynamic agent session is used. If unspecified, the default value is "false".
scopeName	String	The name of the scope. This is a required parameter.
consent	String	Specifies if consent is required or not required for the scope. This is a required parameter.
userResponseId	String	The Id of the user response to be used as the scope value. If unspecified, no user response will be returned as the scope value.
isReturnUserInfo	Boolean	Specifies if the scope is returned with user info. If unspecified, the default value is "false".
isReturnInIdToken	Boolean	Specifies if the scope is returned with Id tokens. If unspecified, the default value is "false".
isReturnInAccessToken	Boolean	Specifies if the scope is returned with access tokens. If unspecified, the default value is "false".

Response Result:

A successful response returns the HTTP "200 OK" status along with the created scope as a JSON object, which contains the following:

Property	Data Type	Description
scopeName	String	The name of the scope.
consent	String	Specifies if consent is required or not required for the scope.
userResponseId	String	The Id of the user response.
userResponseUUID	String	The UUID of the user response.
isReturnUserInfo	Boolean	Specifies if the scope is returned with user info.
isReturnInIdToken	Boolean	Specifies if the scope is returned with Id tokens.
isReturnInAccessToken	Boolean	Specifies if the scope is returned with access tokens.

Examples:

Request Example
JSON <pre>{ "dynamicAgentSession" : true, "scopeName" : "scope1", "consent" : "REQUIRED", "userResponseId" : "OIDC_email", "isReturnUserInfo" : false, "isReturnInIdToken" : false, "isReturnInAccessToken" : false}</pre>
Response Example (Success)
JSON <pre>{ "result": { "scope": { "scopeName": "scope1", "isReturnUserInfo": false, "isReturnInIdToken": false, "consent": "REQUIRED", "userResponseId": "OIDC_email", "isReturnInAccessToken": false, "userResponseUUID": "ze" } }, "amProcessingTimeMillis": 15}</pre>

Possible Errors:

Error Code	Message	Description
70010 (InvalidScope)	Scope name is empty	The scope name was not specified.
	Scope is not found	There is no existing scope with the specified scope name.
	Invalid user response	There is no existing user response corresponding to the specified user response Id.
	Invalid consent	There was no valid value specified for the consent parameter.

Update Scopes

- Use the `authz/oauth/scope/update` API to update an existing scope in the OAuth authorization server.
- Use the **HTTP POST** method for this API call.
- Refer to the [AccessMatrix REST API Specification](#) for the details of this API.

i **Note:** When calling this API, all the relevant parameters should be specified, including those that remain the same. This is because updating a scope replaces all its existing information with the newly provided parameters. Thus, if any parameters are unspecified, the existing values will be replaced with the default values.

The following parameters can be posted as an `application/json` media type to the API:

Request Parameters:

Parameter	Data Type	Description
<code>dynamicAgentSession</code>	Boolean	Specifies if a dynamic agent session is used. If unspecified, the default value is "false".
<code>scopeName</code>	String	The name of the scope. This is a required parameter.
<code>newScopeName</code>	String	The new name of the scope, if applicable. If unspecified, the scope name will remain the same (as specified in <code>scopeName</code>).
<code>consent</code>	String	Specifies if consent is required or not required for the scope. This is a required parameter.
<code>userResponseId</code>	String	The Id of the user response to be used as the scope value. If unspecified, no user response will be returned as the scope value.
<code>isReturnUserInfo</code>	Boolean	Specifies if the scope is returned with user info. If unspecified, the default value is "false".

Parameter	Data Type	Description
isReturnInIdToken	Boolean	Specifies if the scope is returned with Id tokens. If unspecified, the default value is "false".
isReturnInAccessToken	Boolean	Specifies if the scope is returned with access tokens. If unspecified, the default value is "false".

Response Result:

A successful response returns the HTTP "200 OK" status along with the updated scope as a JSON object, which contains the following:

Property	Data Type	Description
scopeName	String	The name of the scope.
consent	String	Specifies if consent is required or not required for the scope.
userResponseId	String	The Id of the user response.
userResponseUUID	String	The UUID of the user response.
isReturnUserInfo	Boolean	Specifies if the scope is returned with user info.
isReturnInIdToken	Boolean	Specifies if the scope is returned with Id tokens.
isReturnInAccessToken	Boolean	Specifies if the scope is returned with access tokens.

Examples:

Request Example	
JSON	<pre>{ "dynamicAgentSession" : true, "scopeName" : "scope1", "newScopeName" : "scope2";</pre>

Request Example	
	<pre> "consent" : "NOT_REQUIRED", "userResponseId" : "OIDC_profile", "isReturnUserInfo" : true, "isReturnInIdToken" : true, "isReturnInAccessToken" : true </pre>
Response Example (Success)	
JSON	<pre> { "result": { "scope": { "scopeName": "scope2", "isReturnUserInfo": true, "isReturnInIdToken": true, "consent": "REQUIRED", "userResponseId": "OIDC_profile", "isReturnInAccessToken": true, "userResponseUUID": "ze" } }, "amProcessingTimeMillis": 15 } </pre>

Possible Errors:

Error Code	Message	Description
70010 (InvalidScope)	Scope name is empty	The scope name was not specified.
	Scope is not found	There is no existing scope with the specified scope name.
	New scope name exists	There is already an existing scope with the same scope name as the specified new scope name.
	Invalid user response	There is no existing user response corresponding to the specified user response Id.
	Invalid consent	There was no valid value specified for the consent parameter.

View Scopes

- Use the authz/oauth/scope/view API to view an existing scope in the OAuth authorization server.
- Use the **HTTP POST** method for this API call.
- Refer to the [AccessMatrix REST API Specification](#) for the details of this API.

The following parameters can be posted as an application/json media type to the API:

Request Parameters:

Parameter	Data Type	Description
dynamicAgentSession	Boolean	Specifies if a dynamic agent session is used. If unspecified, the default value is "false".
scopeName	String	The name of the scope. This is a required parameter.

Response Result:

A successful response returns the HTTP "200 OK" status along with the specified scope as a JSON object, which contains the following:

Property	Data Type	Description
scopeName	String	The name of the scope.
consent	String	Specifies if consent is required or not required.
userResponseId	String	The Id of the user response.
userResponseUUID	String	The UUID of the user response.
isReturnUserInfo	Boolean	Specifies if the scope is returned with user info.
isReturnInIdToken	Boolean	Specifies if the scope is returned with Id tokens.
isReturnInAccessToken	Boolean	Specifies if the scope is returned with access tokens.

Examples:

Request Example
JSON
<pre>{ "dynamicAgentSession" : true, "scopeName" : "scope2" }</pre>
Response Example (Success)
JSON
<pre>{ "result": { "scope": { "scopeName": "scope2", "isReturnUserInfo": true, "isReturnInIdToken": true, "consent": "REQUIRED", "userResponseId": "OIDC_profile", "isReturnInAccessToken": true, "userResponseUUID": "ze" } }, "amProcessingTimeMillis": 15 }</pre>

Possible Errors:

Error Code	Message	Description
70010 (InvalidScope)	Scope name is empty	The scope name was not specified.
	Scope is not found	There is no existing scope with the specified scope name.

Delete Scopes

- Use the `authz/oauth/scope/delete` API to delete existing scopes in the OAuth authorization server.
- Use the **HTTP POST** method for this API call.
- Refer to the [AccessMatrix REST API Specification](#) for the details of this API.

The following parameters can be posted as an `application/json` media type to the API:

Request Parameters:

Parameter	Data Type	Description
dynamicAgentSession	Boolean	Specifies if a dynamic agent session is used. If unspecified, the default value is "false".
scopeName	String	The name of the scope. This is a required parameter.

Response Result:

A successful response returns the HTTP "200 OK" status along with the deleted scope as a JSON object, which contains the following:

Property	Data Type	Description
scopeName	String	The name of the scope.
consent	String	Specifies if consent is required or not required.
userResponseld	String	The Id of the user response.
userResponseUUID	String	The UUID of the user response.
isReturnUserInfo	Boolean	Specifies if the scope is returned with user info.
isReturnInIdToken	Boolean	Specifies if the scope is returned with Id tokens.
isReturnInAccessToken	Boolean	Specifies if the scope is returned with access tokens.

Examples:

Request Example
JSON <pre>{ "dynamicAgentSession" : true, "scopeName" : "scope2" }</pre>
Response Example (Success)
JSON

Request Example

```
{
  "result": {
    "scope": {
      "scopeName": "scope2",
      "isReturnUserInfo": true,
      "isReturnInIdToken": true,
      "consent": "REQUIRED",
      "userResponseId": "OIDC_profile",
      "isReturnInAccessToken": true,
      "userResponseUUID": "ze"
    }
  },
  "amProcessingTimeMillis": 15
}
```

Possible Errors:

Error Code	Message	Description
70010 (InvalidScope)	Scope name is empty	The scope name was not specified.
	Scope is not found	There is no existing scope with the specified scope name.

Whitelisting OAuth Client Scopes

Administrators can define which scope(s) are allowed to be returned with the access token when the /token endpoint or /auth endpoint is called with the scope(s) specified. To configure the whitelisted scopes:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**
2. Click on the relevant OAuth Client under the **OAuth client Id** column.
3. Click the **Edit** button.
4. Select the scope(s) to be whitelisted under the **Whitelisted Scopes (Press Ctrl + Click to select multiple options)** attribute.
5. Click the **Save** button to save the changes.

Depending on whether whitelisted scope(s) have been defined in the OAuth client configuration, the following scenarios are possible:

Scope Specified in Request

- **When OAuth client calls the /auth or /token endpoint:**
 - If whitelisted scope(s) are defined in the OAuth client configuration:
 - If at least one of the specified scope(s) in the request are not whitelisted, then an exception is returned.
 - If all of the specified scope(s) in the request are whitelisted, the authorization server responds with the access token.
- Note:** The list of specified scope(s) in the request does not need to be an exact match with the full list of whitelisted scope(s), so long as all the scope(s) specified have been whitelisted.
- If whitelisted scope(s) are not defined in the OAuth client configuration:
 - When using Client Credential flow (Server to Server application), an exception will be returned. This is because the flow does not involve user authentication (or user consent management). As such, some scopes defined under the OAuth authorization server may not be applicable for this flow (i.e. are limited to OAuth clients using different flows) and will not be returned.
 - When using other OAuth client flows, it is assumed that the OAuth client is able to use all scopes from the OAuth authorization server, and the authorization server responds with an access token. This allows for backward compatibility with older versions where the scope whitelisting feature is not available.

Scope Not Specified in Request

- **When OAuth client calls the /auth or /token endpoint:**
 - The authorization server responds with the access token using the scope(s) provided in the authorization code (for Authorization Code flow) or refresh token.

Note: This behavior applies regardless of whether whitelisted scope(s) are defined in the OAuth client configuration.

4.2. Consent Management

Same-Entity Application vs Different-Entity Application

Generally, applications that require certain scopes should obtain user consent. This allows the application to be authorized in accessing the end user's information. Depending on the application type, different guidelines for consent management will apply:

- **Same-Entity Applications**

Same-entity applications are OAuth clients controlled by the same organization, or when they are also the OP / authorization server. These applications are considered as trusted applications, where user consent to access end user information can be skipped. For example, if this application is an enterprise e-leave application portal for employees, you should not demand user consent from end users if they want to access the application to apply for leave.

- **Different-Entity Applications**

Different-entity applications are OAuth clients controlled by an organization which is considered a different and separate entity from the OP/authorization server. Such applications must not have administrative access to the OP/authorization server. Usually, different-entity applications are for external parties or partners to access protected resources. Different-entity application cannot be trusted and its access to end user information should require user consent. For example, if you are planning to create a developer center that allows users to obtain some resources and integrate their applications with your API (this functionality can be provided by those well-known social network login applications like Facebook, Google, or those sizeable institutions like Government or Financial Institution), you should configure it as a different-entity application.

Configure Application Entity Type

You can define whether your application is controlled by the same entity or different entity based on Consent rule setting in OAuth client. By default, **Consent Rule** is set as "NOT_TRUSTED" for different-entity applications.

To change this setting:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.

-
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
 3. Click the **Edit** button.
 4. Click the **OpenID Connect** tab.
 5. Select "TRUSTED" in the **Consent Rule** attribute if your application is a same-entity application.



Note: Setting Consent Rule to "TRUSTED" means the application does not require end user consent.

6. Click the **Save** button to save the changes.

User Consent Mechanism at OP /auth endpoint



Note: User consent rules are used in conjunction with scopes. Refer to [OIDC Scopes](#) for more details about configuring scopes.

Depending on how the prompt parameter has been configured, it affects how the authorization (/auth) endpoint checks for user consent:

- If the **prompt** parameter has not been configured, the /auth endpoint will check for user consent by default.
- If the **prompt** parameter has been set as "consent", the OP will always prompt the End-User for consent before returning information to the Client. An error is returned if consent cannot be obtained, typically `consent_required`.
- If the **prompt** parameter has been set as "none", the OP will not prompt the user for authentication or consent. An error is returned if:
 - An End-User is not already authenticated.
 - The Client was not granted with consent of the requested scope(s).
 - The Client does not fulfill other conditions for processing the request.

The following illustrations describe the mechanisms of user consent at the /auth endpoint:

- User consent is required for the following scenarios when:
 - The application passes the scope parameter to the /auth endpoint with multiple scope values.



Note: These scope values are space delimited and include `openid`.

- The **prompt** parameter has been set as "consent", even though the user had already granted consent for the requested scopes before.
-

-
- **Consent rule** is set as "NOT_TRUSTED" in OAuth client, at least one of the consent rules of the requested scopes is set as "REQUIRED", and the scope consent has yet to be granted.
 - User consent is not required in the following scenarios when:
 - **Consent rule** is set as "TRUSTED" in OAuth client (that means this is a same-entity application that can be trusted);
 - **Consent rule** is set as "NOT_TRUSTED" in OAuth client (that means this is a different-entity application and it cannot be trusted), but all consent scope is set as "NOT_REQUIRED" in the OP;
 - All "REQUIRED" consent scopes have previously been granted.
-

4.3. Consent Page

Standard Consent Page

Before using the standard consent page, there are certain configurations that must be completed. You may refer to the table below:

Tasks	Reference / Instructions
Configure Common Login Page	Refer to AccessMatrix Common Login Page Deployment Guide - Customize Display Content of Scope(s) for more details on how to add descriptions to the OAuth scope values.
Configure Term of Service and Privacy Policy URLs	Terms Of Service URL and Privacy Policies URL attribute values should be provided if the application requires user consent. 1. Navigate to Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client. 2. Click on the relevant OAuth client under the OAuth Client Id column. 3. Click the Edit button and select the OpenID Connect tab. 4. Specify the values for the Terms Of Service URL and Privacy Policies URL attributes. 5. Click the Save button to save the changes. Note: These URLs will be displayed in the Common Login Page (CLP) consent page when the user is prompted to give consent for a certain scope.

Customized Consent Page

AccessMatrix Universal Access Management (UAM) allows for custom authorization UIs to be used. The user should first be authenticated, and their session token set as a cookie. When the OAuth client requires user consent, the authorization UI should then be displayed for the user to grant the OAuth client access to their data.

The authorization UI must call the following APIs to retrieve the scopes which require user consent, and to save the user's consent on the specified scopes:



Note: For more details, please refer to the Authorization Service in the [AccessMatrix REST API Specification](#).

- `/authz/oauth/scopesRequireUserConsent/get`
 - To retrieve the scope(s) which requires user consent based on the specified scopes.
 - Returns other useful information for consent UI use (e.g. OAuth client name, OAuth client Id, consent term or service URL, and consent privacy policies URL).
- `/authz/oauth/myGrantedScopes/consent`
 - To give consent to the OAuth client for the specified scopes on user's behalf. This API would be called using user's session token after the user has granted consent on the authorization UI.

If required, you can use these APIs to view the user's consented scopes:

- `authz/oauth/myGrantedScopes/get`
 - This API is used to get end user's consented scopes based on their login session (aka session token).
- `authz/oauth/grantedScopes/get`
 - This API is used to retrieve end user's consented scopes by user identity when an admin/agent session is used.

Refer to [OIDC Scopes](#) and [Consent Management](#) for more information on their respective topics.

OAuth Proxy Configuration

Refer to [Configure Login and Consent Page Redirect URL](#) for more details on how to configure the Consent Page Redirect URL.

Using the OAuth /auth Endpoint for User Consent Auto Saving

For existing OAuth Proxy (or CLP) deployment projects that have customized consent pages and rely on the OAuth /auth endpoint for user consent auto saving, it is necessary to ensure that the custom consent page has implemented the `authz/oauth/myGrantedScopes/consent` API to perform direct saving of user consent upon user consent grant.

- If the existing implementation does not have direct saving handling, and still relies on OAuth /auth endpoint for auto saving:
-

-
- Set Enable auto saving of user consent after OAuth Authentication setting of its OAuth Client configuration in Admin Console to "True" for backward compatibility.
 - If the existing implementation does have direct saving handling:
 - Check if the custom implementation has implemented the OAuth Proxy CSRF consent token mechanism properly. If the implementation is in place, specify "true" as the OAuthCsrfPrevention parameter value in OAuth Proxy deployment's **web.xml** file to enforce the corresponding CSRF mechanism.

Considering CSRF Security Implementation

Cross-Site Request Forgery (CSRF) is an attack where a web browser is tricked into executing an unwanted action in an application that a user is logged in to. The attacker causes the victim to perform actions unintentionally, and can potentially gain control over accounts or sensitive data. Preventing CSRF attacks involves implementing security measures such as anti-CSRF tokens.

In a standard deployment where AccessMatrix Common Login Page (CLP) is used as both the user-facing login page and consent page, the necessary CSRF prevention measures are already in place and no further action is required.

It is worth considering if your existing implementation or your new implementation should tap on the security mechanism of OAuth Proxy on CSRF handling for the customized consent page.

i **Note:** Before reading this section, you should understand the mechanism behind OAuth Proxy's CSRF implementation. You may refer to [Customization Handling](#) to understand how CSRF handling is done for consent requests.

The following table will help you to evaluate if OAuth Proxy's CSRF mechanism is required for your existing or new implementation:

OAuth Proxy's CSRF Mechanism	Scenarios	Configuration
Not in use	When SSO is not allowed and the customized login /consent page is used for security sensitive logins. The customized login/ consent page will not need to redirect the browser	Disable OAuth Proxy CSRF validation setting in the OAuth Proxy.

OAuth Proxy's CSRF Mechanism	Scenarios	Configuration
	back to the OAuth Proxy to evaluate if user consent is required. Note: This scenario is not applicable if the consent page of CLP is used for consent handling.	
In use	For existing implementations that always redirect the browser to the OAuth Proxy to evaluate if user consent is required. Note: This is the default behavior of CLP where its consent page will be used for consent handling.	Enable OAuth Proxy CSRF validation setting in the OAuth Proxy.
Partially in use	When SSO is allowed and the implementation will leverage the OAuth Proxy for consent checking. In such cases, the CSRF mechanism on the OAuth Proxy is needed. In a scenario where the user is redirected to the customized login/ consent page for user authentication and followed by consent prompt, the OAuth Proxy CSRF mechanism will be skipped. Note: This scenario is not applicable if the consent page of CLP is used for consent handling.	Enable OAuth Proxy CSRF validation setting in the OAuth Proxy.

For more details, refer to the illustrations below:

1. OAuth Proxy's CSRF mechanism not in use. This decision could be due to the client application being security sensitive (having short OP session idle timeout) or does not allow SSO (users always have to re-login when triggering the client application). After user authentication, the customized login / consent page will proceed to check user consent and therefore it is not necessary to go back to the OAuth Proxy for consent checking handling - to get the CSRF consent token triggered then only redirected back to the custom consent page.



Note: This option is not applicable to customers who are using the consent page of AccessMatrix Common Login Page for consent handling.

- You will need to disable the OAuth Proxy CSRF validation setting in the OAuth Proxy.

-
2. OAuth Proxy's CSRF mechanism is going to be in use. This could be due to the existing implementation always redirects the browser back to OAuth Proxy after user authentication is done in the custom login page. Possibly also because SSO is allowed and so when a user's OP session exists and is still valid, a CSRF consent token would be triggered.

i Note: This is the default behavior of the consent flow of AccessMatrix Common Login Page where its consent page is used for consent handling.

- You will need to enable the OAuth Proxy CSRF validation setting in the OAuth Proxy.

3. OAuth Proxy's CSRF mechanism is going to be partially in use, such as when SSO is allowed.

i Note: This option is not applicable if the consent page of the AccessMatrix Common Login Page is used for consent handling.

- When SSO happens, it leverages on the OAuth proxy CSRF mechanism.
- But if the user has yet to be authenticated, it will leverage on the customized login/ consent page to evaluate if user consent is required.

i Note: You may temporarily turn off the CSRF validation of the OAuth Proxy while waiting for the custom consent page to be enhanced. This setting must be turned on after the enhancement has been completed in order to activate the CSRF validation.

Consent Token Validation

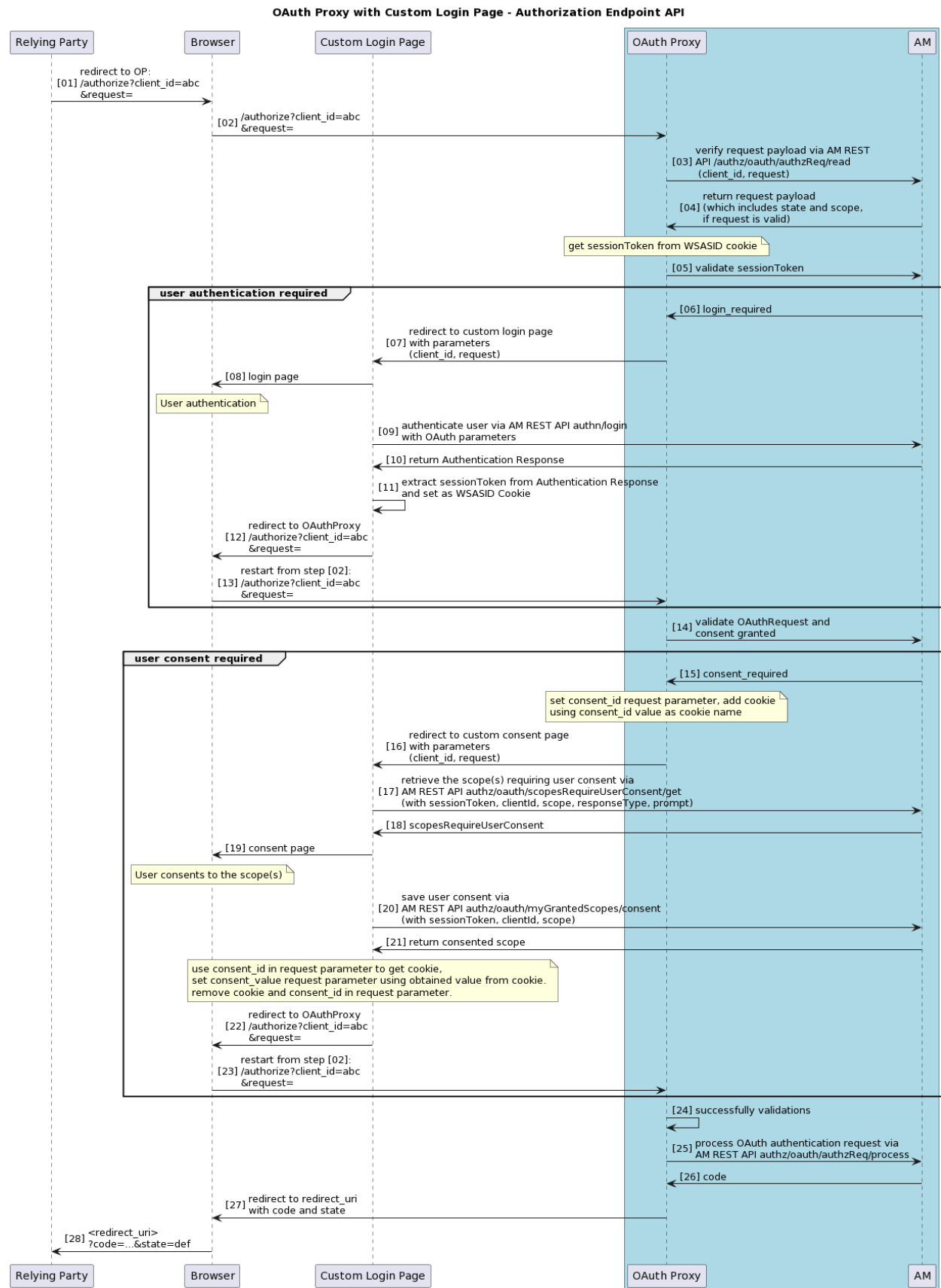
OAuth Proxy will have a CSRF token created if during its evaluation the user is already authenticated and consent is required. This behavior would occur regardless whether it's CSRF validation setting is enabled or not. However, only when the setting is enabled, the CSRF validation would be kicked in to complete the security mechanism.

i Note: To configure this setting, refer to [Configure OAuth Proxy CSRF Validation](#).

Implementing Consent Tokens

If your implementation involves an OAuth Proxy's CSRF mechanism that is in use or partially in use, the custom consent page handling would need to adhere to certain OAuth Proxy CSRF protocol in order for the CSRF security mechanism to work properly.

Below is a sample sequence diagram where OAuth Proxy's CSRF mechanism is going to be in use. The custom consent page must perform the steps below to enable CSRF prevention. The steps make reference to the following sequence diagram:



- At step [16] of the flow, the redirected OAuth request from the OAuth Proxy will contain the parameter `consent_id`.

-
2. At step [21] of the flow, after saving the user consent:
 - a. Use the value of `consent_id` from the request parameter to get a cookie that has the `consent_id`'s value as its name. Keep the cookie value to be used later.
 - b. Delete the cookie.
 - c. Prepare the OAuth Proxy `/auth` redirect URL with the existing OAuth request intact. This is followed by removing the `consent_id` request parameter in redirect URL.
 - d. Given the OAuth Proxy `/auth` redirect URL, trigger browser redirect with a FORM POST request where `consent_value` request parameter is set as the input name and the cookie value as its input value.

Configure OAuth Proxy CSRF Validation

You should make the following configurations in the appropriate **web.xml** file:

1. Navigate to `<CLP-DIR>\WEB-INF\` or `<OAuth_Proxy_DIR>\WEB-INF\` (for CLP or the OAuth Proxy respectively).
2. Open the **web.xml** file and search for the the `OAuthCsrfPrevention` param-name:

XML

```
<context-param>
  <param-name>OAuthCsrfPrevention</param-name>
  <param-value>>false</param-value>
  <!-- Set 'true' to enable CSRF security validation
for OAuth consent; typically when CLP's consent page is
used.
Set 'false' to skip CSRF security validation (not
recommended due security concerns); when used with
custom consent page that does not handle CSRF.
Refer to 'Customized Consent Page' in the 'UAM OAuth
and OIDC OpenID Provider Implementation Guide' for
details. -->
</context-param>
```

3. Ensure that the param-value is set to "true" to enable the validation. Otherwise set to false to skip the validation:

XML

```
<context-param>
  <param-name>OAuthCsrfPrevention</param-name>
  <param-value>true</param-value>
  <!-- Set 'true' to enable CSRF security validation
```

```
for OAuth consent; typically when CLP's consent page is
used.
Set 'false' to skip CSRF security validation (not
recommended due security concerns); when used with
custom consent page that does not handle CSRF.
Refer to 'Customized Consent Page' in the 'UAM OAuth
and OIDC OpenID Provider Implementation Guide' for
details. -->
</context-param>
```

4. Save the changes.

4.4. Login Page

The authentication front end can either be implemented using the AccessMatrix Common Login Page (CLP) or a custom login page.

Standard Login Page

AccessMatrix Common Login Page (CLP) is a front-end login page that integrates with the AccessMatrix server to provide client applications with a comprehensive authentication service. There are several basic operations that can be done in CLP:

- Authentication of users
- Multi-factor Authentication (MFA)
- Reset and change passwords

CLP is bundled with AccessMatrix OAuth Proxy to serve as an integrated authentication component and OIDC endpoints for your OAuth client application.



Note: For more details, refer to the standard deployment diagram in [UAM OAuth and OIDC Integration](#).

It is important to consider the authentication realms to be used for user authentication. The authenticated user session obtained will be set as a session cookie.



Note: For more details on how to manage OP session and cookie settings, refer to [OP Session Management](#).

CLP Deployment and Configurations

To configure:

1. Retrieve **AccessMatrix-`<version>`-clp.war** from AccessMatrix product build (applicable for 5.6.3-GA version and above).
 2. Extract the WAR file to the intended CLP installation directory (i.e. `<CLP_DIR>`) and make changes on the CLP and OAuth Proxy configuration. By default, OAuth proxy component is bundled with the **clp.war** file.
 3. Navigate to `<CLP-DIR>\WEB-INF\classes` and open the **am.commonlogin.config.xml** file.
-

-
4. Look for the <SessionCookie> key word and specify cookie settings according to OP Session Management section below.
 - a. For best security, configure cookie Path with CLP context path. Unless CLP and SSO portal (with federated (e.g. SAML) SSO app listing) are deployed under the same domain and it is intended for the CLP deployment to leverage on SSO portal cookie to achieve SSO for the federated applications listed in SSO Portal.
 5. Configure CLP's general authentication settings such as authentication realm configuration, UI customization, etc. by referring to the [AccessMatrix CLP Deployment Guide](#).
 6. Save the changes to this XML file.
 7. Navigate to <CLP-DIR>\WEB-INF\ and open **web.xml** file.
 8. Look for restAPIURL param-name and modify the param-value to connect to the AccessMatrix server REST API.
 9. Save the changes to this XML file.

OP Session Management

OP Session Cookie Configurations (standard)

The login page is required to set the WSASID cookie according to the following attributes:

Cookie Attribute	Value
Cookie name	WSASID
HTTP-Only	true
Secure	true
max-age	-1 Note: This attribute is not configurable for OAuth Proxy or AccessMatrix CLP.
SameSite	Strict

The following illustrates a sample cookie setting of the WSASID cookie in HTTP:

```
HTML
```

```
Set-Cookie: <cookie-name>=<WSASID>; Expires=Thu, 21 Oct
2021 07:28:00 GMT; Secure; HttpOnly;SameSite=Strict;
Path=/docs; Domain=OIDCOP.org
```

Custom Login Page

Prior to the customization planning, you may want to refer to [UAM OAuth and OIDC Integration](#) to understand the customization deployment diagram. You will also need to consider OP session management. For more details on how to manage OP sessions and cookie settings, refer to [OP Session Management](#).

This section introduces several REST APIs for implementing a customized login page depending on your use case:

i **Note:** For more details regarding these APIs, refer to the [AccessMatrix REST API](#).

- [User Authentication UI Customization](#)
- [Advanced Customization](#)

User Authentication UI Customization

The following API allows for customizing the User Authentication UI.

authn/login

This API is used to perform user authentication with AccessMatrix Universal Authentication Server (UAS). If the authentication is successful, it returns a session token that must be set as a WSASID cookie.

This API should be called if the customization involves the login page.

i **Note:** For more details on how to configure the Login Page Redirect URL, refer to [Configure Login and Consent Page Redirect URL](#).

Advanced Customization

The following APIs allow for advanced customization.

authz/oauth/authzReq/read

This API is used to perform basic OIDC request validation. It validates and reads OAuth authentication requests. Redirect URIs, OAuth clients, as well as the integrity and authenticity of the request object (if in use) will be validated.

It returns all information, including parameters residing in the request object. This includes information for display purposes, such as the name of the OAuth Client.

This API should be called if:

- The customization involves the OAuth /auth endpoint.
- The OAuth Proxy of OAuth /auth endpoint is in use under any of the following scenarios:

 **Note:** This includes AccessMatrix CLP.

- When the OAuth request is redirected to the login page via the internet.

 **Note:** This is not recommended due to security and performance implications.

- When used alongside request object implementation.

authz/oauth/authzReq/process

This API is used to perform OIDC authentication processing when a user session token is passed. It starts the OAuth authentication flow, where the authorization code is obtained and subsequently returned to the redirect URL.

This API should be called if the customization involves the OAuth /auth endpoint.

Implementing Web-based Browser Redirection

Once user authentication has been completed, the AM5 session token obtained from AM5 /authn/login REST API must be set as a HTTP-Only Secure cookie with other secure cookie settings.

 **Note:** For deployments with OAuth Proxy (or AccessMatrix CLP), they must share the same domain and cookie settings to facilitate cookie reading.

The cookie path and cookie domain should match the configuration in the OAuth Proxy. Otherwise, you will need to ensure that it is still accessible by the OAuth proxy.

For more details on how to configure the OAuth Proxy, refer to [Configure Session Token Cookie Domain and Path](#).

Implementing Native Mobile Applications

It is important to ensure secure storage of the AM5 session and tokens (e.g. Access Token) using platform-specific secure storage solutions. Avoid storing sensitive data in Shared Preferences.

OP Session Timeout Configurations in Admin Console

The OP session is subject to idle timeout. However, the session token's idle time and expiry can be configured using the Admin Console.



Note: This configuration is applicable to both custom and standard implementations of the login page.

Configure Session Token Idle Time

Session Idle Time can be defined either in realm-based session policy or in **Global Setting**.

To set the **Session Idle Time** under a realm-based session policy:

1. Navigate to **Administration Dashboard > Security Policy > Authentication > Realm-Based Session Policy**.
 2. Click the **Create** button to create a new realm-based session policy.
 3. Provide the values for **Realm-Based Session Policy Id**, **Realm-Based Session Policy Name**, **Session Idle Time (in seconds)** and **Session Lifetime (in seconds)** attributes.
 4. Click the **Save** button to save the changes.
 5. Navigate to **Configuration > Authentication > Authentication Realm**.
 6. Click on the relevant authentication realm under the **Authentication Realm Id** column.
 7. Click the **Edit** button.
 8. Under the **Realm-Based Session Policy** attribute, assign the policy created in the previous steps.
-

-
9. Click the **Save** button to save the changes.

To set the **Session Idle Time** in **Global Setting**:

1. Navigate to **Administration Dashboard > Configuration > Global Setting**.
 2. Click the **Edit** button and select the **Session** tab.
 3. Specify a value in **Session Idle Time (in seconds)**.
 4. Click the **Save** button to save the changes.
-

4.5. OIDC Tokens

Prior to the implementation of the intended OAuth flow for your OAuth client application, it is important to understand the use of the various tokens and how they work.

Access Token

The access token is a credential used to access protected resources. It is a string representing an authorization that is issued to the client.

To manage how user info is returned with the access token, refer to [Retrieving User Info](#).

Introspect Access Token

OAuth 2.0 defines a protocol that returns information about an access token. The `/introspect` endpoint allows server-side OAuth client to get the status and information about the access token. Refer to the [/introspect](#) section for additional information.

APIs Secured by Access Token

/userinfo OIDC endpoint

- This OIDC endpoint retrieves user info.
- Returns any claims for the currently logged-in user.

Examples:

Request Example	
You must include an access token (in the Bearer header or in POST with Bearer body). Example of using Bearer header:	
Code	<pre>Authorization: Bearer \${access_token} "https://{oauthproxyURL}/userinfo"</pre>
Response Example (Success)	
Returns a JSON document with claims about the currently authenticated end user.	

Request Example
JSON <pre>{ "phone_number": "+1 (310) 123-4567", "phone_number_verified": false, "sub": "demouser" }</pre>

Refresh Access Token

Using /token OIDC endpoint

- This OIDC endpoint returns access token and refresh token.
- The call to the /token endpoint requires the use of the **HTTP POST** method and the Form Serialization. It also requires authentication. Authentication can be done via, basic authentication of the client ID and secret, or private key JWT. You can find the client ID and secret under your OAuth Client basic information section.

Request Parameters:

Parameter	Data Type	Description
client_id	String	Required if client has a secret and client credentials are not provided in Authorization header.
client_secret	String	Required if client has a secret and client credentials are not provided in Authorization header.
client_assertion	String	Client-generated JWT signed with an asymmetric key to authenticate the client. Refer to the OpenID Connect specification for the claim values contained in the JWT. This parameter is required for the private_key_jwt authentication method.
client_assertion_type	String	An absolute URI that defines the format of the assertion. The value must be "urn:ietf:params:oauth:client-assertion-type:jwt-bearer".

Parameter	Data Type	Description
		This parameter is required for the private_key_jwt authentication method.
grant_type	String	Specify "refresh_token" to indicate it's for refresh token use.
refresh_token	String	This value refers to a valid refresh token that was returned from this endpoint previously.

Response Results:

Property	Data Type	Description
access_token	String	A refreshed access token.
token_type	String	"Bearer" would be returned.
refresh_token	String	A refreshed refresh token.
id_token	String	A refreshed ID token.
error	String	The error returned, if not successful.
error_description	String	The error description, if not successful.

Possible Errors:

Error	Possible Causes
login_required	The AM5 session token embedded within the refresh token was not successfully refreshed. This occurs because the embedded session is invalid, or because the embedded session had already been replaced due to session concurrency control settings.

Using REST API

- Use the authz/oauth/refreshTokenReq/process API to create custom OIDC token endpoint.
- Refer to the [AccessMatrix REST API Specification](#) for details of this API.



Note: To make your endpoint conform to the OAuth 2.0 token endpoint standard for the refresh token grant, refer to http://openid.net/specs/openid-connect-core-1_0.html#RefreshTokens.

Revoke Access Token

Using /revoke endpoint

- This OIDC endpoint revokes access token or refresh token.

Request Parameters:

Parameter	Data Type	Description
client_id	String	Required if client credentials, e.g. client id and client secret are not provided in Authorization header.
client_secret	String	Required if client's token authentication endpoint is set to client secret and the client credentials, i.e. client Id and client secret are not provided in the authorization header.
client_assertion	String	Client-generated JWT signed with an asymmetric key to authenticate the client. Refer to the OpenID Connect specification for the claim values contained in the JWT. This parameter is required for the private_key_jwt authentication method.
client_assertion_type	String	An absolute URI that defines the format of the assertion. The value must be "urn:ietf:params:oauth:client-assertion-type:jwt-bearer". This parameter is required for the private_key_jwt authentication method.
token_type_hint	String	Specify "access_token" to indicate that the token to be revoked is an access token. This parameter is optional.
token	String	Refers to the token to be revoked. It could be an access token or refresh token.

Response Results:

Property	Data Type	Description
error	String	The error returned, if not successful.
error_description	String	The error description, if not successful.

Using REST API

- Use authz/oauth/revokeTokenReq/process API to create a custom /revoke endpoint.
- Refer to the [AccessMatrix REST API Specification](#) for details of these APIs.

Using Admin Console

When users perform OAuth or OIDC flow, their OAuth tokens (e.g. refresh token or access token) would be persisted. To invalidate or revoke user's OAuth tokens:

1. Navigate to **Operation Dashboard > User & Segment > User**.
2. Search for user on the **Search User** page based on the available search criteria.
3. When you select the intended user based on the **Search User Result**, then the **View User** page will be displayed. Look for the **OAuth Session Persistence** tab, and you will see listing of active OAuth token, if available. Select the token(s) to be revoked, then click **Invalidate**.

ID Token

OpenID Connect introduces an ID token which is a JSON web token (JWT) that contains information about an authentication event and claims about the authenticated user that clients can rely on. The ID token (id_token) consists of three period-separated, base64URL-encoded JSON segments: a header, the payload, and the signature.

To manage how user info is returned with the ID token, refer to [Retrieving User Info](#).

Example:

ID Token Header	
JSON	
{ "alg": "RS256",	

ID Token Header
<pre>{ "kid": "45js03w0djwedsw" }</pre>
ID Token Payload
JSON <pre>{ "ver": 1, "sub": "00uid4BxXw6I6TV4m0g3", "iss": "https:{myAxMxUAMDomain.com}", "aud": "uAaunofWkaDJxukCFeBx", "iat": 1449624026, "exp": 1449627626, "amr": ["pwd"], "jti": "ID.4eAWJOCMB3SX8XewDfVR", "auth_time": 1449624026, "at_hash": "cpqKfdQA5eH891Ff5oJr_Q", "name": "John Doe", "nickname": "Jimmy", "preferred_username": "john.doe@example.com", "given_name": "John", "middle_name": "James", "family_name": "Doe", "profile": "https://profile.wordpress.com/john.doe", "zoneinfo": "America/Los_Angeles", "locale": "en-US", "updated_at": 1311280970, "email": "john.doe@example.com", "email_verified": true, "address": { "street_address": "123 Hollywood Blvd.", "locality": "Los Angeles", "region": "CA", "postal_code": "90210", "country": "US" }, "phone_number": "+1 (425) 555-1212" }</pre>
ID Token Signature
This is the digital signature that AccessMatrix UAM signed using the public key identified by the kid property in the header section.

Obtain an ID Token

Clients can use any of the following sequence of operations to obtain an ID token:

- **Authorization Code with PKCE flow, Hybrid with PKCE flow or Authorization Code flow**

The client obtains an authorization code from the authorization server's authorization endpoint and uses this to obtain an ID token and access token from the authorization server's token endpoint.

- **Implicit flow**

The client obtains the ID token and optionally, the access token directly from the authorization server's authorization endpoint. Clients should always validate ID tokens to ensure their integrity.



Note: The ID tokens returned by the authorization endpoint (Implicit flow) or the token endpoint (Authorization Code flow) are identical, except for some key differences in the implicit flow:

- The nonce parameter is required and must be included in the request.
- If the response includes an access token, the at_hash claim is required.
- If the response does not include an access token, the at_hash claim is not required.

Validate the ID Token

To validate the ID token, refer to [C.Validating JWTs](#).

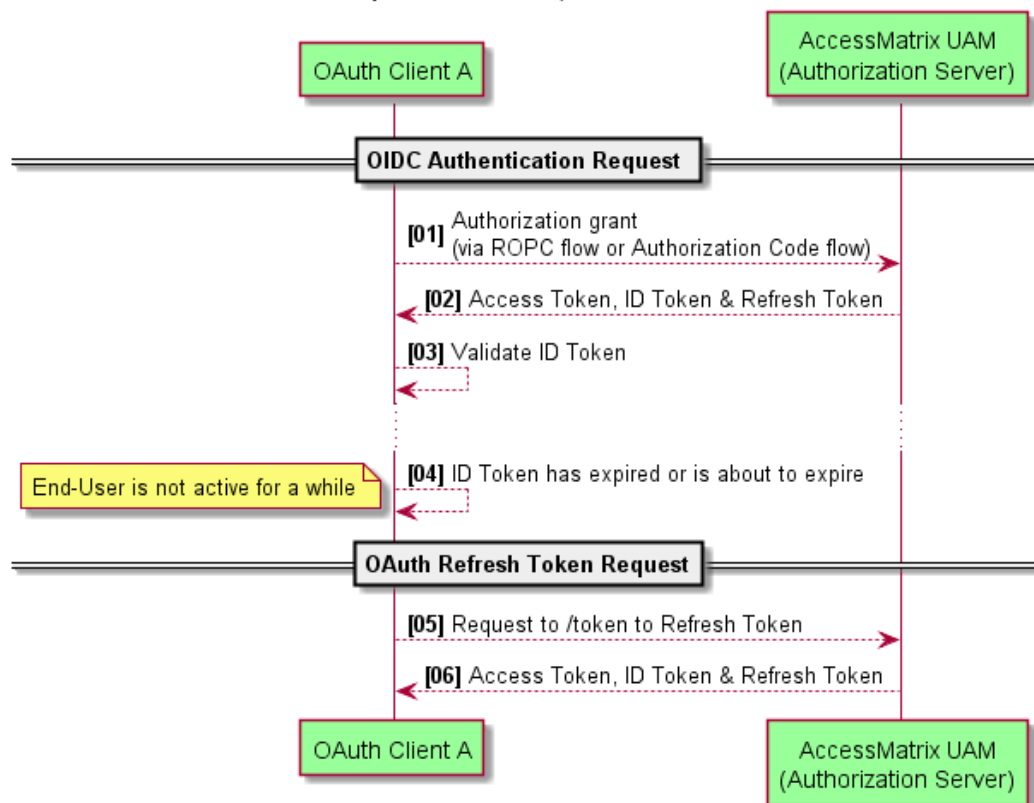
Refresh Token

Refresh token is a credential used to obtain access tokens. It is a string representing the authorization granted to the client by the resource owner.

Use Case

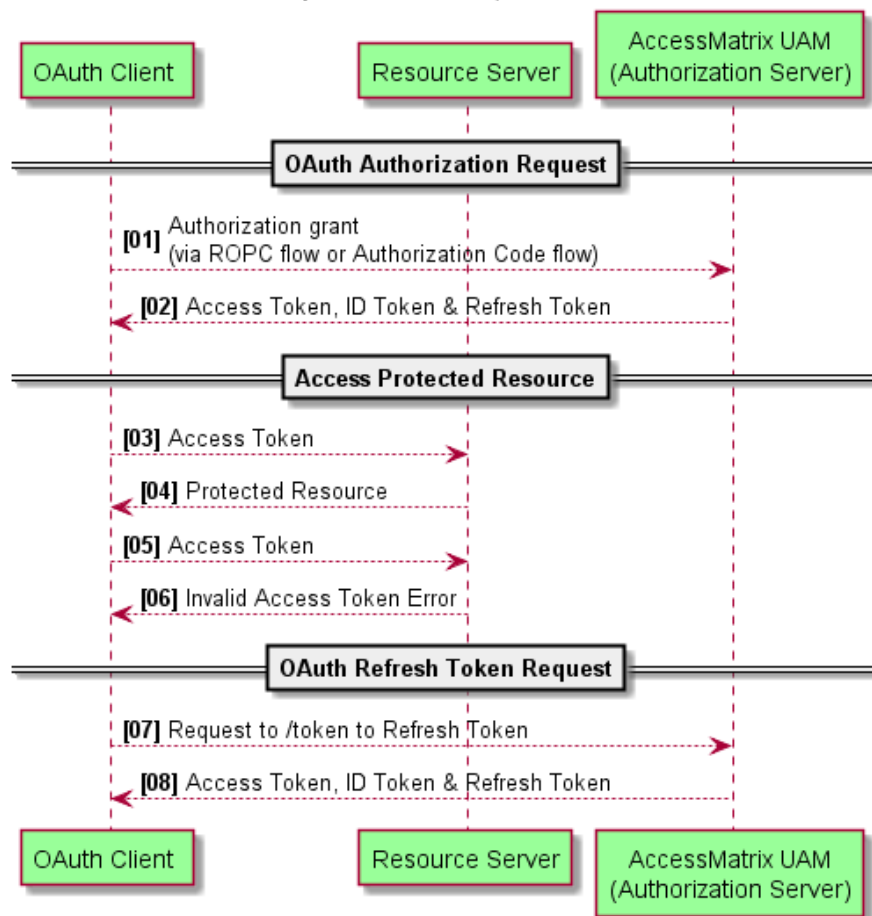
- Refresh token before/after ID token is expired
-

AccessMatrix Authorization Server: OpenID Connect (Refresh token before/after ID Token is expired)



- Refresh token after access token is expired

AccessMatrix Authorization Server: OpenID Connect (Refresh token after access token is expired)



Return a Refresh Token

Refresh tokens are available for certain grants:

- Authorization Code with PKCE flow
- Hybrid with PKCE flow
- Authorization Code flow
- Hybrid flow
- Resource Owner Password Credentials flow

To enable returning of refresh tokens:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
 2. Click the corresponding OAuth client from the search result.
 3. Click the **Edit** button.
-

-
4. Set the **Return Refresh Token** attribute to "true".
 5. Click **Save** to save the changes.

Obtain a Refresh Token

i **Note:** Once OAuth client configuration is set to return refresh token for the corresponding flow, a new refresh token will also be returned by default when the refresh token process is triggered.

- **Authorization Code with PKCE Flow / Hybrid with PKCE Flow / Authorization Code Flow / Hybrid Flow**

To get a refresh token using this flow, send a request to your authorization endpoint (/auth), then use the authorization server's /authorize endpoint to get an authorization code. Send this code to the /token endpoint to get an access token and a refresh token. Refer to [Admin Console Client Registration](#) at the respective flow for more information.

- **Resource Owner Password Credentials Flow**

For this flow, use the authorization server's /token endpoint directly. Refer to [Admin Console Client Registration - ROPC Flow](#) for more information.

Use a Refresh Token

To refresh your access token, send a token request with a grant_type of "refresh_token".

The call to the /token endpoint needs to use the **HTTP POST** method and Form Serialization. It also requires authentication. Authentication can be done via basic authentication of the client ID and secret or via the private key JWT authentication method. You can find the client ID and secret under the **OAuth Client** page's basic information section.

Request Example

Code

```
POST https://{yourOAuthProxy}/oauth-proxy/token
accept:application/json
authorization:'Basic MG9hYmg3M...'
cache-control:no-cache
content-type:application/x-www-form-urlencoded
grant_type=refresh_token
redirect_uri=http://localhost:8080
scope=openid
refresh_token=MIOF-U1zQbyfa3MUfJHhvnUqIut9ClH0xjlDXGJAYqo
```

If the refresh token is valid, then you get back a new access/refresh token combination:

Response Example (Success)	
JSON	<pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ", "token_type": "Bearer", "expires_in": 3600, "scope": "openid", "refresh_token": "MIOf-U1zQbyfa3MUfJHhvnUqIut9ClH0xjlDXGJAYqo" }</pre>

Refresh Token Expiry and Lifetime Behavior

Refresh token expiry refers to the validity period of a refresh token before it must be renewed. A valid and active refresh token is only meant for one-time use and must be renewed after being used for a token refresh, even if it has yet to pass the expiry period. The refresh token lifetime is the maximum validity period within which refresh tokens can continue to be renewed. The starting time of a refresh token's validity is referred to as the grant time. The refresh token lifetime usually starts from the grant time of the first refresh token unless the **Use Refresh Token Integration Date** attribute is enabled.

To configure the behaviour of the refresh token expiry and lifetime, the following attributes can be specified in the OAuth client configuration:

- **Refresh Token Expiry (in seconds)** - this attribute determines the duration of a refresh token's validity.
 - **Refresh Token Lifetime (in hours)** - this attribute determines the duration of the refresh token's lifetime.
 - **Use Refresh Token Integration Date*** - when set to "true", the integrationDate request parameter passed to the token endpoint will be used as the grant time of the refresh token (where grant time refers to the starting time of a refresh token's validity). The refresh token expiry will also be made the same as the refresh token lifetime, ignoring the value specified in the **Refresh Token Expiry** attribute. When set to "false", the generation date of the first refresh token will be used as the grant time of the refresh token and this will be used to compute the refresh token lifetime. In this case, the refresh token expiry will always start from the generation
-

date of the refresh token - provided it is still within the refresh token lifetime - and the expiry time will be computed based on the **Refresh Token Expiry** attribute.



Note: The **Use Refresh Token Integration Date** attribute is only applicable to OAuth client configuration for the Authorization Code flow & Hybrid flow for a Server-side (Web) Application. This is only intended to be used for SGFinDex.

Revoke Refresh Token

Revocation of refresh token will invalidate the related access tokens and the underlying authorization grant. It can also revoke all the consented scopes of the user for the corresponding OAuth client.

The behavior of revoking refresh token can be configured in the **OAuth Client** configuration page:

- **Revoke Token Upon User Session Invalidation** - setting this attribute to "true" will revoke all OIDC tokens linked to the invalidated user session.
- **Invalidate User Session Upon Token Revocation** - setting this attribute to "true" will invalidate the user session of the revoked refresh token.
- **Revoke Consented Scopes When Revoking Token** - setting this attribute to "Revoke All" will remove all the consented scopes of the user.

To revoke the refresh token, refer to [Revoke Access Token](#) for more details.

Token Format, Cryptography and Life Cycle Management

Configure ID Token Format

ID Token is configured to be in JWT format by default. However, it can also be configured to support JWE format.

Perform the following steps to configure the ID Token to be in JWE format:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
 2. Click on the relevant OAuth client under the **OAuth Client Id** column.
 3. Click the **Edit** button and select the **OpenID Connect** tab.
 4. Specify the **Return ID Token in JWE Format** attribute as "true".
-

Note: By default, this attribute is set to "false".

5. Click the **Save** button to save the changes.

Configure Access Token Format

Access Token is configured to be in Opaque format by default. However, it can also be configured to support JWE or JWT format.

Perform the following steps to configure the Access Token to be in either JWE or JWT formats:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click the **Edit** button.
3. Specify the **Access Token Format** attribute as JWT or JWE.

Note: By default, this attribute is set to "Opaque".

4. Click the **Save** button to save the changes.

Configure OIDC Token(s) Signing Key

OIDC tokens are typically generated as JWT with digital signature. The signing key for ID token (or Access Token if applicable) can be system-generated or configured in key store.

Note: It is highly recommended to use a system-generated signing key instead of a key store signing key. The Key store signing key does not support key rotation, and is hence meant for legacy applications.

Perform the following steps to configure system generate signing key:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Authz Server**.
 2. Click the **Edit** button.
 3. Fill in the following attributes under the **System generated key pair for signing in ID Token (JWT)** section:
 - **RSA key length** - Options: "2048", "4096"
Default: 2048
-

-
- **Wrapping Key** - Wrapping key is required to generate OpenID Connect signing key pair. This key is used to protect the system-generated signing key pair.
 - **No. of public keys to remember for verification** - Value should not be less than 2. Default: 2

4. Click the **Save** button to save the changes.



Notes:

- To generate the first signing key, click **Generate OpenID Connect Signing Key Pair** button. The **Key generation for ID Token (JWT) signing** dialog will be displayed. Click **OK** to confirm the generation of new key.
- To rotate signing key, use the system-generated signing key pair instead of key store signing key pair. Click **Generate OpenID Connect Signing Key Pair** button.

Configure OIDC Token(s) Encryption Key

Encryption key for ID token (or Access Token if applicable) can be specified via **JWKS String** or **JWK URL**.

Perform the following steps to configure system generate signing key:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client** Id column.
3. Click the **Edit** button and fill in the following attributes under the **Attribute** tab:
 - **JWKS Endpoint URL** - JWKS URL published by OAuth Client to allow the retrieval of public key to encrypt ID token (or Access Token if applicable)
Default: 2048
 - **JWKS String** - JWK in JSON format
4. Fill in the following attributes to configure JWE related algorithms:
 - a. **JWE Algorithm** - This attribute configures cryptographic algorithm used to encrypt or determine the value of the CEK(Content Encryption Key). Options: "RSA-OAEP-26", "ECDH-ES+256KW".
 - i. Recommended value: **ECDH-ES+256KW**

-
- b. **JWE Encryption Algorithm** - Content encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Options: "A256CBC-HS512", "A256GCM".

- i. Recommended value: **A256GCM**

- 5. Click the **Save** button to save the changes.

Configure Token Expiry or Validity

Each issued token has its own expiry or validity period. To change the expiry time of access token or refresh token (in seconds):

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Modify the values of the following attributes:
 - **Access Token Expiry (in seconds)**
 - **Refresh Token Expiry (in minutes)**
 - **Refresh Token Lifetime (in hours)**
5. Click the **Save** button to save the changes.

To change the expiry time of ID tokens:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth Authz Server**.
 2. Click the **Edit** button.
 3. Modify the value of the **ID Token (JWT) validity time (in minutes)** attribute.
 4. Click the **Save** button to save the changes.
-

4.6. Cross-Origin Resource Sharing (CORS) for OIDC Endpoints

Cross-Origin Resource Sharing (CORS) is a protocol that enables scripts of HTTP requests running on a browser client to interact with resources across the domain. For security reasons, the browser client can only make calls to URLs of the same origin by default. However, there are legitimate scenarios to enable cross-origin access. For instance, to allow CORS with OIDC endpoints to support OIDC authentication and authorization.

To support CORS with OIDC endpoints, the whitelisted web origin(s) of the OAuth client(s) must be configured to allow cross-origin resource sharing with the endpoint(s). If the browser client that initiated a request to an OIDC endpoint is whitelisted, a CORS response with the Access-Control-Allow-Origin header would be included and returned to the corresponding client to facilitate cross-origin resource sharing.

CORS for OIDC Endpoints

CORS is automatically enabled for all web origins for these endpoints:

- JWK endpoint (/jwks.json)



Note: It is still possible to enable CORS for this endpoint via Tomcat CORS filter. For more details on how to configure this, please refer to [Enable CORS via web.xml File](#).

- OAuth authorization server's metadata endpoint (/well-known/openid-configuration)

To enable CORS for the endpoints below, the administrators can add or remove allowed origin as needed for its OAuth client via the [Admin Console](#):

- OAuth authorize endpoint (/auth)



Note: This endpoint can also be used for silent authentication.

- OAuth token endpoint (/token)
 - OAuth token revocation endpoint (/revoke)
 - OAuth token introspection endpoint (/introspect)
 - OAuth user info endpoint (/userinfo)
-

In CLP (or the OAuth Proxy), the bundled CORS filter collects the web origins configured in each OAuth client configuration to form the entire set of rules for CORS by adding them to the list of whitelisted origins.

Enable CORS via Admin Console

Enable CORS for OIDC endpoints by performing the following steps:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Specify the **JavaScript Web Origin URIs** attribute by performing the following steps:
 - a. Enter the origin of the browser client to be whitelisted.
 - b. Click the **Add** button.

 **Note:** Web origin checking is case-insensitive.

5. Click the **Save** button to complete the configuration.

 **Note:** Any changes you make to this configuration should take effect immediately or at most, with a latency of 3 to 5 minutes without restarting CLP (or OAuth Proxy).

Enable CORS via web.xml File

Alternatively, CORS support from tomcat can be used instead by configuring the **web.xml** file. To do so:

1. Navigate to `<CLP-DIR>\WEB-INF\` or `<OAuth_Proxy_DIR>\WEB-INF\` (for CLP or the OAuth Proxy respectively).
2. Open the **web.xml** file and search for the the CorsFilter filter-name to locate the sample configuration included in the **web.xml** file:

XML

```
<!-- /jwks.json -->
<!--<filter>
  <filter-name>CorsFilter</filter-name>
  <filter-
class>org.apache.catalina.filters.CorsFilter</filter-
```

```

class>
  <init-param>
    <param-name>cors.allowed.origins</param-name>
    <param-value></param-value>
  </init-param>
  <init-param>
    <param-name>cors.allowed.methods</param-name>
    <param-value>GET, POST, OPTIONS</param-value>
  </init-param>
  <init-param>
    <param-name>cors.supportedHeaders</param-name>
    <param-value>Accept, Origin</param-value>
  </init-param>
  <init-param>
    <param-name>cors.preflight.maxage</param-name>
    <param-value>1800</param-value>
  </init-param>
</filter>
<filter-mapping>
  <filter-name>CorsFilter</filter-name>
  <url-pattern>/jwks.json</url-pattern>
</filter-mapping>-->

```

3. Uncomment this code block and/or create a new code block using this format for each endpoint as necessary.
4. Manually whitelist the endorsed web origins of all of the corresponding OAuth clients by adding them to the param-value of cors.allowed.origins.
5. Comment out all the bundled CORS filter settings in the **web.xml** file. This refers to all filters with com.isprint.oauth.proxy.filter.cors.CorsFilter as their filter-class.
6. For the JWK endpoint (/jwks.json), the existing handling of the bundled CORS filter must be disabled as well. To do so, search for the OAuthJWKJsonServlet servlet-name and make the following configurations:

XML

```

<servlet>
  <servlet-name>OAuthJWKJsonServlet</servlet-name>
  <servlet-
class>com.isprint.oauth.proxy.OAuthJWKJsonServlet</servlet
-class>
  <init-param>
    <param-name>AccessControlAllowAllOrigins</param-name>
    <param-value>false</param-value>
    <!-- This setting is related to CORS (Cross-Origin
Resource Sharing). If set to "true", all web origins
would be allowed to access this endpoint. Please set it

```

```
to "false" if you intend to use tomcat's CORsFilter to  
whitelist your intended web origins instead . -->  
  </init-param>  
</servlet>
```

7. Save the changes.

4.7. Identity Mapping

Identity Mapping allows the user identity of the OAuth client (in the form of an ID token) to be linked to the authenticated user in AccessMatrix (acting as the OP).

The ID token is a JSON web token (JWT) that contains information about an authentication event and claims about the authenticated user that clients can rely on. Within this token, the sub claim serves as a unique identifier for the user, which the OAuth client application uses to recognize the user.



Note: The sub claim refers to subject claim, which is typically linked to the user's login ID.

In AM, this claim can be configured using the ***Value to return as Subject (sub) value** attribute in the OAuth client. There are 3 main ways to configure this claim:

- [Identity Mapping via User Store Field/Attributes](#)
- [Identity Mapping via User Response](#)
- [Identity Mapping via Secondary Account](#)

Identity Mapping via User Store Field/Attributes

The following user fields/ attributes are typically used for retrieval of external user identity for identity mapping:

- Id
- DN
- Email
- Name

Using the Admin Console:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
 2. Click on the relevant OAuth client under the **OAuth Client Id** column.
 3. Click the **Edit** button and select the **OpenID Connect** tab.
 4. Select a user field / attribute in the ***Value to return as Subject (sub) value** attribute.
 5. Click the **Save** button to save the changes.
-

Identity Mapping via User Response

If the user identity cannot be obtained based on the existing common attributes, it can be retrieved from existing fields or attributes defined in the User Response. Alternatively, a custom plugin of User Response can be used to retrieve the user identity.



Note: For more information on User Response or how it can be configured, refer to [AccessMatrix Common Administration Guide - Configure User Response](#).

Using the Admin Console:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button and select the **OpenID Connect** tab.
4. Specify "User Response" in the ***Value to return as Subject (sub) value** attribute.
5. Modify the **User Response to return as Subject (sub) value** accordingly.
6. Click the **Save** button to save the changes.

Identity Mapping via Secondary Account

A secondary account is used to provide mapping between AccessMatrix user with OAuth client user by mapping AccessMatrix user UUID with OAuth client user identity.

There are two ways to configure a secondary account:

- [Using Admin Console](#)
- [Using REST API](#)

Using Admin Console

The Admin Console can be used to configure secondary accounts.

Add Secondary Account

To add a secondary account for an OAuth client end user, perform the following steps:

-
1. Navigate to **Operation Dashboard > User & Segment > User** and search for the relevant user's name.
 2. Click on the relevant user under the **User Id** column to display the **View User page**.
 3. Click the **Secondary Account** button.
 4. Click the **Add Secondary Account** button to display the **Add Secondary Account** dialog box.
 5. Provide values for each of the fields:
 - **App ID - Type** - Select the relevant OAuth client using the drop-down list (e.g. "client1 - OAuthClient").
 - **External Identity** - Enter the external Id (e.g. login Id, unique Id) of the user for the OAuth client application (e.g. "demouser@gmail.com").
 - **Profile Name** - Enter a suitable profile name (e.g. "Profile 1").
 6. Click the **Apply** button to complete the linking process.
 7. Click the **Back** button to complete the configurations.

Manage Secondary Accounts

To manage a user's secondary accounts, perform the following steps:

1. Navigate to **Operation Dashboard > User & Segment > User** and search for the relevant user's name.
 2. Click on the relevant user under the **User Id** column to display the **View User page**.
 3. Click the **Secondary Account** button.
 4. The secondary account you created for the OAuth client application will be listed in the **Secondary Account** tab with **Application Id, Application Object Class, External Identity**, and **Profile Name**.
 - **Application Id** - This will display the OAuth client's Id (e.g. client1).
 - **Application Object Class**- This will display the corresponding Object Class of the OAuth client - OAuthClient.
 - **External Identity** - This will display the user identity (e.g. login Id, unique Id) of the OAuth client application (e.g. demouser@gmail.com).
 - **Profile Name** - This will display the profile name configured in the previous section (e.g. Profile 1)
-

-
5. To delete a secondary account, tick the corresponding checkbox and click the **OK** button.

Configure OAuth Client

To configure the OAuth client to use secondary account as its user identity, perform the following steps:

i **Note:** Proceed with this step only when the necessary secondary account configurations have been completed.

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth client Id** column.
3. Click the **Edit** button and select the **OpenID Connect** tab.
4. Specify "Secondary Account" as the ***Value to return as Subject (sub) value** attribute.
5. Click the **Save** button to save the changes.

Using REST API

The AccessMatrix REST API service can also be used to configure secondary accounts.

i **Note:** For more details on the various APIs, refer to [AccessMatrix REST API Specification](#).

Provision OAuth Client End User Identity

To add the user identity of the OAuth client to the AccessMatrix user, use the `authz/2ndacct/update` API.

The OAuth client's user identity can be provisioned using a secondary account specified as a JSON object. This can be either a single account, or an array of accounts.

i **Note:** For more details on the account parameters, refer to [Appendices - Account Parameters](#).

Manage Secondary Account

There are several APIs available to manage the secondary account of end users:

- To retrieve a user's secondary account Id, use the `authz/oauth/accessToken/user/2ndacct/get` API.
 - To retrieve the user identity of OAuth client for a specific AccessMatrix user, use the `authz/2ndacct/getByProfile` API.
-

i **Note:** For more details on the account parameters, refer to [Appendices - Account Parameters](#).

- To delete the user identity of OAuth client for a specific AccessMatrix user, use the `authz/2ndacct/deleteByProfile` API.

i **Note:** For more details on the account parameters, refer to [Appendices - Account Parameters](#).

Configure OAuth Client

To configure the OAuth client in the Admin Console to use secondary account as its user identity, perform the following steps:

i **Note:** Proceed with this step only when provisioning has been completed.

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
 2. Click on the relevant OAuth client under the **OAuth Client Id** column.
 3. Click the **Edit** button and select the **OpenID Connect** tab.
 4. Modify the **Value to return as Subject (sub) value** attribute to "Secondary Account".
 5. Click the **Save** button to save the changes.
-

5. UAM OAuth and OIDC OP Implementation

OAuth 2.1 and AccessMatrix UAM

OAuth 2.1 focuses on the client developer simplicity while providing specific authorization flows for web applications, desktop applications, mobile phones, and living room devices.

OpenID Connect provides user authentication and an SSO layer which is lighter and easier to use than Security Assertion Markup Language (SAML). The specification and associated Request for Comments (RFCs) are developed by the Internet Engineering Task Force (IETF) OAuth Working Group (WG). The main framework and Bearer Token Usage were published in 2012. Other documents are still being worked on within the OAuth working group.

At a high level, the main API endpoints are:

- **Authorization Endpoint**
 - To initiate an OAuth 2.1 request.
 - **Token Endpoint**
 - For the Authorization Code grant, this endpoint allows access token and optionally refresh token and ID token to be exchanged using an Authorization Code.
 - For the Resource Owner Password Credentials grant, this endpoint retrieves the access token and optionally refresh token and ID token.
 - This endpoint is used to refresh access token using a refresh token.
 - **JSON Web Key Set (JWKS) Endpoint**
 - This endpoint retrieves JWKS for JWT signature verification.
 - **Provider Metadata Endpoint**
 - This endpoint is used to query information about the AccessMatrix OpenID Connect configuration. This URL returns a JSON listing of the OpenID/OAuth endpoints, supported scopes and claims, public keys used to sign the tokens, and other details. Relying Party can use this information to construct a request to the OpenID Connect Provider.
 - **User Info Endpoint**
 - This is an OAuth 2.1 Protected Resource that returns claims about the authenticated end user (in JSON format). To obtain the requested claims about the end user, the client makes a request to the UserInfo Endpoint using an access token obtained through OpenID Connect Authentication.
-

- **Revoke Token Endpoint**

- This endpoint is used to revoke access token or refresh token.

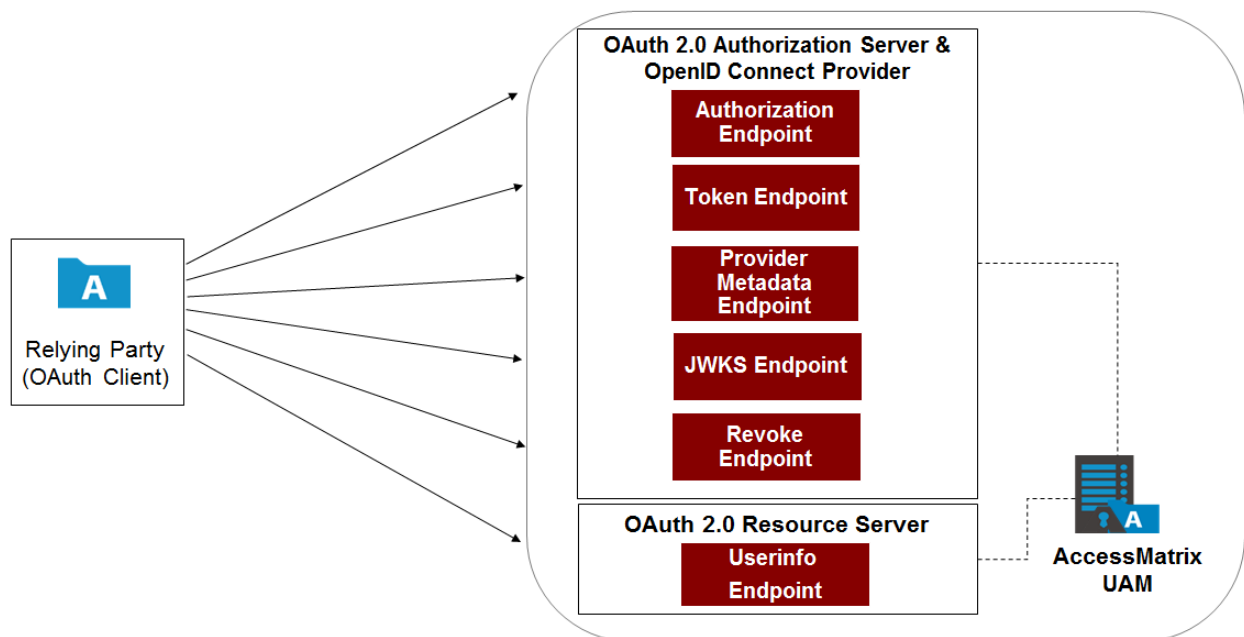
- **Token Introspection Endpoint**

- This endpoint is used to check the validity of access token or refresh token.

OpenID Connect and AccessMatrix UAM

OpenID Connect is a simple identity layer on top of the OAuth 2.1 protocol which allows computing clients to verify the identity of an end user based on the authentication performed by an authorization server. It is an SSO protocol for authenticating end users and asserting their identities.

- OIDC provides a signed id_token for the client.
- OIDC provides access to the AccessMatrix UAM Authorization Server.



AccessMatrix UAM is the identity provider responsible for verifying the identities of users and applications that exist in an organization's directory. It is also responsible for issuing ID tokens upon successful authentication of the users and applications.

Below is the basic authentication flow with AccessMatrix UAM as the identity provider:

1. The application sends a request to AccessMatrix UAM.
 2. AccessMatrix UAM authenticates the client.
 3. AccessMatrix UAM authenticates the user.
-

-
4. AccessMatrix UAM approves or denies the requested scopes.
 5. AccessMatrix UAM mints a token and sends it in the response.
 6. The application validates the ID token's integrity.



Important Note: AccessMatrix UAM uses public key cryptography to sign tokens and verify its validity.

OpenID Provider Security Handling

AccessMatrix does validation following the OAuth and OIDC standard. For example: Ensures that redirect URI parameter is properly validated against the registered redirect URI of the Relying Party Enforces, as OpenID Provider, the checking of the user login session (sessionToken) before OAuth and OIDC tokens are generated

- Ensures that authorization code is meant to be consumed only once

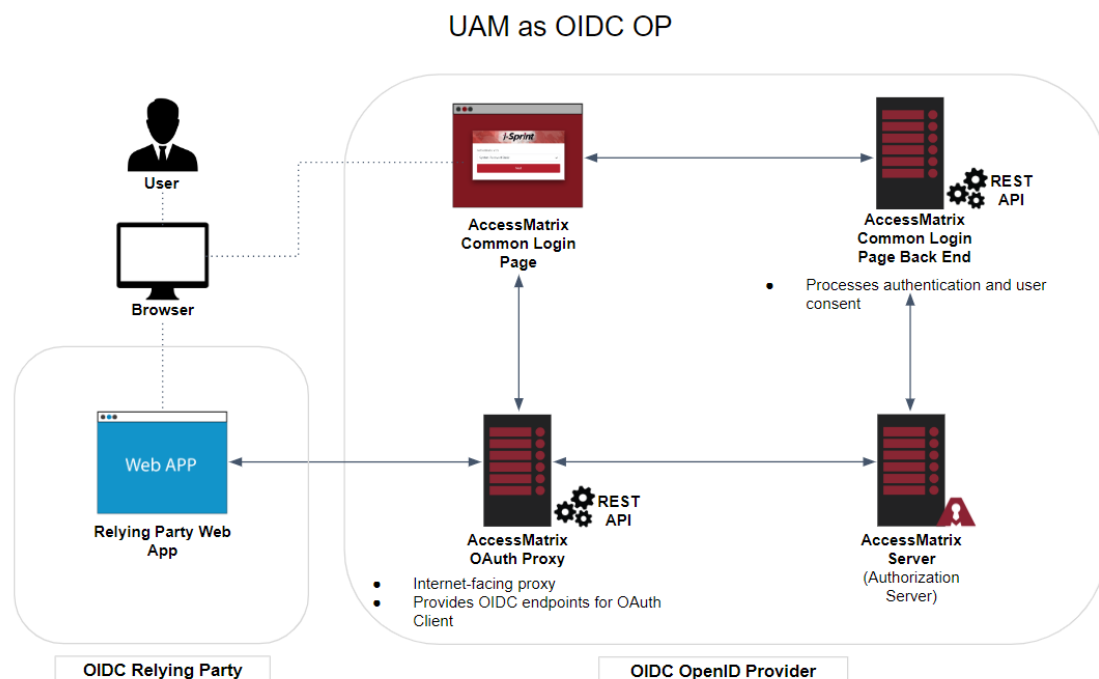
5.1. UAM OAuth and OIDC Integration

AccessMatrix UAM is a certified OIDC provider for the Basic OpenID Connect Provider (OP), Implicit OP, and Config OP profiles. It also conforms to the OAuth 2.1 standard.

The following sections contain diagrams that illustrate how AccessMatrix UAM acts as the OpenID Connect Provider.

Standard Deployment of UAM as OIDC OP

The standard deployment of UAM as the OIDC OP utilizes the AccessMatrix Common Login Page (CLP) and OAuth Proxy. Here, UAM serves as the identity and authorization platform for the OAuth client application.



The standard deployment uses the following components:

- **Relying Party Web App** - The RP web app consists of the OAuth client's user-facing front end (and potentially a back-end server, represented with the front end as one item here for simplicity). It communicates with the OIDC OP endpoints (published by the OAuth Proxy) to authenticate/perform SSO for its user and request access to protected resources.
- **AccessMatrix OAuth Proxy** - The OAuth proxy is a lightweight component designed to provide the OAuth client with easy access to the OIDC OP endpoints. It does this by receiving, validating,

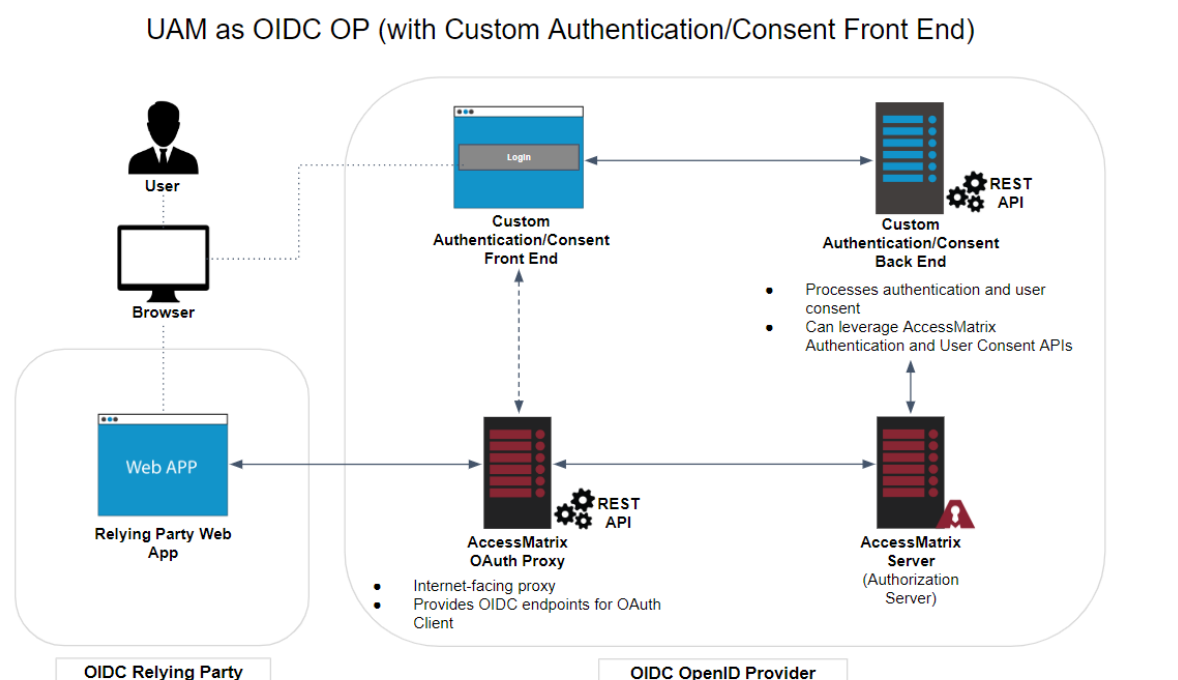
and forwarding requests from the OAuth client to the AccessMatrix server. Unlike the other components comprising the OP, this component is internet-facing so that it can communicate with the OAuth client. It can be configured to redirect to the custom authentication front end for user authentication and consent prompt.

- **AccessMatrix CLP Front End and Back End** - CLP is a AccessMatrix web-based authentication and consent component that comes bundled with AccessMatrix. It provides both a user-facing front end, the login page, as well as a back end that processes user authentication and consent by communicating with the AccessMatrix server.
- **AccessMatrix Server** - The AccessMatrix server acts as the OP's authorization server. It validates the OAuth client's requests, verifies the user's identity and permissions, and issues the OIDC tokens to the RP web app.

Customized Deployment of UAM as OIDC OP with a Custom Authentication/Consent Front End

The AccessMatrix Common Login Page (CLP) allows basic UI customization if a project requires its own authentication/consent front end and back end components. The custom authentication/consent back end can make use of the Authentication and User Consent APIs provided by AccessMatrix UAS.

Such a deployment can be customized in various ways to meet customers' diverse requirements. The diagram below reflects a typical custom deployment.



A typical custom deployment uses the following components:

- **Relying Party Web App** - The RP web app consists of the OAuth client's user-facing front end (and potentially a back-end server, represented with the front end as one item here for simplicity). It communicates with the OIDC OP endpoints (published by the OAuth Proxy) to authenticate/perform SSO for its user and request access to protected resources.
- **AccessMatrix OAuth Proxy** - The OAuth proxy is a lightweight component designed to provide the OAuth client with easy access to the OIDC OP endpoints. It does this by receiving, validating, and forwarding requests from the OAuth client to the AccessMatrix server. Unlike the other components comprising the OP, this component is internet-facing so that it can communicate with the OAuth client. It can be configured to redirect to the custom authentication front end for user authentication and consent prompt.
- **Custom Authentication/Consent Front End** - A custom front end where the user is prompted to authenticate themselves and provide consent to the RP web app to access their protected resources.
- **Custom Authentication/Consent Back End** - The back end component that processes user authentication and consent by communicating with the AccessMatrix server. It can leverage the Authentication and User Consent APIs provided by AccessMatrix UAS.
- **AccessMatrix Server** - The AccessMatrix server acts as the OP's authorization server. It validates the OAuth client's requests, verifies the user's identity and permissions, and issues the OIDC tokens to the RP web app.

Refer to [Custom Login Page and OAuth Proxy Integration](#) for more information on the integrating a custom authentication/consent component with AccessMatrix UAS.

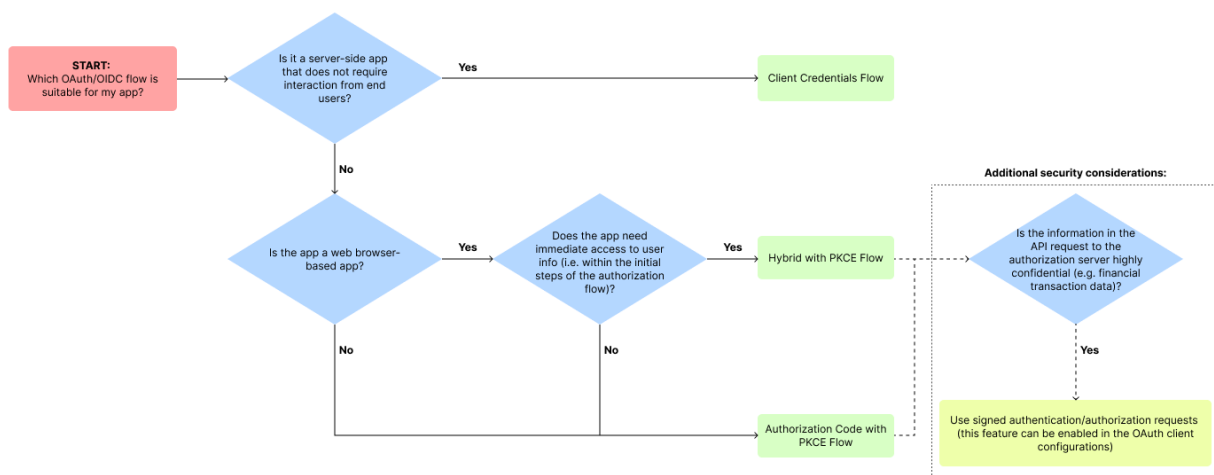
5.2. Implementation Planning

Planning prior to environment setup is critical to the success of UAM OAuth 2.0 and OIDC implementation. Understanding the customer's nature of the business, studying the customer's requirements and its existing environments will help to identify the actions to make prior to actual implementation.

The following sections discuss the implementation considerations that should be assessed during planning.

Choosing the Right OAuth/OIDC Flow

This section discusses the requirements to be considered during OAuth 2.0 integration. The following diagram provides a quick way to identify the type of OAuth client you are using and the most suitable OAuth/OIDC flow. The sections below that provide further details on the considerations you should have when selecting a flow.



The OAuth flow to use will depend on the type of client you are building:

- **If the OAuth client application does not requires interaction from end users**

Usually such application is a server-to-server application. It is a confidential client running on a web server and it is specifically designed to invoke Dynamic Client Registration endpoint. Confidential clients must authenticate with the authorization server when making requests to the token endpoints.

- **Determine if the OAuth client application is public or confidential**

- **Confidential** - if the client application is a server and the client credentials (e.g. access token and client secret) are not exposed at the client side of the application. This applies

to the mobile application which has the server-side back end to process OAuth request (e.g. authorization/authentication request and access token request).

- **Public** - if the client application cannot maintain the confidentiality of the client credentials (i.e. access token, client secret). Applications like single-page applications, any mobile or native applications cannot keep secrets from malicious users.
- **If the OAuth client application is a web based application**
 - Usually web based application has its client code downloaded from a web server and executes within a browser on the device used by the end user. This applies to a Single-page Application (SPA) and Mobile Native Application. It is recommended that [Authorization Code Flow with PKCE](#) to be used.
 - If the OAuth client application is a server-side web application (i.e. a web application which server back end's source code is not publicly exposed and where it is possible that its back end is microservices but stateful) and yet allows end-user web browser interaction, the [Authorization Code Flow with PKCE](#) should be used.
- **If the OAuth client application requires immediate access to user info**

If your application is able to securely store client secrets, it is possible to use the [Hybrid Flow with PKCE](#), which allows your front-end application to have immediate access to an ID token while still requiring backend processing to get access token to access the protected resources. Depending on your requirements, this flow may provide the flexibility needed to meet your business objectives. However, if such flexibility is not required, it is recommended that you use [Authorization Code Flow with PKCE](#).
- **(Deprecated in OAuth 2.1) If the OAuth client application is trusted with user credential and if the redirect-based flow is not possible**
 - If the resource owner (such as device operating system or a highly privileged application) owns both the client application and the resource that it is accessing, then the application can be trusted to store resource owner login credential. However, [Resource Owner Password Credentials Flow](#) should only be used when other flows or other redirect-based flows are not feasible, as it cannot realize SSO and support multi-step authentications.

•

Type of Application	OIDC Flow Type
Server-side application	Authorization Code Flow with PKCE

Type of Application	OIDC Flow Type
Single-page application (SPA)	Authorization Code Flow with PKCE
Mobile native application	Authorization Code Flow with PKCE
Absolute trusted application	Authorization Code Flow with PKCE
Server to server application	Client Credentials Flow (OAuth only)
Application that can securely store client secrets	Hybrid Flow with PKCE

- **Identify what type of token(s) your application needs**

The OAuth flow to use will also depend on the type of token required by the application. Refer to the table below to know which type of tokens are supported by the corresponding OAuth flow:

OAuth Flow Type	Access Token	ID Token	Refresh Token
Client Credentials Flow	Yes	No	No
Hybrid Flow with PKCE	Yes	Yes	Yes
Authorization Code Flow with PKCE	Yes	Yes	Yes
Authorization Code Flow	Yes	Yes	Yes
Implicit Flow	Yes	Yes	No
Resource Owner Password Credentials Flow	Yes	Yes	Yes
Hybrid Flow	Yes	Yes	Yes

- **Identify the OAuth flow to use**

Determine which OAuth flow to use according to the type of client application and tokens required. Refer to [OAuth 2.1](#) and [OIDC Flows](#) for the details of each OAuth flow.

Implementation Considerations

Security Hardening

Here is the list of the security measures that help to mitigate the risk of the security attacks (e.g. man-in-the-middle (MitM) attacks, replay attack, CSRF attack):

- **Using Request Object together with nonce or state** This requires the OAuth Client application to sign the OAuth request (in JWT format) to prove its integrity
-

and authenticity to the OpenID Provider and to ensure that the data has not been tampered with. The request object can include parameters that help mitigate replay attacks, such as nonce and state values:

- **Nonce:** A unique value included in the authentication request to mitigate replay attacks. The authorization server includes this nonce in the ID token, and the client verifies it to ensure the token is fresh and has not been reused.
 - **State:** A value used to maintain state between the client and the server during the authentication process. It helps prevent Cross-Site Request Forgery (CSRF) attacks and can also assist in detecting replay attacks.
 - **State and nonce must be random and validated** As part of the OIDC authentication flow implementation, state and nonce (if applicable) parameters must be randomly generated properly by the Relying Party and stored properly as part of the user's application session. When the Relying Party receives any redirection from the OpenID Provider, Relying Party needs to properly validate the state and nonce. This is to avoid request forgery where the authentication request was initiated by an outsider (not the Relying Party).
 - **Short JWT request lifetimes (if request object is used)** Issue request object with short lifetimes. This reduces the window of opportunity for an attacker to reuse a stolen token.
 - **Using OAuth Authorization Code Flow with PKCE** This is a highly recommended flow to use in order to secure the web based clients. PKCE adds a layer of security by requiring a dynamically generated code verifier to obtain the tokens to prevent CSRF.
 - **Code Challenge:** A hashed version of the code verifier sent in the authorization request.
 - **Code Verifier:** A cryptographically random string created by the client. It is sent with the token request for authorization code validation.
 - **Use a secure client authentication method when authenticating to authorization server using the token endpoint** The Relying Party should use the `private_key_jwt` authentication method instead of the `client_secret_post` or `client_secret_basic` methods whenever possible. This is because private key-based authentication is more secure; client secret-based authentication requires
-

both the client and the authorization server to store client-related credentials, while private key-based authentication does not require the authorization server to store any client-related credentials. This reduces the risk of potential exposure. With private key-based authentication, the client-generated, signed JWT also has an expiry date, which limits the time window for replay attacks. Lastly, the private key can be stored and used for signing within a Hardware Security Module (HSM), which will prevent its extraction and reduce the risk of client impersonation.

- **Not to embed AccessMatrix sessionToken in ID Token if not required**

AccessMatrix has an option to embed AccessMatrix sessionToken in ID token. AccessMatrix sessionToken is considered as OpenID Provider's user login session. Thus, it should not be shared to the Relying Party. However, if the Relying Party is fully trusted and when embedding sessionToken is really required, only then this option may be enabled. That means you may enable it only if the Relying Party will use the embedded sessionToken to make an API call to AccessMatrix. If the Relying Party does not have any use case to make an API call to AccessMatrix using sessionToken, then the embedding of sessionToken in ID token should be disabled.

- **Configure access token and ID token timeout** AccessMatrix allows configuring validation of access token, ID token, and refresh token (if applicable). Please ensure that it is configured just long enough to meet the requirement and security policy. Please refrain from setting them unnecessarily long to reduce the risk of unauthorized access when the tokens are leaked by a Relying Party.

- **Safeguarding access token, refresh token, and ID token** Access token, refresh token, and ID token are important information that need to be safeguarded by the Relying Party. If they are stolen or leaked out, it allows anyone presenting the tokens to access the resource server (e.g. application API

that depends on the access token). You should consider encrypt ID token (in JWE format) and access token (if it is returned in JWT format).

- **ID token validation** ID token must be properly validated that it is issued by OpenID Provider and that it is issued for the correct Relying Party.
- **HTTPS Only** Ensure that all communications use HTTPS to encrypt the data in transit and protect it from interception and tampering.
- **Up-to-date TLS configuration** Ensure that TLS is correctly configured and up-to-date to protect against vulnerabilities.
- **Content Security Policy (CSP)** Implement a robust CSP to prevent cross-site scripting (XSS) and other injection attacks.
- **HTTP Strict Transport Security (HSTS)** Use HSTS to enforce the use of HTTPS and protect against protocol downgrade attacks.
- **Resource Server validation**

For SPA-Microservices, access token and ID token are issued to the Relying Party's SPA (client in the client-server role). It is important that when the client-side needs to make a call to the server, especially for sensitive matter, that ID token and/or access token are further validated by the server-side (e.g. Microservices). In an application where there are client and server component, the server should not simply trust unverifiable information given by the client-side. A common mistake for this is that the client-side gets the ID token; thus, it can get the user identity given by the OpenID Provider. When this client-side needs to make an API call to server (e.g. to get account information or to do single sign-on), the server-side should expect that the ID token and/or access token be sent over so that it can verify the user identity properly, as given by the OpenID Provider, before processing the API call (e.g. to return account information).

Performance Considerations

- **ID Token**

In general, in terms of performance, most of the processing is done when the ID token is generated as it needs to be signed by OpenID Connect Provider private key. Cryptographic operation using private key is expensive in general. ID token may also contains claims. The more claims that it has, the longer the ID token is. It depends on the usage of ID token, if it is being passed around as user session, then this may have an impact in term of performance as well.

- **Refresh Operation**

Refresh operation means the Relying Party provides a refresh token to OpenID Connect Provider so that OpenID Connect Provider issues a new access token and ID token. As mentioned above, the generation of ID token is an expensive process. Thus, refresh token should only be used when the access token or ID token is about to expire to minimize the frequency of refresh operations.

- **Signing Key Size**

ID token signing key size affects the processing resource used to generate ID token. The longer the key size, then the slower it is to sign the ID token. There needs to be a balance between security and performance.

- **Introspection of Access Token in JWT Format**

Generally, token introspection's performance (via /introspect endpoint) of Access Token in JWT format is relatively better than Access Token in opaque format. Token introspection of Access Token in JWT also can be done by an external party (e.g. Resource Server or API gateway) that has such capability as it only requires JWT signature and expiry verification.

If Access Token in JWT format are strictly not allowed or supported by the OpenId Provider, you may refer to this [link](#) to switch off OAuth Proxy capability to introspect Access Token in JWT format.

6. OAuth Client

The AccessMatrix UAM solution simplifies the implementation of the OAuth 2.1 and OIDC authorization framework. The following steps will guide you to integrate your OAuth client application with AccessMatrix UAM as the OpenID Connect Provider.

Implementation Checklist

Below are the tasks for UAM OAuth 2.1 and OIDC implementation checklist:

1. Decide on the OAuth 2.1/OIDC flow to use. Refer to [Choosing the Right OAuth/OIDC Flow](#) for useful information on this.
 2. Once the OAuth 2.1/OIDC flow has been decided, register your OAuth client application via one of the following methods by referring to instructions specific to your selected flow:
 - [Admin Console Client Registration](#)
 - [Dynamic Client Registration](#)
 - (Optional) Enable the use of secondary accounts to store user identities for your OAuth client. This can be useful if OAuth client's end-user repository is already in Admin Console, but the OAuth client requires a distinct user identity and that data has yet to be provisioned to the user repository. Refer to [Configure Secondary Account for OAuth Client User Identity](#) for details.
 3. (Optional) Read [OIDC Scopes](#) and [Consent Management](#) to find out whether you need to use scopes (apart from "openid") and user consent prompts in your application.
 4. Read [Access Control](#) to enforce restrictions on application access according to group membership.
 5. Read [Retrieving User Info](#) to manage how user info can be obtained by the application.
-

6.1. Secure Authorization Request

In OAuth 2.0, the typical authorization request utilizes query parameters that are sent through user agents (i.e. web browsers). This method is easy to implement, but inherently insecure due to potential weaknesses such as tainted parameters and unauthenticated sources. This is due to an inability to use application layer security to provide confidentiality and integrity protection.

For a more secure way of authorization, JWT-Secured Authorization Requests (JARs) are used. JARs securely push authorization parameters by sending request parameters in a JSON Web Token (JWT). This allows the request to be signed by JSON Web Signature (JWS) and optionally encrypted by JSON Web Encryption (JWE). This ensures that the integrity, source authentication, and confidentiality properties of the authorization request are intact. This type of JWT is referred to as a Request Object.

The Request Object is a way to send OIDC Authentication Request in JWT format. OIDC parameters that are normally sent to the authorization endpoint are packed into the JWT as claims. The Request Object is sent to the authorization endpoint in the request parameter.



Note: Parameters such as `client_id`, `response_type`, and `scope` are still required to be provided in the query parameters.

The advantage of using the Request Object is to add authenticity that the request is coming from the said Relying Party (RP) as JWT is signed using the Relying Party's private key.

When AccessMatrix Universal Access Management (UAM) acts as the OpenID Provider (OP), it will verify the Request Object using the RP's public key and the **Request Object Signing Algorithm** configured in the OAuth client. UAM accepts RP public keys configured either as JWKS string or JWKS endpoint.

For more details about the Request Object, read about it in [OIDC Core Specification: Request Object](#).

Configure Request Object Signature Verification Key

Request Object Signature Verification key can be specified via **JWKS String** or **JWK URL**.

Perform the following steps to configure:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Specify either of the following attributes under the **Attribute** tab:
 - **JWKS Endpoint URL** - JWKS URL published by OAuth Client to allow the retrieval of the public key to encrypt ID token (or Access Token if applicable)



Note: Default value is "2048".

- **JWKS String** - JWK in JSON format
5. Select an algorithm using the drop-down list for **Request Object Signing Algorithm**.



Note: This attribute specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is specified, the OAuth client is allowed to use any of the supported algorithms (i.e. "RS256", "RS384", "RS512", "ES256", "ES384", or "ES512") for Request Object signing.

6. Click the **Save** button to save the changes.

6.2. Recommended Flows

AccessMatrix UAM supports various OAuth/OIDC flows. The flows listed below are the flows that are recommended based on the OAuth 2.1 specification.

- [Authorization Code flow with PKCE](#)
- [Hybrid Flow with PKCE](#)
- [Client Credentials Flow](#)

From a security standpoint, flows that are using PKCE should be accompanied with [request](#) objects to further secure the authorization request.



Note: For more details on request objects, refer to [Secure Authorization Request](#).

6.3. OAuth Client Registration

This page contains instructions on how to register an OAuth Client in the AccessMatrix system. This can be done in one of two ways:

- [Manually via the Admin Console](#)
- [Dynamically via REST API \(Dynamic Client Registration\)](#)

6.3.1. Admin Console Client Registration

To register your OAuth client, navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.

In one of the following sections, select the flow that you wish to use for the next steps:

- [Recommended Flows](#)
- [Deprecated Flows](#)

	Important Note: Do not use deprecated flows when creating new applications!
---	--

Common Configurations

The following sections details configurations that are shared between one or more OAuth client types.

Enable Client Credentials Flow

It is possible for a single OAuth client to support both the Hybrid and Client Credential grant types. To do so, the **Allow client_credentials Grant** setting must be enabled during the creation of the OAuth client:

1. Refer to steps 1-3 in the respective **Configure the OAuth Client** section.
2. Under step 4, ensure that you tick the **Allow client_credentials Grant** checkbox.
3. Complete the remaining steps in the **Configure the OAuth Client** section.

Alternatively, to enable this feature for an existing OAuth client:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click on the relevant OAuth client. The **View OAuth Client** page is displayed.
 2. Click **Edit Application Type**.
 3. Click **Next >** until you reach the **Client Info** page.
 4. Tick the **Allow client_credentials Grant** checkbox.
 5. Click **Next >** and **Finish**.
 6. Click **Save**.
-

Configure the Authorization Code Validity Time

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Authz Server**. The default OAuth server is displayed.
2. Click on the **Edit** button.
3. Under **JWT/OIDC Tokens** tab, configure **Authorization Code Validity Time (in seconds)** or keep it as the default 60 seconds.
4. Click on the **Save** button to save the changes.

Exchange the Code for Tokens

To exchange the code for access and ID tokens, pass this code to the authorization server's /token endpoint using the **HTTP POST** method and the Form Serialization (using application/x-www-form-urlencoded content type). If AccessMatrix UAM's default authorization server (a.k.a. OAuth Proxy) is being used, then the request URL would look something like this:

POST

POST Request Sample

Code

```
POST /oauth-proxy/token HTTP/1.1
Host: {yourDefaultOAuthProxyDomain}.com
Content-Length: 307
Content-Type: application/x-www-form-urlencoded
Authorization: Basic czZC.....
grant_type=authorization_code&redirect_uri=http%3A%2F%2Flocalhost&code=P5I7mdxx...truncated...CSq
```



Important Note: The call to the /token endpoint requires authentication. Authentication can be done via basic authentication of the client ID and secret or via the private key JWT authentication method. You can find the client ID and secret under the OAuth client basic information section.

The successful response will be returned in application/JSON media type:

JSON

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store
Pragma: no-cache
```

```
{  
    "access_token": "SlAV32hkKG",  
    "token_type": "Bearer",  
    "refresh_token": "8xLOxBtZp8",  
    "expires_in": 3600,  
    "id_token": "  
eyJhbGciOiJSUzI1NiIsImtpZCI6IjFlOWdkazcifQ.ewogImlzcyI6ICJodHRwOi8vc2VydmVYLnMvYy92OTlWbmVwbGUuY292TiwKICJzdWIioAiAimjQzMjg5NzYxMDAxLiwiOicAIiczZCaGRSa3F0MyIsCiAibm9uY2UiOiAibi0wUzZfV3pBMklqIiwKICJleHAiOiAAMzExMjgxOTcwLAo  
gmhlhDI6IDEzMTEyODANzAKfQ.ggW8hz1EuVLuxNuuiJKX_V8a_OMxzR0EHR9R6jgdqrOO  
F4daGU96Sr_P6qJp6IcmD3HP99ObilPRs-cwh3L0-  
pl46waJ8Ihehcwl7F09JdijmbQkvPeB2T9CJNqeGpe-gccMg4vfKjkM8FcGvnzZUN4_KSP0aAp1tOJlzZwgjqGBYKhIoT x7Tp dQyHE5lcMiKPxfEI QIL Vq0pc_E2 Dz L7emopWoao zTF_m0_N0 YzFC6 g6 EJbo EorSK5hoDalrcvrYLsrQA ZZZkf ly uVC y ixEO v9 GfnQC3_os jzw2PAith fubEEBLuvVk4XUVrWO Lrl10nx7 RkKU8NXNH q-rvKMzqq"  
}
```

6.3.1.1. Admin Console Client Registration for Recommended Flows

This section contains the client registration steps for the following recommended flows:

- [Client Credentials](#)
- [Hybrid with PKCE](#)
- [Authorization Code with PKCE](#)

Client Credentials

This section contains the steps for the Client Credentials flow implementation for server-to-server applications.

- [Configure the OAuth Client \(Client Credentials\)](#)
- [Configure the Application to Use the Client Credentials Flow](#)
- [Access Protected Resources](#)

Configure the OAuth Client (Client Credentials)

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Client Credentials Flow** radio button and click **Next >**.



Note: If needed, you can click the **Flowchart for Choosing OAuth/OIDC Flow** button for more information on selecting the appropriate flow type.

3. Click **Next >**.



Note: If needed, you can expand the **About this flow** section to find out more about this flow.

4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
OAuth Client Id	Unique Id of OAuth client.
OAuth Client Name	Name of OAuth client.
Description	Description of OAuth client.

5. Fill in the basic setting information:
-

Attribute Name	Description
Grant Type	Value is "client_credentials". (uneditable)
Allowed to call introspection for other OAuth Clients	Setting this to "true" will allow this Introspection OAuth Client, such as an API Gateway or a resource server, to call introspection endpoint on behalf of other OAuth Clients. Default: false (unchecked)

6. Click the **Finish** button. If the OAuth client creation was successful, the **Create OAuth Client** page will be populated with the information configured in the **OAuth Client Configuration Wizard**.

7. (OPTIONAL) You may also fill in or modify the values of the following attributes:

Attribute Name	Description
Status	To activate or disable the OAuth client. "Activated" means OAuth client is enabled. "Disabled" means AccessMatrix will reject authorization and token requests from this OAuth client. Existing tokens (e.g. access token) will also be rejected on best-case basis when verification is done by AccessMatrix. Default: Activated
Attributes Tab	
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200. Default: 3600
Whitelisted Scopes (Press Ctrl + Click to select multiple options)	Determines which scope(s) are allowed to be returned when the OAuth client calls the /token endpoint with scope(s) specified. Note: The list of scopes available for selection is based on the list of scopes configured in the Edit OAuth Authz Server page. Refer to Scope and User Consent for more information.
Access Token Format	Specifies the format of the access token. Options: "Opaque"

Attribute Name	Description
	○ "JWT" Default: Opaque
Client Secret	
Client Secret Validity Period (in minutes)	Sets the validity period of the previous client secret after a client secret reset is triggered. During this period, both the previous client secret and the new client secret will be usable. Set the value to "0" to immediately invalidate the previous client secret upon a client secret reset. Value should be from 0 to 525600. Default: 0

8. Save the changes to complete the setup.

i **Note:** You may need to refer to the **OAuth client_id** attribute and click on the **OAuth Client Secret** button to obtain the client secret when configuring your trusted OAuth client application.

Configure the Application to Use the Client Credentials Flow

To obtain an access token, there is a need to send a request from the server application to the authorization server's token endpoint (/token). If using the AccessMatrix UAM default authorization server (a.k.a. OAuth Proxy), then the request URL should be similar to the example below:

POST

POST Request Sample

Code

```
POST /token HTTP/1.1
Host: server.example.com
Content-Type: application/x-www-form-urlencoded
grant_type=client_credentials
&client_id=uUv2XHaGWRe15g551FFAmw
&client_secret=CGogWdtK85i4SnTXpsv-yQhj
&scope=custom_scope(s)
```

! **Important Note:** The call to the /token endpoint requires authentication. In this case, it is the basic authentication of the client ID and Secret. You can find the client ID and secret under the OAuth client basic information section.

Below are the parameters being passed:

Parameter	Description
grant_type	The value is "client_credentials".
client_id	The client Id generated by AccessMatrix server for each registered OAuthClient.
client_secret	The client secret generated by AccessMatrix server for each registered OAuthClient.
scope	Optional: In the Client Credentials flow, credential owner is not a user and it is not possible to ask for consent; thus, normally scope is not specified. Separate each scope by a blank space.

The OAuth client configuration can be configured to return the access token in JWT format. Refer to the following sample response payload and contents of the access token (in JWT format):

JWT String	
JSON	<pre>eyJraWQiOiJJPQVMtcE5LNllnNnZteHciLCJhbGciOiJSUzI1NiIsInR5cCI6ImF0K2p3dCJ9.eyJpc3MiOiJpLXNwcmcludC5jb20iLCJqdGkiOiJ2NjFQZTlGb1ZlTFNQaVg4SUZCWGdBIiwiaXVkiOiJ0e2s5RFJwQ0M4eHp0MzhjdFR0b1RwdyIsInN1YiI6bnVsbCwiaWF0IjoxNjcyODEwNDYxLCJleHAiOiJlY2NzI4MTQwNjEsIm5iZiI6MTY3Mjg3MDM0MSwiYW1fYXQiOiIwNjAwMDFFQ3hCamtIVDVRZDZCX2RBSzF5MUozTDh3RnF2TkhlWXFOOFg3czZncHJFWTlhR0ViTjNYOGpUd0dQRDNjYlRtOXZxUGI1cGNMaW1MY2lpWFZTN2NQMUJ5Wm95ajV4MGNDUXhId3VUVGlsb0lxcilUWmdkdXVuRTNkbGJDDkx BbnJzbHNTVnptdjBEb1RONDV6bU1mNHNTY1NKbS1TaXBpd09HZHpNUnpnWTdubU5mT2J4TENCY2ZvM0V6ZW44VDBrYWpmWmh3UnJCNkNSMud4SkpGN2dSYUd0QSIIsInNjb3B1IjoieY3JlYXRlQm9vayBmaW5kdXNlciIsImFtciI6IjAiLCJjbGllbnRfaWQiOiJPaZlEUUnBDQzh4enQzOGN0VE5vVHB3IiwiaXV0aG5fcmVhbGlfaWQiOiJEWU5BTU1DX0FHRU5UX1JFQUxNIn0.FgEPuZh686xfaM6QUr78mOyUCS2UE0d7ETAAXrCCiZrxTothBtbWX3lFtdaDv2_NNsJU2uN5uGS4Ynvngxlt3B6c_3-rQzuBT219l8AKlfgPvjOrN6bk-8MJosDsaLYJZJDr3_aNnHd3XH0g664cie4CmvQqhjEz-AGloUsfrcSt2Vtr9p6ahn-LNJIDWgZPaJ0Vr_H5Ph20oNZRAwqPblxmupOf-Zvla5xYT6f_SaDfFl7uqO9Zsyq71TQb3ssIwy_zID5ep4i9M-T4ba1TDv8Q5g77h3ufW2GIgqMxwBDjdGRmy4VFpPepUCZ_nTOHRK0SFRYcZyiMSaebjwv8qPw</pre>
JWT Header	
JSON	<pre>{ "kid": "OAS-pNK6Yg6vmxw", "alg": "RS256", "typ": "at+jwt"}</pre>

JWT String
JWT Payload
JSON <pre> { "iss": "i-sprint.com", "jti": "v61Pe9FoVeLSPiX8IFBXgA", "aud": "Ok9DRpCC8xzt38ctTNoTpW", "sub": null, "iat": 1672810461, "exp": 1672814061, "nbf": 1672810341, "am_at": "060001ECxBjkHT5Qd6B_dAK1y1J3L8wFqvNHeYqN8X7s6gprEY9aGEbN3X8jTwGPD3c bTm9vqPb5pcLimLciiXVS7cPlByZoyj5x0cCQxHwuTTiloIqr- TZgduunE3JlbCvLAnrslsmVzmv0DnTN45zmMf4smcSJm- SipOwOGdzMRzgY7nmNfObxLCBcfo3Ezen8T0kajfZhwRrB6CR1GxJJF7gRaGtA", "scope": "createBook finduser", "amr": "0", "client_id": "Ok9DRpCC8xzt38ctTNoTpW", "authn_realm_id": "DYNAMIC_AGENT_REALM" } </pre>

Access Protected Resources

- Use the APIs described [here](#) to validate access token prior to accessing the protected resource.
- Use access token to access APIs (e.g. protected resource). Refer to [APIs Secured by Access Token](#).

Hybrid with PKCE

This section contains the steps for the Hybrid flow implementation for hybrid applications. A hybrid application refers to an application with the ability to securely store client secrets that requires immediate access to user info.

- [Configure the OAuth Client \(Hybrid with PKCE\)](#)
- [Configure the Authorization Code Validity Time](#)
- [Configure the Application to Use the Hybrid Flow](#)
- [Exchange the Code for Tokens](#)
- [Validate Response](#)
- [Configure the Application to Use Client Credentials Flow](#)
- [Access Protected Resources](#)

Configure the OAuth Client (Hybrid with PKCE)

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Hybrid with PKCE Flow** button and click **Next >**.

i **Note:** If needed, you can click the **Flowchart for Choosing OAuth/OIDC Flow** button for more information on selecting the appropriate flow type.

3. Click **Next >**.

i **Note:** If needed, you can expand the **About this flow** section to find out more about this flow.

4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
OAuth Client Id	Unique Id of OAuth client.
OAuth Client Name	Name of OAuth client.
Description	Description of OAuth client.
Allow client_credentials Grant	When enabled, this allows the OAuth Client to also make use of the Client Credentials grant type (i.e. the access token can be generated without an authorization code) after valid client authentication. Default: false

5. Configure the basic settings:

Attribute Name	Description
Grant Type	Value is "authorization_code". (uneditable)
Response Type	Indicates the additional token(s) to be included with the authorization code in a successful response to OAuth authentication requests using the Authorization Endpoint or API. Depending on the combination of response type values, the following successful responses are returned: o "code id_token" - Successful response includes authorization code and ID token. Default: code id_token

Attribute Name	Description
Use OpenID Connect	This attribute is checked by default. (uneditable)
Token Endpoint Authentication Method	Specifies the authentication method used to obtain and exchange the authorization code for an access token. The default value is "ClientSecret". If "PrivateKeyJWT" is selected, ensure that either the JWKS Endpoint URL or the JWKS String attribute value is specified.
Authentication Method for Token Introspection, Revocation and Refresh	Specifies the authentication method used for token introspection, token revocation and refreshing tokens. The recommended method varies depending on the client type used: - For confidential clients, this method must match the method specified for Token Endpoint Authentication Method. - For public clients, the "PrivateKeyJWT" option is recommended, as this method provides better security by verifying if the requester is a known client. The "none" option should not be selected for either client type, as it is not secure and has been deprecated. It should only be used for backwards compatibility. Default: PrivateKeyJWT
OAuth Redirect URIs	Specifies the URI that the end users will be redirected to, along with the authorization code returned by the OAuth authorization server. (mandatory)
Consent rule	Indicates if the OAuth client can be trusted or not. Options: "TRUSTED" "NOT_TRUSTED"
*Value to return as Subject (sub) value	Indicates where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this

Attribute Name	Description
	OAuth client. This setting will be used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. (mandatory) Options: ○ "Id" ○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account" ○ "User Response"
User Response to return as Subject (sub) value	This attribute is only required if the Value to return as Subject (sub) attribute is configured with "User Response". This allows other alternatives to retrieve a user identity (using user response's predefined field/attributes, or by customization of user response).

6. Click the **Finish** button. If the OAuth client creation was successful, the **Create OAuth Client** page will be populated with the information configured in the **OAuth Client Configuration Wizard**.

7. (OPTIONAL) You may also fill in or modify the values of the following attributes:

Attribute Name	Description
Status	To activate or disable the OAuth client. ○ "Activated" means OAuth client is enabled. ○ "Disabled" means AccessMatrix will reject authorization and token requests from this OAuth client. Existing tokens (e.g. access token) will also be rejected on best-case basis when verification is done by AccessMatrix. Default: Activated
Attributes Tab	
JavaScript Web Origin URIs	This configuration is mandatory if the OAuth client is a browser client that needs to have cross origin access to OIDC endpoints as explained here. It is also applicable if the AccessMatrix OIDC JavaScript library is

Attribute Name	Description
	used by the OAuth client to verify session. Note: This list of web origins must be in a plain domain name and protocol (i.e. http/https). Non-default ports (i.e. 8443 or 8080) must be specified. Paths and wildcards are not supported. Example of valid URI: "https://oidc.example.com"
Required Authentication Level	This is to indicate the minimum authentication level of the authenticated user required by this OAuth client. Default: 1
Whitelisted Scopes (Press Ctrl + Click to select multiple options)	Determines which scope(s) are allowed to be returned when the OAuth client calls the /token or /auth endpoint with scope(s) specified. Note: The list of scopes available for selection is based on the list of scopes configured in the Edit OAuth Authz Server page. Refer to Scope and User Consent for more information.
Embed Session Token	This is to embed the session token in the authorization code, refresh token or ID token, if applicable. If this attribute is enabled, the expiry of ID token, access token, and refresh token will be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting . For refresh token, the expiry is Session Idle Time minus 60 seconds. Default: false
Allow All Users to Access	Enabling this feature will allow all authenticated users to access this OAuth client. For advanced user access control, this feature must be disabled and additional configuration should be performed through the OAuth Client Privilege tab in the user's View User page to define which OAuth client is accessible for that particular user. Default: true
Allow Multiple Profiles	This is to allow a user to have multiple login profiles (secondary account) for the same application. Default: false
Logo URI	The URI that references a logo for the client application.
JWKS Endpoint URL	URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.

Attribute Name	Description
JWKS String	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. JWKS String takes precedence over JWKS Endpoint URL if both are specified.
URL to initiate SSO	Note: It is required only by SSO portal or custom IDP portal that lists user's entitled OIDC RP application(s).
Request Object	
Request Object Signing Algorithm	Specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is specified, the OAuth client is allowed to use any of the supported algorithms for Request Object signing. Options: ○ "RS256" ○ "RS384" ○ "RS512" ○ "ES256" ○ "ES384" ○ "ES512"
Authorization Code	
Authorization Code Format	Specifies the format for the authorization Code. If specified as "JWT", the authorization code will be returned as JWT, where additional custom claims can be embedded based on the Authorization Code User Response attribute. Options: ○ "Opaque" ○ "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_code parameter containing the authorization code.

Attribute Name	Description
Authorization Code User Response	User response to be embedded in the JWT as custom claims. The Authorization Code Format attribute must be specified as "JWT". Note: The JWT will be returned in the code parameter, which contains the embedded custom claims.
Access Token	
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200. Default: 3600
Access Token Format	Specifies the format of the access token. Options: ○ "Opaque" ○ "JWT" ○ "JWE" For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: Opaque
Access Token User Response	User response to be returned in the access token. Applicable for JWT and JWE format.
JSON Web Encryption (JWE)	
JWE Algorithm	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Options: ○ "RSA-OAEP-256" ○ "ECDH-ES+A256KW"
JWE Encryption Algorithm	Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Options: ○ "A256CBC-HS512" ○ "A256GCM"

Attribute Name	Description
Refresh Token	
Return Refresh Token	Indicates whether the refresh token should be returned. Default: false
Refresh Token Expiry (in minutes)	Sets the validity period of a refresh token. Value should be from 10 to 525600. Default: 1440
Refresh Token Lifetime (in hours)	Lifetime expiry for the refresh token, starting from when it is first generated. Refresh token can be renewed up until this expiry. "0" means no lifetime expiry. Value should be from 0 to 1440. Default: 168 hours
Refresh Token Format	Specifies the format for the refresh token. If specified as "JWT", the refresh token will be returned as JWT, where additional custom claims can be embedded based on the Refresh Token User Response attribute. Options: ○ "Opaque" ○ "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_rt parameter containing the refresh token.
Refresh Token User Response	User response to be embedded in the JWT as custom claims. The Refresh Token Format attribute must be specified as "JWT". Note: The JWT will be returned in the code parameter, which contains the embedded custom claims.
Use Refresh Token Integration Date	When set to "true", the integrationDate request parameter passed to the token endpoint will be used as the grant time of the refresh token (where grant time refers to the starting time of a refresh token's validity). The refresh token expiry will also be made the same as the refresh token lifetime, ignoring the

Attribute Name	Description
	value specified in the Refresh Token Expiry attribute such that: ○ Refresh token lifetime expiry = integration date + Refresh Token Lifetime attribute value ○ Refresh token expiry = integration date + Refresh Token Lifetime attribute value Note: This is for SGFinDex. When set to "false", the generation date of the first refresh token will be used as the grant time of the refresh token and this will be used to compute the refresh token lifetime. In this case, the refresh token expiry will always start from the generation date of the refresh token - provided it is still within the refresh token lifetime - and the expiry time will be computed based on the Refresh Token Expiry attribute, such that: ○ Refresh token lifetime expiry = first refresh token's generation time + Refresh Token Lifetime attribute value ○ Refresh token expiry = refresh token's generation time + Refresh Token Expiry attribute value Default: false
OIDC Tokens and User Session	
Revoke Token Upon User Session Invalidation	If this attribute is set to "true", then the invalidation of a user session would also invalidate the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited. When the OIDC tokens (access token, refresh token, and authorization code) are created, they are linked to the particular user session used for the authorization. When that session is invalidated, the OIDC tokens linked to it also get invalidated. Default: false
Invalidate User Session Upon Token Revocation	If this attribute is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. The OIDC tokens (access token, refresh token) are generated based on an active user session. When a token is revoked, its corresponding user session would

Attribute Name	Description
	also get invalidated. Default: false
Client Secret	
Client Secret Validity Period (in minutes)	Sets the validity period of the previous client secret after a client secret reset is triggered. During this period, both the previous client secret and the new client secret will be usable. Set the value to "0" to immediately invalidate the previous client secret upon a client secret reset. Value should be from 0 to 525600. Default: 0
OpenID Connect Tab	
Default Authentication Expiry (in seconds)	Authentication expiry is used to check against user login session.
Attribute claim value Response configuration	This attribute value has to be a user response object to include additional claims of the user in ID token without user consent.
Return ID Token in JWE Format	Setting to "true" will return the ID token as a JWE token. For JWE format: - Ensure that JWE Algorithm and JWE Encryption Algorithm attributes are configured. - Either JWKS Endpoint URL or JWKS String attribute value is specified. Default: false
User Consent	
Enable Auto Saving of User Consent After OAuth Authentication	Enables the granting of consent for requested scopes by the authorization endpoint without the actual user's consent. This behavior may be a potential security loophole and is hence set to "false" by default. It should only be set to "true" if disabling this breaks the existing handling of customized consent pages that still relies on the old behavior where this feature is enabled. Default: false

Attribute Name	Description
Revoke Consented Scopes When Revoking Token	Actions on consented scope(s) of user upon refresh token revocation. Options: ○ "Revoke All" ○ "Keep" Default: Keep
Terms Of Service URL	URL of the terms of service agreement.
Privacy Policies URL	URL for the statement or declaration that explicitly details the company policy regarding how it handles user and customer information.
Policy	
Application Policy	The application policy that will be applied to the OAuth client. This allows the OAuth client to whitelist specific authentication or reauthentication methods for users. This setting may be required if: ○ Enable User Token Revocation with Security Events attribute has been set to "true" in the OAuth Authz Server. ○ CLP is required to enforce authentication using only whitelisted authentication mechanisms configured in the policy. Note: For more details on how to configure the policy, refer to the UAM Administration Guide - Application Policy Configuration.

8. Save the changes to complete the setup.



Note: You may need to refer to the **OAuth client_id** attribute and click on the **OAuth Client Secret** button to obtain the client secret when configuring your trusted OAuth client application.

Configure the Authorization Code Validity Time

Refer to the steps in [Configure the Authorization Code Validity Time](#).

Configure the Application to Use Hybrid Flow

This flow is similar to the Authorization Code flow, except that the `response_type` is multi-valued, where its value is a combination of "code" and "id_token".



Note: Providing access tokens on the first response is not secure. As such, response type values that contain "token" are not supported.

The authorization server supports the use of the **HTTP GET** and **POST** methods at the authorization endpoint.

- If the **HTTP GET** method is used, the request parameters are serialized using URI Query String Serialization.
- If the **HTTP POST** method is used, the request parameters are serialized using Form Serialization.

If using the AccessMatrix UAM default authorization server (a.k.a. OAuth Proxy), then the request URL should be similar to the example below:

Using HTTP GET:

GET

GET Request Sample

Code

```
GET /auth?
  response_type=code id_token
  &scope=openid profile
  &client_id=s6BhdRkqt3
  &redirect_uri=https://client.example.org/
  &state=af0ifjsldkj
  &nonce=n-0S6_WzA2Mj HTTP/1.1
Host: server.example.com
```

Using HTTP POST:

POST

POST Request Sample

Code

```
POST /auth HTTP/1.1
Host: server.example.com
Content-Type: application/x-www-form-urlencoded

response_type=code id_token
&scope=openid profile
&client_id=s6BhdRkqt3
&redirect_uri=https://client.example.org/
&state=af0ifjsldkj
&nonce=n-0S6_WzA2Mj
```

Below are the parameters being passed:

Parameter	Description
client_id	The OAuth Client Id of the created OAuth client in Admin Console. Refer to Configure the OAuth Client .
response_type	Value is "code" and "id_token". The endpoint should return authorization code and ID token.
scope	Value is "openid". It indicates that the token endpoint (/token) will return an ID token. Other additional scopes can also be requested.
redirect_uri	The callback location where the user-agent will be redirected to along with the code. The OAuth Redirect URIs attribute value specified in Configure the OAuth Client must contain this URI.
state	A random alphanumeric string that the authorization server will reproduce when redirecting the user-agent back to the client. This is used to help prevent cross-site request forgery.
nonce	The value that will be returned in the ID token. It is used to mitigate replay attacks.



Note:

- For more information on these parameters, see the [OpenID Connect Endpoints](#) section for reference.
- If the user does not have an existing session, the HTTP request will open the login page.
- If the user has an existing session, or after the user is authenticated, the user will be redirected to the specified redirect_uri along with a code and a token and/or id_token as a hash fragment:

Code

```
HTTP/1.1 302 Found
Location: https://client.example.org/cb#
code=Splxl0BeZQQYbYS6WxSbIA
&token_type=bearer
&id_token=eyJ0...NiJ9.eyJ1c...I6IjIifX0.DeWt4Q
...ZXso
&expires_in=3600
&state=af0ifjsldkj
```

This code validity is based on the value specified in [Configure the Authorization Code Validity Time](#). During this validity period, it can be exchanged for tokens.

Exchange the Code for Tokens

Refer to the steps in [Exchange the Code for Tokens](#).

Validate Response

To validate tokens, refer to [C.Validating JWTs](#).

Configure the Application to Use the Client Credentials Flow

Refer to [Configure the Application to Use the Client Credentials Flow](#) under the **Client Credentials** tab for details on how to use the Client Credentials flow.

Access Protected Resources

- Use the APIs described [here](#) to validate access token prior to accessing the protected resource.
- Use access token to access APIs (e.g. protected resource). Refer to [APIs Secured by Access Token](#).

Authorization Code with PKCE

This section contains the steps for the Authorization Code flow with PKCE implementation for native applications.

- [Configure the OAuth Client \(Authorization Code with PKCE\)](#)
- [Configure the Authorization Code Validity Time](#)
- [Configure the Application to Use the Authorization Code Flow with PKCE](#)
- [Exchange the Code for Tokens](#)
- [Validate Response](#)

-
- [Access Protected Resources](#)

Configure the OAuth Client (Authorization Code with PKCE)

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Authorization Code with PKCE Flow** radio button and click **Next >**.

i **Note:** If needed, you can click the **Flowchart for Choosing OAuth/OIDC Flow** button for more information on selecting the appropriate flow type.

3. For **Client Type**, select either the "Confidential" or "Public" , depending on your requirements, and click **Next >**.

i **Note:** If needed, you can expand the **About this flow** section to find out more about this flow.

4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
OAuth Client Id	Unique Id of OAuth client.
OAuth Client Name	Name of OAuth client.
Description	Description of OAuth client.

5. Configure the basic settings:

Attribute Name	Description
Grant Type	Value is "authorization_code". (uneditable)
Token endpoint authentication method	Specifies the authentication method used to obtain and exchange the authorization code for an access token. The default value is "ClientSecret". If "PrivateKeyJWT" is selected, ensure that either the JWKS Endpoint URL or the JWKS String attribute value is specified.
Response Type	Value is "code". (uneditable)
Use OpenID Connect	This attribute is checked by default. (uneditable)

Attribute Name	Description
OAuth Redirect URIs	Specifies the URI that the end users will be redirected to, along with the authorization code returned by the OAuth authorization server. (mandatory)
Consent rule	Indicates if the OAuth client can be trusted or not. Options: ○ "TRUSTED" ○ "NOT_TRUSTED"
*Value to return as Subject (sub) value	Indicates where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this OAuth client. This setting will be used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. (mandatory) Options: ○ "Id" ○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account" ○ "User Response"
User Response to return as Subject (sub) value	This attribute is only required if the Value to return as Subject (sub) attribute is configured with "User Response". This allows other alternatives to retrieve a user identity (using user response's predefined field/attributes, or by customization of user response). Options: ○ "GroupMembership" ○ "OIDC_address" ○ "OIDC_email" ○ "OIDC_phone" ○ "OIDC_profile"

6. Click the **Finish** button. If the OAuth client creation was successful, the **Create OAuth Client** page will be populated with the information configured in the **OAuth Client Configuration Wizard**.

7. (OPTIONAL) You may fill in or modify the values of the following attributes.

Attribute Name	Description
Status	To activate or disable the OAuth client.

Attribute Name	Description
	○ "Activated" means OAuth client is enabled. ○ "Disabled" means AccessMatrix will reject authorization and token requests from this OAuth client. Existing tokens (e.g. access token) will also be rejected on best-case basis when verification is done by AccessMatrix. Default: Activated
Attributes Tab	
Authentication Method for Token Introspection, Revocation and Refresh	Specifies the authentication method used for token introspection, token revocation and refreshing tokens. The recommended method varies depending on the client type used: ○ For confidential clients, this method must match the method specified for Token Endpoint Authentication Method. ○ For public clients, the "PrivateKeyJWT" option is recommended, as this method provides better security by verifying if the requester is a known client. The "none" option should not be selected for either client type, as it is not secure and has been deprecated. It should only be used for backwards compatibility. Default: PrivateKeyJWT
JavaScript Web Origin URIs	This configuration is mandatory if the OAuth client is a browser client that needs to have cross origin access to OIDC endpoints as explained here. It is also applicable if the AccessMatrix OIDC JavaScript library is used by the OAuth client to verify session. Note: This list of web origins must be in a plain domain name and protocol (i.e. http/https). Non-default port (i.e. 8443 or 8080) must be specified. Path and wildcard are not supported. Example of valid URI: https://oidc.example.com

Attribute Name	Description
Required Authentication Level	This is to indicate the minimum authentication level of the authenticated user required by this OAuth client. Default: 1
Whitelisted Scopes (Press Ctrl + Click to select multiple options)	Determines which scope(s) are allowed to be returned when the OAuth client calls the /token or /auth endpoint with scope(s) specified. Note: The list of scopes available for selection is based on the list of scopes configured in the Edit OAuth Authz Server page. Refer to Scope and User Consent for more information.
Embed Session Token	This is to embed session token in authorization code, refresh token or ID token, if applicable. If this attribute is enabled, the expiry of ID token, access token, and refresh token would be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting . For refresh token, the expiry is Session Idle Time minus 60 seconds. Default: false
Allow All Users to Access	Enabling this feature will allow all authenticated users to access this OAuth client. For advanced user access control, this feature must be disabled and additional configuration should be performed through the OAuth Client Privilege tab in the user's View User page to define which OAuth client is accessible for that particular user. Default: true
Allow Multiple Profiles	This is to allow user to have multiple login profiles (secondary account) for the same application. Default: false
JWKS Endpoint URL	URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.
JWKS String	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. JWKS String takes precedence over JWKS Endpoint URL if both are specified.

Attribute Name	Description
URL to initiate SSO	Note: It is required only by SSO portal or custom IDP portal that lists user's entitled OIDC RP application(s).
Request Object	
Request Object Signing Algorithm	Specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is specified, the OAuth client is allowed to use any of the supported algorithms can be used for Request Object signing. Options: ○ "RS256" ○ "RS384" ○ "RS512" ○ "ES256" ○ "ES384" ○ "ES512"
Authorization Code	
Authorization Code Format	Specifies the format for the authorization Code. If specified as "JWT", the authorization code will be returned as JWT, where additional custom claims can be embedded based on the Authorization Code User Response attribute. Options: ○ "Opaque" ○ "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_code parameter containing the authorization code.
Authorization Code User Response	User response to be embedded in the JWT as custom claims. The Authorization Code Format attribute must be specified as "JWT". Note: The JWT will be returned in the code

Attribute Name	Description
	parameter, which contains the additional embedded custom claims.
Access Token	
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200. Default: 3600
Access Token Format	Specifies the format of the access token. Options: ○ "Opaque" ○ "JWT" ○ "JWE" For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: Opaque
Access Token User Response	User response to be returned in access token. Applicable for JWT and JWE format.
JSON Web Encryption (JWE)	
JWE Algorithm	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Options: ○ "RSA-OAEP-256" ○ "ECDH-ES+A256KW"
JWE Encryption Algorithm	Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Options: ○ "A256CBC-HS512" ○ "A256GCM"
Refresh Token	

Attribute Name	Description
Return Refresh Token	Indicates whether the refresh token should be returned. Default: false
Refresh Token Expiry (in minutes)	Sets the validity period of a refresh token. Value should be from 10 to 525600. Default: 1440
Refresh Token Lifetime (in hours)	Lifetime expiry for the refresh token, starting from when it is first generated. Refresh token can be renewed up until this expiry. "0" means no lifetime expiry. Value should be from 0 to 1440. Default: 168 hours
Refresh Token Format	Specifies the format for the refresh token. If specified as "JWT", the refresh token will be returned as JWT, where additional custom claims can be embedded based on the Refresh Token User Response attribute. Options: "Opaque" "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_rt parameter containing the refresh token.
Refresh Token User Response	User response to be embedded in the JWT as custom claims. The Refresh Token Format attribute must be specified as "JWT". Note: The JWT will be returned in the code parameter, which contains the embedded custom claims.
OIDC Tokens and User Session	
Revoke Token Upon User Session Invalidation	If this attribute is set to "true", then the invalidation of a user session would also invalidate the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited.

Attribute Name	Description
	When the OIDC tokens (access token, refresh token, and authorization code) are created, they are linked to the particular user session used for the authorization. When that session is invalidated, the OIDC tokens linked to it also get invalidated. Default: false
Invalidate User Session Upon Token Revocation	If this attribute is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. The OIDC tokens (access token, refresh token) are generated based on an active user session. When a token is revoked, its corresponding user session would also get invalidated. Default: false
OpenID Connect Tab	
Default Authentication Expiry (in seconds)	Authentication expiry is used to check against user login session.
Attribute claim value Response configuration	This attribute value has to be a user response object to include additional claims of user in ID token without user consent.
Return ID Token in JWE Format	Setting to "true" will return the ID token as a JWE token. For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: false
User Consent	
Enable auto saving of user consent after OAuth Authentication	Enables the granting of consent for requested scopes by the authorization endpoint without the actual user's consent. This behavior may be a potential security loophole and is hence set to "false" by default. It should only be set to "true" if disabling this breaks the existing handling of a customized

Attribute Name	Description
	consent page that still relies on the old behavior where this feature is enabled. Default: false
Revoke Consented Scopes When Revoking Token	Actions on consented scope(s) of user upon refresh token revocation. Options: ○ "Revoke All" ○ "Keep" Default: Keep
Terms Of Service URL	URL of the terms of service agreement.
Privacy Policies URL	URL for the statement or declaration that explicitly details the company
Policy	
Application Policy	The application policy that will be applied to the OAuth client. This allows the OAuth client to whitelist specific authentication or reauthentication methods for users. This setting may be required if: ○ Enable User Token Revocation with Security Events attribute has been set to "true" in the OAuth Authz Server. ○ CLP is required to enforce authentication using only whitelisted authentication mechanisms configured in the policy. Note: For more details on how to configure the policy, refer to the UAM Administration Guide - Application Policy Configuration.

8. Save the changes to complete the setup.

Configure the Authorization Code Validity Time

Refer to the steps in [Configure the Authorization Code Validity Time](#).

Configure the Application to Use the Authorization Code Flow with PKCE

Similar to the regular Authorization Code flow, this flow requires your application to make a request to your authorization server's authorize endpoint. In addition to that, a code challenge is required to be generated and passed along with the request.

Here is the details to generate a code verifier and challenge in your native application:

- **Code verifier:** Random URL-safe string with a minimum length of 43 characters.
- **Code challenge:** Base64 URL-encoded SHA-256 hash of the code verifier.

The PKCE generator code in your application should create output as such:

The `code_challenge` is a Base64-URL-encoded string of the SHA256 hash of the `code_verifier`. Your app will save the `code_verifier` for later, and send the `code_challenge` along with the authorization request to your authorization server's authorize URL.

If you are using AccessMatrix UAM default authorization server (a.k.a. OAuth Proxy), then your request URL would look something like this:

GET

GET Request Sample

Code

```
GET /auth?
  response_type=code
  &scope=openid
  &client_id=0oabygpxgk9lXaMgF0h7
  &redirect_uri=yourApp%3A%2Fcallback&
  &state=sh87c-386e...truncated...3d8967e9
  &code_challenge_method=S256
  &code_challenge=qjrzSW9gMiUgpUvqgEPE4_-
8swvyCtfOVvg55o5S_es
HTTP/1.1 Host: server.example.com
```

Below are the parameters being passed:

Parameter	Description
client_id	The OAuth Client Id of the created OAuth client in Admin Console. Refer to Configure the OAuth Client .

Parameter	Description
response_type	Value is "code". It indicates that authorization code grant type is used.
scope	Value is "openid". It indicates that the token endpoint (/token) should return an ID token.
redirect_uri	The callback location where the user-agent will be redirected to along with the code. The OAuth Redirect URIs attribute value specified in Configure the OAuth Client must contain this URI.
state	A random alphanumeric string that the authorization server will reproduce when redirecting the user-agent back to the client. This is used to help prevent cross-site request forgery.
code_challenge_method	The hash method used to generate the challenge which will always be S256.
code_challenge	The code challenge used for PKCE.



Note:

- For more information on these parameters, see the [OpenID Connect Endpoints](#) section for reference.
- If the user does not have an existing session, the HTTP request will open the login page.
- If the user has an existing session or after the user is authenticated, then the user will be redirected to the specified redirect_uri along with an authorization code.

Code

```
yourApp:/callback?code=P5I7mdxx...truncated...CSqc&state=s-386e...truncated...3d8967e9
```

This code can only be used once and is valid for 10 minutes, during which it can be exchanged for tokens.

Exchange the Code for Tokens

Refer to the steps in [Exchange the Code for Tokens](#).

Validate Response

To validate tokens, refer to [C. Validating JWTs](#).

Access Protected Resources

- Use the APIs described [here](#) to validate access token prior to accessing the protected resource.
- Use access token to access APIs (e.g. protected resource). Refer to [APIs Secured by Access Token](#).

6.3.1.2. Admin Console Client Registration for Deprecated Flows

This section contains flows that have been deprecated and are no longer included in the latest OAuth 2.1 draft specification. You should avoid using these flows for new applications. They continue to be supported only as an option for existing, legacy applications. Refer to the following:

- [Authorization Code](#)
- [Implicit](#)
- [Hybrid](#)

Authorization Code

i **Note:** Based on the new OAuth best practice guidelines and the latest OAuth 2.1 draft specification, the PKCE challenge must be used with the Authorization Code grant whenever possible. It is recommended that you use the Authorization Code with PKCE flow instead.

This section contains the steps for the Authorization Code flow implementation for server-side applications.

- [Configure the OAuth Client \(Authorization Code\)](#)
- [Configure the Authorization Code Validity Time](#)
- [Enable Client Credentials Flow](#)
- [Configure the Application to Use the Authorization Code Flow](#)
- [Exchange the Code for Tokens](#)
- [Validate Response](#)
- [Configure the application to use Client Credentials Flow](#)
- [Access Protected Resources](#)

Configure the OAuth Client (Authorization Code)

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Authorization Code Flow** radio button and click **Next >**.

i **Note:** If needed, you can click the **Flowchart for Choosing OAuth/OIDC Flow** button for more information on selecting the appropriate flow type.

-
3. Click **Next >**.

i **Note:** If needed, you can expand the **About this flow** section to find out more about this flow.

4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
OAuth Client Id	Unique Id of OAuth client.
OAuth Client Name	Name of OAuth client.
Description	Description of OAuth client.

5. Fill in the basic setting information:

Attribute Name	Description
Grant Type	Value is "authorization_code". (uneditable)
Token endpoint authentication method	Specifies the authentication method used to obtain and exchange the authorization code for an access token. The default value is "ClientSecret". If "PrivateKeyJWT" is selected, ensure that either the JWKS Endpoint URL or the JWKS String attribute value is specified.
Response Type	Value is "code". (uneditable)
Use OpenID Connect	This attribute is checked by default. (uneditable)
OAuth Redirect URIs	Specifies the URI that the end users will be redirected to, along with the authorization code returned by the OAuth authorization server. (mandatory)
Consent rule	Indicates if the OAuth client can be trusted or not. Options: ○ "TRUSTED" ○ "NOT_TRUSTED"
*Value to return as	Indicates where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token)

Attribute Name	Description
Subject (sub) value	for this OAuth client. This setting will be used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. (mandatory) Options: ○ "Id" ○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account" ○ "User Response"
User Response to return as Subject (sub) value	This attribute is only required if the Value to return as Subject (sub) attribute is configured with "User Response". This allows other alternatives to retrieve a user identity (using user response's predefined field/attributes, or by customization of user response).

- Click the **Finish** button. If the OAuth client creation was successful, the **Create OAuth Client** page will be populated with the information configured in the **OAuth Client Configuration Wizard**.
- If your OAuth client is in a domain different from the OIDC endpoints, it is mandatory to whitelist your origin using the **JavaScript Web Origin URIs** configuration.
- (OPTIONAL) You may also fill in or modify the values of the following attributes:

Attribute Name	Description
Status	To activate or disable the OAuth client. ○ "Activated" means the OAuth client is enabled. ○ "Disabled" means AccessMatrix will reject authorization and token requests from this OAuth client. Existing tokens (e.g. access token) will also be rejected on best-case basis when verification is done by AccessMatrix. Default: Activated
Attributes Tab	
Allow client_credentials Grant	When enabled, this allows the OAuth Client to also make use of the Client Credentials grant type (i.e. the access token can

Attribute Name	Description
	be generated without an authorization code) after valid client authentication. Default: false
JavaScript Web Origin URIs JavaScript Web Origin URIs	This configuration is mandatory if the OAuth client is a browser client that needs to have cross origin access to OIDC endpoints as explained here. It is also applicable if the AccessMatrix OIDC JavaScript library is used by the OAuth client to verify session. Note: This list of web origins must be in a plain domain name and protocol (i.e. http/https). Non-default port (i.e. 8443 or 8080) must be specified. Path and wildcard are not supported. Example of valid URI: https://oidc.example.com
Required Authentication Level	This is to indicate the minimum authentication level of the authenticated user required by this OAuth client. Default: 1
Whitelisted Scopes (Press Ctrl + Click to select multiple options)	Determines which scope(s) are allowed to be returned when the OAuth client calls the /token or /auth endpoint with scope(s) specified. Note: The list of scopes available for selection is based on the list of scopes configured in the Edit OAuth Authz Server page. Refer to Scope and User Consent for more information.
Embed Session Token	This is to embed session token in authorization code, refresh token or ID token, if applicable. If this attribute is enabled, the expiry of ID token, access token, and refresh token would be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting . For refresh token, the expiry is Session Idle Time minus 60 seconds. Default: false
Allow All Users to Access	Enabling this feature will allow all authenticated users to access this OAuth client. For advanced user access control, this feature must be disabled and additional configuration should be performed through the OAuth Client Privilege tab in the user's View User page to define which OAuth client is accessible for that particular user. Default: true
Allow Multiple Profiles	This is to allow a user to have multiple login profiles (secondary account) for the same application. Default: false
Logo URI	The URI that references a logo for the client application.

Attribute Name	Description
JWKS Endpoint URL	URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.
JWKS String	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. JWKS String takes precedence over JWKS Endpoint URL if both are specified.
URL to initiate SSO	Note: It is required only by SSO portal or custom IDP portal that lists user's entitled OIDC RP application(s).
Request Object	
Request Object Signing Algorithm	Specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is specified, the OAuth client is allowed to use any of the supported algorithms for Request Object signing. Options: ○ "RS256" ○ "RS384" ○ "RS512" ○ "ES256" ○ "ES384" ○ "ES512"
Authorization Code	
Authorization Code Format	Specifies the format for the authorization Code.If specified as "JWT", the authorization code will be returned as JWT, where additional custom claims can be embedded based on the Authorization Code User Response attribute. Options: ○ "Opaque" ○ "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_code parameter containing the authorization code.

Attribute Name	Description
Authorization Code User Response	User response to be embedded in the JWT as custom claims. The Authorization Code Format attribute must be specified as "JWT". Note: The JWT will be returned in the code parameter, which contains the additional embedded custom claims.
Access Token	
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200. Default: 3600
Access Token Format	Specifies the format of the access token. Options: ○ "Opaque" ○ "JWT" ○ "JWE" For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: Opaque
Access Token User Response	User response to be returned in access token. Applicable for JWT and JWE format.
JSON Web Encryption (JWE)	
JWE Algorithm	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Options: ○ "RSA-OAEP-256" ○ "ECDH-ES+A256KW"
JWE Encryption Algorithm	Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Options: ○ "A256CBC-HS512" ○ "A256GCM"

Attribute Name	Description
Refresh Token	
Return Refresh Token	Indicates whether the refresh token should be returned. Default: false
Refresh Token Expiry (in minutes)	Sets the validity period of a refresh token. Value should be from 10 to 525600. Default: 1440
Refresh Token Lifetime (in hours)	Lifetime expiry for the refresh token, starting from when it is first generated. Refresh token can be renewed up until this expiry. "0" means no lifetime expiry. Value should be from 0 to 1440. Default: 168 hours
Use Refresh Token Integration Date	When set to "true", the <code>integrationDate</code> request parameter passed to the token endpoint will be used as the grant time of the refresh token (where grant time refers to the starting time of a refresh token's validity). The refresh token expiry will also be made the same as the refresh token lifetime, ignoring the value specified in the Refresh Token Expiry attribute such that: - Refresh token lifetime expiry = integration date + Refresh Token Lifetime attribute value - Refresh token expiry = integration date + Refresh Token Lifetime attribute value Note: This is for SGFinDex. When set to "false", the generation date of the first refresh token will be used as the grant time of the refresh token and this will be used to compute the refresh token lifetime. In this case, the refresh token expiry will always start from the generation date of the refresh token - provided it is still within the refresh token lifetime - and the expiry time will be computed based on the Refresh Token Expiry attribute, such that: - Refresh token lifetime expiry = first refresh token's generation time + Refresh Token Lifetime attribute value - Refresh token expiry = refresh token's generation time + Refresh Token Expiry attribute value Default: false

Attribute Name	Description
Refresh Token Format	Specifies the format for the refresh token. If specified as "JWT", the refresh token will be returned as JWT, where additional custom claims can be embedded based on the Refresh Token User Response attribute. Options: ○ "Opaque" ○ "JWT" Default: Opaque Note: The JWT will be returned in the <code>code</code> parameter, which includes the <code>am_rt</code> parameter containing the refresh token.
Refresh Token User Response	User response to be embedded in the JWT as custom claims. The Refresh Token Format attribute must be specified as "JWT". Note: The JWT will be returned in the <code>code</code> parameter, which contains the embedded custom claims.
OIDC Tokens and User Session	
Revoke Token Upon User Session Invalidation	If this attribute is set to "true", then the invalidation of a user session would also invalidate the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited. When the OIDC tokens (access token, refresh token, and authorization code) are created, they are linked to the particular user session used for the authorization. When that session is invalidated, the OIDC tokens linked to it also get invalidated. Default: false
Invalidate User Session Upon Token Revocation	If this attribute is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. The OIDC tokens (access token, refresh token) are generated based on an active user session. When a token is revoked, its corresponding user session would also get invalidated. Default: false
Client Secret	

Attribute Name	Description
Client Secret Validity Period (in minutes)	Sets the validity period of the previous client secret after a client secret reset is triggered. During this period, both the previous client secret and the new client secret will be usable. Set the value to "0" to immediately invalidate the previous client secret upon a client secret reset. Value should be from 0 to 525600. Default: 0
OpenID Connect Tab	
Default Authentication Expiry (in seconds)	Authentication expiry is used to check against user login session.
Attribute claim value Response configuration	This attribute value has to be a user response object to include additional claims of the user in ID token without user consent.
Return ID Token in JWE Format	Setting to "true" will return the ID token as a JWE token. For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: false
User Consent	
Enable auto saving of user consent after OAuth Authentication	Enables the granting of consent for requested scopes by the authorization endpoint without the actual user's consent. This behavior may be a potential security loophole and is hence set to "false" by default. It should only be set to "true" if disabling this breaks the existing handling of a customized consent page that still relies on the old behavior where this feature is enabled. Default: false
Revoke Consented Scopes When Revoking Token	Actions on consented scope(s) of user upon refresh token revocation. Options: ○ "Revoke All" ○ "Keep"

Attribute Name	Description
	Default: Keep
Terms Of Service URL	URL of the terms of service agreement.
Privacy Policies URL	URL for the statement or declaration that explicitly details the company policy regarding how it handles user and customer information.
Policy	
Application Policy	The application policy that will be applied to the OAuth client. This allows the OAuth client to whitelist specific authentication or reauthentication methods for users. This setting may be required if: ○ Enable User Token Revocation with Security Events attribute has been set to "true" in the OAuth Authz Server. ○ CLP is required to enforce authentication using only whitelisted authentication mechanisms configured in the policy. Note: For more details on how to configure the policy, refer to the UAM Administration Guide - Application Policy Configuration.

9. Save the changes to complete the setup.

i **Note:** You may need to refer to the **OAuth client_id** attribute and click on the **OAuth Client Secret** button to obtain the client secret when configuring your trusted OAuth client application.

Configure the Authorization Code Validity Time

Refer to the steps in [Configure the Authorization Code Validity Time](#).

Enable Client Credentials Flow

Refer to the steps in [Enable Client Credentials Flow](#).

Configure the Application to Use the Authorization Code Flow

To obtain an authorization code, there is a need to send a request using the application to the authorization server's authorize endpoint (/auth).

The authorization server supports the use of the **HTTP GET** and **POST** methods at the authorization endpoint.

- If the **HTTP GET** method is used, the request parameters are serialized using URI Query String Serialization.
- If the **HTTP POST** method is used, the request parameters are serialized using Form Serialization.

If using the AccessMatrix UAM default authorization server (a.k.a. OAuth Proxy), then the request URL should be similar to the example below:

Using HTTP GET:

GET

GET Request Sample

Code

```
GET /auth?
    response_type=code
    &scope=openid
    &client_id=s6BhdRkqt3
    &redirect_uri=https%3A%2F%2Fclient.example.org%2Fcb
    &state=af0ifjsldkj HTTP/1.1
Host: server.example.com
```

Using HTTP POST:

POST

POST Request Sample

Code

```
POST /auth HTTP/1.1
Host: server.example.com
Content-Type: application/x-www-form-urlencoded

    response_type=code
    &scope=openid
```

```
&client_id=s6BhdRkqt3
&redirect_uri=https%3A%2F%2Fclient.example.org%2Fcb
&state=af0ifjsldkj
```

Below are the parameters being passed:

Parameter	Description
client_id	The OAuth Client Id of the created OAuth client in Admin Console. Refer to Configure the OAuth Client .
response_type	Value is "code". It indicates that the token endpoint (/token) should return an ID token.
scope	Value is "openid". It indicates that the token endpoint (/token) should return an ID token.
redirect_uri	The callback location where the user-agent will be redirected to along with the code. The OAuth Redirect URIs attribute value specified in Configure the OAuth Client must contain this URI.
state	A random alphanumeric string that the authorization server will reproduce when redirecting the user-agent back to the client. This is used to help prevent cross-site request forgery.



Note:

- For more information on these parameters, see the [OpenID Connect Endpoints](#) section for reference.
- If the user does not have an existing session, the HTTP request will open the login page.
- If the user has an existing session or after the user is authenticated, then the user will be redirected to the specified redirect_uri along with a code. The parameters are returned as query parameters to the redirect_uri specified in the authorization request using the application/x-www-form-urlencoded format:

Code

```
HTTP/1.1 302 Found
Location: https://client.example.org/cb?
code=Splxl0BeZQQYbYS6WxSbIA
&state=af0ifjsldkj
```

This code validity is based on the value specified in [Configure the Authorization Code Validity Time](#), during which it can be exchanged for tokens.

Exchange the Code for Tokens

Refer to the steps in [Exchange the Code for Tokens](#).

Validate Response

- To validate tokens, refer to [C. Validating JWTs](#).
- To validate state, check that the `state` value returned from the OIDC response matches with the `state` value issued by the application.

Configure the Application to Use the Client Credentials Flow

Refer to [Configure the Application to Use the Client Credentials Flow](#) under the **Client Credentials** tab for details on how to use the Client Credentials flow.

Access Protected Resources

- Use the APIs described [here](#) to validate access token prior to accessing the protected resource.
- Use access token to access APIs (e.g. protected resource). Refer to [APIs Secured by Access Token](#).

Implicit



Note: Based on the new OAuth best practice guidelines and OAuth 2.1 draft specification, it is recommended that Single-Page Applications (SPAs) use the Authorization Code with PKCE flow instead of the Implicit flow.

This section contains the steps for the Implicit flow implementation for single-page applications.

- [Configure the OAuth Client \(Implicit\)](#)
 - [Configure the Application to Use the Implicit Flow](#)
 - [Validate Response](#)
 - [Access Protected Resources](#)
-

Configure the OAuth Client (Implicit)

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Implicit Flow** button and click **Next >**.

i **Note:** If needed, you can click the **Flowchart for Choosing OAuth/OIDC Flow** button for more information on selecting the appropriate flow type.

3. Click **Next >**.

i **Note:** If needed, you can expand the **About this flow** section to find out more about this flow.

4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
OAuth Client Id	Unique Id of OAuth client.
OAuth Client Name	Name of OAuth client.
Description	Description of OAuth client.

5. Fill in the basic setting information:

Attribute Name	Description
Grant Type	Value is "implicit". (uneditable)
Response Type	Value is "token", "id_token", or "token id_token". (uneditable)
Use OpenID Connect	This attribute is checked by default. (uneditable)
OAuth Redirect URIs	Specifies the URI that the end users will be redirected to, along with the access token returned by the OAuth authorization server. (mandatory)
Consent rule	Indicates if the OAuth client can be trusted or not. Options: ○ "TRUSTED" ○ "NOT_TRUSTED"

Attribute Name	Description
*Value to return as Subject (sub) value	Indicates where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this OAuth client. This setting will be used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. (mandatory) Options: ○ "Id" ○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account" ○ "User Response"
User Response to return as Subject (sub) value	This attribute is only required if the Value to return as Subject (sub) attribute is configured with "User Response". This allows other alternatives to retrieve a user identity (using user response's predefined field/attributes, or by customization of user response). Options: ○ "GroupMembership" ○ "OIDC_address" ○ "OIDC_email" ○ "OIDC_phone" ○ "OIDC_profile"

- Click the **Finish** button. If the OAuth client creation was successful, the **Create OAuth Client** page will be populated with the information configured in the **OAuth Client Configuration Wizard**.
- If your OAuth client is in a domain different from the OIDC endpoints, it is mandatory to whitelist your origin using the **JavaScript Web Origin URIs** configuration.
- (OPTIONAL) You may fill in or modify the values of the following attributes.

Attribute Name	Description
Status	To activate or disable the OAuth client. ○ "Activated" means OAuth client is enabled. ○ "Disabled" means AccessMatrix will reject authorization and token requests from this OAuth

Attribute Name	Description
	client. Existing tokens (e.g. access token) will also be rejected on best-case basis when verification is done by AccessMatrix. Default: Activated
Attributes Tab	
JavaScript Web Origin URIs	This configuration is mandatory if the OAuth client is a browser client that needs to have cross origin access to OIDC endpoints as explained here. It is also applicable if the AccessMatrix OIDC JavaScript library is used by the OAuth client to verify session. Note: This list of web origins must be in a plain domain name and protocol (i.e. http/https). Non-default port (i.e. 8443 or 8080) must be specified. Path and wildcard are not supported. Example of valid URI: https://oidc.example.com
Required Authentication Level	This is to indicate the minimum authentication level of the authenticated user required by this OAuth client. Default: 1
Whitelisted Scopes (Press Ctrl + Click to select multiple options)	Determines which scope(s) are allowed to be returned when the OAuth client calls the /auth endpoint with scope(s) specified. Note:The list of scopes available for selection is based on the list of scopes configured in the Edit OAuth Authz Server page. Refer to Scope and User Consent for more information.
Embed Session Token	This is to embed session token in authorization code, refresh token or ID token, if applicable. If this attribute is enabled, the expiry of ID token, access token, and refresh token would be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting. For refresh token, the expiry is Session Idle Time minus 60 seconds. Default: false
Allow All Users to Access	Enabling this feature will allow all authenticated users to access this OAuth client. For advanced user access control, this feature must be disabled and additional configuration should be performed through the OAuth Client Privilege tab in the user's View User page to define which OAuth

Attribute Name	Description
	client is accessible for that particular user. Default: true
Allow Multiple Profiles	This is to allow a user to have multiple login profiles (secondary account) for the same application. Default: false
Logo URI	The URI that references a logo for the client application.
JWKS Endpoint URL	URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.
JWKS String	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. JWKS String takes precedence over JWKS Endpoint URL if both are specified.
Request Object	
Request Object Signing Algorithm	Specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is specified, the OAuth client is allowed to use any of the supported algorithms can be used for Request Object signing. Options: ○ "RS256" ○ "RS384" ○ "RS512" ○ "ES256" ○ "ES384" ○ "ES512"
Access Token	
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200. Default: 3600
Access Token Format	Specifies the format of the access token. Options: ○ "Opaque" ○ "JWT"

Attribute Name	Description
	○ "JWE" For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: Opaque
Access Token User Response	User response to be returned in access token. Applicable for JWT and JWE format.
JSON Web Encryption (JWE)	
JWE Algorithm	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Options: ○ "RSA-OAEP-256" ○ "ECDH-ES+A256KW"
JWE Encryption Algorithm	Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Options: ○ "A256CBC-HS512" ○ "A256GCM"
OIDC Tokens and User Session	
Revoke Token Upon User Session Invalidation	If this attribute is set to "true", then the invalidation of a user session would also invalidate the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited. When the OIDC tokens (access token) are created, they are linked to the particular user session used for the authorization. When that session is invalidated, the OIDC tokens linked to it also get invalidated. Default: false
Invalidate User Session Upon Token Revocation	If this attribute is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. The OIDC tokens (access token) are generated based on an active user session. When a token is revoked, its

Attribute Name	Description
	corresponding user session would also get invalidated. Default: false
OpenID Connect Tab	
Default Authentication Expiry (in seconds)	Authentication expiry is used to check against user login session.
Attribute claim value Response configuration	This attribute value has to be a user response object to include additional claims of the user in ID token without user consent.
Return ID Token in JWE Format	Setting to "true" will return the ID token as a JWE token. For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: false
User Consent	
Enable auto saving of user consent after OAuth Authentication	Enables the granting of consent for requested scopes by the authorization endpoint without the actual user's consent. This behavior may be a potential security loophole and is hence set to "false" by default. It should only be set to "true" if disabling this breaks the existing handling of a customized consent page that still relies on the old behavior where this feature is enabled. Default: false
Revoke Consented Scopes When Revoking Token	Actions on consented scope(s) of user upon refresh token revocation. Options: ○ "Revoke All" ○ "Keep" Default: Keep
Terms Of Service URL	URL of the terms of service agreement.

Attribute Name	Description
Privacy Policies URL	URL for the statement or declaration that explicitly details the company

9. Save the changes to complete the setup.

Configure the Application to Use the Implicit Flow

This flow is very similar to the Authorization Code flow except that the `response_type` is "token" and/or "id_token" instead of "code".

The authorization server supports the use of the **HTTP GET** and **POST** methods at the authorization endpoint.

- If the **HTTP GET** method is used, the request parameters are serialized using URI Query String Serialization.
- If the **HTTP POST** method is used, the request parameters are serialized using Form Serialization.

The client browser makes a request to the authorization server's authorize endpoint (/auth). If using the AccessMatrix UAM default authorization server (a.k.a. OAuth Proxy), then the request URL should be similar to the example below:

Using HTTP GET:

GET

GET Request Sample

Code

```
GET /authorize?
  response_type=id_token%20token
  &client_id=s6BhdRkqt3
  &redirect_uri=https%3A%2F%2Fclient.example.org%2Fcb
  &scope=openid%20profile
  &state=af0ifjsldkj
  &nonce=n-0S6_WzA2Mj HTTP/1.1
Host: server.example.com
```

Below are the parameters being passed:

Parameter	Description
client_id	The "OAuth Client Id" of the created OAuth client in Admin Console. Refer to Configure the OAuth Client .
response_type	Value is token. It can also be "id_token" or both.
scope	Value is "openid". It indicates that the token endpoint (/token) will return an ID token. Other additional scopes can also be requested.
redirect_uri	The callback location where the user-agent will be redirected to along with the access_token. The OAuth Redirect URIs attribute value specified in Configure the OAuth Client must contain this URI.
state	A random alphanumeric string that the authorization server will reproduce when redirecting the user-agent back to the client. This is used to help prevent cross-site request forgery.
nonce	The value that will be returned in the ID token. It is used to mitigate replay attacks.



Note:

- For more information on these parameters, see the [OpenID Connect Endpoints](#) section for reference.
- If the user does not have an existing session, the HTTP request will open the login page.
- If the user has an existing session or after the user is authenticated, then the user will be redirected to the specified redirect_uri along with a token as a hash fragment (URI with Fragment):

Code

```
HTTP/1.1 302 Found
  Location: https://client.example.org/cb#
    access_token=SlAV32hkKG
    &token_type=bearer
    &id_token=eyJ0 ... NiJ9.eyJlc ... I6IjIifX0.DeWt4Q
... ZXso
    &expires_in=3600
    &state=af0ifjsldkj
```

The application must now extract the token(s) from the URI and store them.

Validate Response

To validate tokens, refer to [C. Validating JWTs](#).

Access Protected Resources

- Use the APIs described [here](#) to validate access token prior to accessing the protected resource.
- Use access token to access APIs (e.g. protected resource). Refer to [APIs Secured by Access Token](#).

Resource Owner Password Credentials

i **Note:** Based on the new OAuth best practice guidelines and the latest OAuth 2.1 draft specification, the Resource Owner Password Credentials flow is deemed highly insecure and should not be used for new applications. It is recommended that you use the Authorization Code with PKCE flow instead.

This section contains the steps for the Resource Owner Password Credentials flow implementation for trusted applications.

- [Configure the OAuth Client \(Resource Owner Password Credentials\)](#)
- [Configure the Application to Use the Resource Owner Password Credentials Flow](#)
- [Validate Response](#)
- [Access Protected Resources](#)

Configure the OAuth Client (Resource Owner Password Credentials)

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Resource Owner Password Credentials Flow** button and click **Next >**.

i **Note:** If needed, you can click the **Flowchart for Choosing OAuth/OIDC Flow** button for more information on selecting the appropriate flow type.

3. Click **Next >**.

i **Note:** If needed, you can expand the **About this flow** section to find out more about this flow.

4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
OAuth Client Id	Unique Id of OAuth client.

Attribute Name	Description
OAuth Client Name	Name of OAuth client.
Description	Description of OAuth client.

5. Fill in the basic setting information:

Attribute Name	Description
Grant Type	Value is "password". (uneditable)
Token endpoint authentication method	This method is preconfigured with "ClientSecret". This means that the server application is required to present client secret for client authentication when calling /token endpoint. However, by specifying "none", this allows the application to bypass client authentication that would make it unsecure. Use your discretion in setting this value and beware of the security implication.
Use OpenID Connect	This attribute is checked by default. (uneditable)
*Value to return as Subject (sub) value	Indicate where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this OAuth client. This setting will be used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. (mandatory) Options: ○ "Id" ○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account" ○ "User Response"
User Response to return as Subject (sub) value	This attribute is only required if the Value to return as Subject (sub) attribute is configured with "User Response". This allows other alternatives to retrieve a user identity (using user response's predefined field/attributes, or by customization of user response). Options: ○ "GroupMembership" ○ "OIDC_address" ○ "OIDC_email" ○ "OIDC_phone"

Attribute Name	Description
	○ "OIDC_profile"

6. Click the **Finish** button. If the OAuth client creation was successful, the **Create OAuth Client** page will be populated with the information configured in the **OAuth Client Configuration Wizard**.

7. (OPTIONAL) You may also fill in or modify the values of the following attributes:

Attribute Name	Description
Status	To activate or disable the OAuth client. ○ "Activated" means OAuth client is enabled. ○ "Disabled" means AccessMatrix will reject authorization and token requests from this OAuth client. Existing tokens (e.g. access token) will also be rejected on best-case basis when verification is done by AccessMatrix. Default: Activated
Attributes Tab	
Required Authentication Level	This is to indicate the minimum authentication level of the authenticated user required by this OAuth client. Default: 1
Whitelisted Scopes (Press Ctrl + Click to select multiple options)	Determines which scope(s) are allowed to be returned when the OAuth client calls the /token endpoint with scope(s) specified. Note: The list of scopes available for selection is based on the list of scopes configured in the Edit OAuth Authz Server page. Refer to Scope and User Consent for more information.
Embed Session Token	This is to embed session token in refresh token or ID token, if applicable. If this attribute is enabled, the expiry of ID token, access token, and refresh token would be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting. For refresh token, the expiry is Session Idle Time minus 60 seconds. Default: false
Allow All Users to Access	Enabling this feature will allow all authenticated users to access this OAuth client. For advanced user access control, this feature must be disabled and additional configuration

Attribute Name	Description
	should be performed through the OAuth Client Privilege tab in the user's View User page to define which OAuth client is accessible for that particular user. Default: true
Allow Multiple Profiles	This is to allow user to have multiple login profiles (secondary account) for the same application. Default: false
JWKS Endpoint URL	URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.
JWKS String	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. JWKS String takes precedence over JWKS Endpoint URL if both are specified.
Access Token	
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200. Default: 3600
Access Token Format	Specifies the format of the access token. Options: ○ "Opaque" ○ "JWT" ○ "JWE" For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: Opaque
Access Token User Response	User response to be returned in access token. Applicable for JWT and JWE format.
JSON Web Encryption (JWE)	
JWE Algorithm	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Options: ○ "RSA-OAEP-256"

Attribute Name	Description
	○ "ECDH-ES+A256KW"
JWE Encryption Algorithm	Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Options: ○ "A256CBC-HS512" ○ "A256GCM"
Refresh Token	
Return Refresh Token	Indicates whether the refresh token should be returned. Default: false
Refresh Token Expiry (in minutes)	Sets the validity period of a refresh token. Value should be from 10 to 525600. Default: 1440
Refresh Token Lifetime (in hours)	Lifetime expiry for the refresh token, starting from when it is first generated. Refresh token can be renewed up until this expiry. "0" means no lifetime expiry. Value should be from 0 to 1440. Default: 168 hours
OIDC Tokens and User Session	
Revoke Token Upon User Session Invalidation	If this attribute is set to "true", then the invalidation of a user session would also invalidate the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited. When the OIDC tokens (access token, refresh token) are created, they are linked to the particular user session used for the authorization. When that session is invalidated, the OIDC tokens linked to it also get invalidated. Default: false
Invalidate User Session Upon Token Revocation	If this attribute is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. The OIDC tokens (access token, refresh token) are generated based on an active user session. When a token is revoked, its corresponding user session would also get invalidated. Default: false
Client Secret	

Attribute Name	Description
Client Secret Validity Period (in minutes)	Sets the validity period of the previous client secret after a client secret reset is triggered. During this period, both the previous client secret and the new client secret will be usable. Set the value to "0" to immediately invalidate the previous client secret upon a client secret reset. Value should be from 0 to 525600. Default: 0
OpenID Connect Tab	
Attribute claim value Response configuration	This attribute value has to be a user response object to include additional claims of user in ID token without user consent.
Return ID Token in JWE Format	Setting to "true" will return the ID token as a JWE token. For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: false

8. Save the changes to complete the setup.



Note: You may need to refer to the **OAuth client_id** attribute and click on the **OAuth Client Secret** button to obtain the client secret when configuring your trusted OAuth client application.

Configure the Application to Use the Resource Owner Password Credentials Flow

Before implementing this flow, there is a need to capture the user's password in any manner you deem suitable. Once you have collected the credentials, all that is required is a single API call to the /token endpoint:

POST

POST Request Sample

Code

```
POST /oauth-proxy/token HTTP/1.1
Host: {yourDefaultOAuthProxyDomain}.com
```

```
Content-Length: 307
Content-Type: application/x-www-form-urlencoded
Authorization: Basic czZC.....
grant_type=password&username=yourusername&password=password&scope=openid
```



Important Note: The call to the /token endpoint requires authentication. In this case, it is the basic authentication of the client ID and Secret. You can find the client ID and secret under the OAuth Client basic information section.

Below are the parameters being passed:

Parameter	Description
grant_type	The value is "password".
scope	The value is "openid" which means that the /token endpoint will return an ID token. Other additional scopes can also be requested.
username	The username of a user registered in AccessMatrix UAM server.
password	The password of the user registered in AccessMatrix UAM server.

Validate Response

To validate tokens, refer to [C. Validating JWTs](#).

Access Protected Resources

- Use the APIs described [here](#) to validate access token prior to accessing the protected resource.
- Use access token to access APIs (e.g. protected resource). Refer to [APIs Secured by Access Token](#).

Hybrid



Note: Based on the new OAuth best practice guidelines and the latest OAuth 2.1 draft specification, the PKCE challenge must be used with the Authorization Code grant whenever possible. It is recommended that you use the Hybrid with PKCE flow instead.

This section contains the steps for the Hybrid flow implementation for hybrid applications. A hybrid application refers to an application with the ability to securely store client secrets that requires immediate access to user info.

-
- [Configure the OAuth Client \(Hybrid\)](#)
 - [Configure the Authorization Code Validity Time](#)
 - [Configure the Application to Use the Hybrid Flow](#)
 - [Exchange the Code for Tokens](#)
 - [Validate Response](#)
 - [Configure the Application to Use Client Credentials Flow](#)
 - [Access Protected Resources](#)

Configure the OAuth Client (Hybrid)

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client** and click the **Create** button. The **OAuth Client Configuration Wizard** is displayed.
2. Select the **Hybrid Flow** radio button and click **Next >**.



Note: If needed, you can click the **Flowchart for Choosing OAuth/OIDC Flow** button for more information on selecting the appropriate flow type.

3. Click **Next >**.



Note: If needed, you can expand the **About this flow** section to find out more about this flow.

4. Fill in the OAuth client information and click **Next >**:

Attribute Name	Description
OAuth Client Id	Unique Id of OAuth client.
OAuth Client Name	Name of OAuth client.
Description	Description of OAuth client.
Allow client_credentials Grant	When enabled, this allows the OAuth Client to also make use of the Client Credentials grant type (i.e. the access token can be generated without an authorization code) after valid client authentication. Default: false

5. Fill in the basic setting information:
-

Attribute Name	Description
Grant Type	Value is "authorization_code". (uneditable)
Token endpoint authentication method	Specifies the authentication method used to obtain and exchange the authorization code for an access token. The default value is "ClientSecret". If "PrivateKeyJWT" is selected, ensure that either the JWKS Endpoint URL or the JWKS String attribute value is specified.
Response Type	Indicates the additional token(s) to be included with the authorization code in a successful response to OAuth authentication requests using the Authorization endpoint or API. A successful response includes the authorization code and ID token. Note: Providing access tokens on the first response is insecure. Hence, response type values containing "token" are not available. Default: code id_token
Use OpenID Connect	This attribute is checked by default. (uneditable)
OAuth Redirect URIs	Specifies the URI that the end users will be redirected to, along with the authorization code returned by the OAuth authorization server. (mandatory)
Consent rule	Indicates if the OAuth client can be trusted or not. Options: ○ "TRUSTED" ○ "NOT_TRUSTED"
*Value to return as Subject (sub) value	Indicates where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this OAuth client. This setting will be used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. (mandatory) Options: ○ "Id" ○ "Name" ○ "DN" ○ "Email" ○ "Secondary Account"

Attribute Name	Description
	"User Response"
User Response to return as Subject (sub) value	This attribute is only required if the Value to return as Subject (sub) attribute is configured with "User Response". This allows other alternatives to retrieve a user identity (using user response's predefined field/attributes, or by customization of user response).

- Click the **Finish** button. If the OAuth client creation was successful, the **Create OAuth Client** page will be populated with the information configured in the **OAuth Client Configuration Wizard**.
- If your OAuth client is in a domain different from the OIDC endpoints, it is mandatory to whitelist your origin using the **JavaScript Web Origin URIs** configuration.
- (OPTIONAL) You may also fill in or modify the values of the following attributes:

Attribute Name	Description
Status	To activate or disable the OAuth client. "Activated" means OAuth client is enabled. "Disabled" means AccessMatrix will reject authorization and token requests from this OAuth client. Existing tokens (e.g. access token) will also be rejected on best-case basis when verification is done by AccessMatrix. Default: Activated
Attributes Tab	
JavaScript Web Origin URIs	This configuration is mandatory if the OAuth client is a browser client that needs to have cross origin access to OIDC endpoints as explained here. It is also applicable if the AccessMatrix OIDC JavaScript library is used by the OAuth client to verify session. Note: This list of web origins must be in a plain domain name and protocol (i.e. http/https). Non-default port (i.e. 8443 or 8080) must be specified. Path and wildcard are not supported. Example of valid URI: https://oidc.example.com
Required Authentication Level	This is to indicate the minimum authentication level of the authenticated user required by this OAuth client. Default: 1

Attribute Name	Description
Whitelisted Scopes (Press Ctrl + Click to select multiple options)	Determines which scope(s) are allowed to be returned when the OAuth client calls the /token or /auth endpoint with scope(s) specified. Note: The list of scopes available for selection is based on the list of scopes configured in the Edit OAuth Authz Server page. Refer to Scope and User Consent for more information.
Embed Session Token	This is to embed the session token in the authorization code, refresh token or ID token, if applicable. If this attribute is enabled, the expiry of ID token, access token, and refresh token will be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting . For refresh token, the expiry is Session Idle Time minus 60 seconds. Default: false
Allow All Users to Access	Enabling this feature will allow all authenticated users to access this OAuth client. For advanced user access control, this feature must be disabled and additional configuration should be performed through the OAuth Client Privilege tab in the user's View User page to define which OAuth client is accessible for that particular user. Default: true
Allow Multiple Profiles	This is to allow a user to have multiple login profiles (secondary account) for the same application. Default: false
Logo URI	The URI that references a logo for the client application.
JWKS Endpoint URL	URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.
JWKS String	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. JWKS String takes precedence over JWKS Endpoint URL if both are specified.
URL to initiate SSO	Note: It is required only by SSO portal or custom IDP portal that lists user's entitled OIDC RP application(s).
Request Object	
Request Object Signing Algorithm	Specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is

Attribute Name	Description
	specified, the OAuth client is allowed to use any of the supported algorithms for Request Object signing. Options: ○ "RS256" ○ "RS384" ○ "RS512" ○ "ES256" ○ "ES384" ○ "ES512"
Authorization Code	
Authorization Code Format	Specifies the format for the authorization Code. If specified as "JWT", the authorization code will be returned as JWT, where additional custom claims can be embedded based on the Authorization Code User Response attribute. Options: ○ "Opaque" ○ "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_code parameter containing the authorization code.
Authorization Code User Response	User response to be embedded in the JWT as custom claims. The Authorization Code Format attribute must be specified as "JWT". Note: The JWT will be returned in the code parameter, which contains the additional embedded custom claims.
Access Token	
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200. Default: 3600

Attribute Name	Description
Access Token Format	Specifies the format of the access token. Options: ○ "Opaque" ○ "JWT" ○ "JWE" For JWE format, ensure that: ○ The JWE Algorithm and JWE Encryption Algorithm attributes are configured. ○ Either the JWKS Endpoint URL or JWKS String attribute value is specified. Default: Opaque
Access Token User Response	User response to be returned in the access token. Applicable for JWT and JWE format.
JSON Web Encryption (JWE)	
JWE Algorithm	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Options: ○ "RSA-OAEP-256" ○ "ECDH-ES+A256KW"
JWE Encryption Algorithm	Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Options: ○ "A256CBC-HS512" ○ "A256GCM"
Refresh Token	
Return Refresh Token	Indicates whether the refresh token should be returned. Default: false
Refresh Token Expiry (in minutes)	Sets the validity period of a refresh token. Value should be from 10 to 525600. Default: 1440
Refresh Token Lifetime (in hours)	Lifetime expiry for the refresh token, starting from when it is first generated. Refresh token can be renewed up until this expiry. "0" means no lifetime expiry.

Attribute Name	Description
	Value should be from 0 to 1440. Default: 168 hours
Use Refresh Token Integration Date	When set to "true", the <code>integrationDate</code> request parameter passed to the token endpoint will be used as the grant time of the refresh token (where grant time refers to the starting time of a refresh token's validity). The refresh token expiry will also be made the same as the refresh token lifetime, ignoring the value specified in the Refresh Token Expiry attribute such that: - Refresh token lifetime expiry = integration date + Refresh Token Lifetime attribute value - Refresh token expiry = integration date + Refresh Token Lifetime attribute value Note: This is for SGFinDex. When set to "false", the generation date of the first refresh token will be used as the grant time of the refresh token and this will be used to compute the refresh token lifetime. In this case, the refresh token expiry will always start from the generation date of the refresh token - provided it is still within the refresh token lifetime - and the expiry time will be computed based on the Refresh Token Expiry attribute, such that: - Refresh token lifetime expiry = first refresh token's generation time + Refresh Token Lifetime attribute value - Refresh token expiry = refresh token's generation time + Refresh Token Expiry attribute value Default: false
Refresh Token Format	Specifies the format for the refresh token. If specified as "JWT", the refresh token will be returned as JWT, where additional custom claims can be embedded based on the Refresh Token User Response attribute. Options: "Opaque" "JWT"

Attribute Name	Description
	Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_rt parameter containing the refresh token.
Refresh Token User Response	User response to be embedded in the JWT as custom claims. The Refresh Token Format attribute must be specified as "JWT". Note: The JWT will be returned in the code parameter, which contains the embedded custom claims.
OIDC Tokens and User Session	
Revoke Token Upon User Session Invalidation	If this attribute is set to "true", then the invalidation of a user session would also invalidate the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited. When the OIDC tokens (access token, refresh token, and authorization code) are created, they are linked to the particular user session used for the authorization. When that session is invalidated, the OIDC tokens linked to it also get invalidated. Default: false
Invalidate User Session Upon Token Revocation	If this attribute is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. The OIDC tokens (access token, refresh token) are generated based on an active user session. When a token is revoked, its corresponding user session would also get invalidated. Default: false
Client Secret	
Client Secret Validity Period (in minutes)	Sets the validity period of the previous client secret after a client secret reset is triggered. During this period, both the previous client secret and the new client secret will be usable. Set the value to "0" to immediately invalidate the previous client secret upon a client secret reset. Value should be from 0 to 525600. Default: 0
OpenID Connect Tab	

Attribute Name	Description
Default Authentication Expiry (in seconds)	Authentication expiry is used to check against user login session.
Attribute claim value Response configuration	This attribute value has to be a user response object to include additional claims of the user in ID token without user consent.
Return ID Token in JWE Format	Setting to "true" will return the ID token as a JWE token. For JWE format: - Ensure that JWE Algorithm and JWE Encryption Algorithm attributes are configured. - Either JWKS Endpoint URL or JWKS String attribute value is specified. Default: false
User Consent	
Enable auto saving of user consent after OAuth Authentication	Enables the granting of consent for requested scopes by the authorization endpoint without the actual user's consent. This behavior may be a potential security loophole and is hence set to "false" by default. It should only be set to "true" if disabling this breaks the existing handling of a customized consent page that still relies on the old behavior where this feature is enabled. Default: false
Revoke Consented Scopes When Revoking Token	Actions on consented scope(s) of user upon refresh token revocation. Options: "Revoke All" "Keep" Default: Keep
Terms Of Service URL	URL of the terms of service agreement.
Privacy Policies URL	URL for the statement or declaration that explicitly details the company policy regarding how it handles user and customer information.
Policy	

Attribute Name	Description
Application Policy	The application policy that will be applied to the OAuth client. This allows the OAuth client to whitelist specific authentication or reauthentication methods for users. This setting may be required if: ○ Enable User Token Revocation with Security Events attribute has been set to "true" in the OAuth Authz Server. ○ CLP is required to enforce authentication using only whitelisted authentication mechanisms configured in the policy. Note: For more details on how to configure the policy, refer to the UAM Administration Guide - Application Policy Configuration.

9. Save the changes to complete the setup.

i Note: You may need to refer to the **OAuth client_id** attribute and click on the **OAuth Client Secret** button to obtain the client secret when configuring your trusted OAuth client application.

Configure the Authorization Code Validity Time

Refer to the steps in [Configure the Authorization Code Validity Time](#).

Configure the Application to Use the Hybrid Flow

This flow is similar to the Authorization Code flow, except that the `response_type` is multi-valued, where its value is a combination of "code" and "id_token".

i Note: Providing access tokens on the first response is not secure. As such, response type values that contain "token" are not supported.

The authorization server supports the use of the **HTTP GET** and **POST** methods at the authorization endpoint.

- If the **HTTP GET** method is used, the request parameters are serialized using URI Query String Serialization.

-
- If the **HTTP POST** method is used, the request parameters are serialized using Form Serialization.

If using the AccessMatrix UAM default authorization server (a.k.a. OAuth Proxy), then the request URL should be similar to the example below:

Using HTTP GET:

GET

GET Request Sample

Code

```
GET /auth?
  response_type=code id_token
  &scope=openid profile
  &client_id=s6BhdRkqt3
  &redirect_uri=https://client.example.org/
  &state=af0ifjsldkj
  &nonce=n-0S6_WzA2Mj HTTP/1.1
Host: server.example.com
```

Using HTTP POST:

POST

POST Request Sample

Code

```
POST /auth HTTP/1.1
Host: server.example.com
Content-Type: application/x-www-form-urlencoded

response_type=code id_token
&scope=openid profile
&client_id=s6BhdRkqt3
&redirect_uri=https://client.example.org/
&state=af0ifjsldkj
&nonce=n-0S6_WzA2Mj
```

Below are the parameters being passed:

Parameter	Description
client_id	The OAuth Client Id of the created OAuth client in Admin Console. Refer to Configure the OAuth Client .

Parameter	Description
response_type	Value is "code" with "token" and/or "id_token". The endpoint should return the authorization code plus access_token and/or ID token.
scope	Value is "openid". It indicates that the token endpoint (/token) will return an ID token. Other additional scopes can also be requested.
redirect_uri	The callback location where the user-agent will be redirected to along with the code. The OAuth Redirect URIs attribute value specified in Configure the OAuth Client must contain this URI.
state	A random alphanumeric string that the authorization server will reproduce when redirecting the user-agent back to the client. This is used to help prevent cross-site request forgery.
nonce	The value that will be returned in the ID token. It is used to mitigate replay attacks.



Note:

- For more information on these parameters, see the [OpenID Connect Endpoints](#) section for reference.
- If the user does not have an existing session, the HTTP request will open the login page.
- If the user has an existing session, or after the user is authenticated, the user will be redirected to the specified redirect_uri along with a code and a token and/or id_token as a hash fragment:

Code

```
HTTP/1.1 302 Found
Location: https://client.example.org/cb#
  access_token=SlAV32hkKG
  &code=Sp1xl0BeZQQYbYS6WxSbIA
  &token_type=bearer
  &id_token=eyJ0 ... NiJ9.eyJ1c ... I6IjIifX0.DeWt4Q
... ZXso
  &expires_in=3600
  &state=af0ifjslkdj
```

This code validity is based on the value specified in [Configure the Authorization Code Validity Time](#). During this validity period, it can be exchanged for tokens.

Exchange the Code for Tokens

Refer to the steps in [Exchange the Code for Tokens](#).

Validate Response

To validate tokens, refer to [C. Validating JWTs](#).

Configure the Application to Use the Client Credentials Flow

Refer to [Configure the Application to Use the Client Credentials Flow](#) under the **Client Credentials** tab for details on how to use the Client Credentials flow.

Access Protected Resources

- Use the APIs described [here](#) to validate access token prior to accessing the protected resource.
- Use access token to access APIs (e.g. protected resource). Refer to [APIs Secured by Access Token](#).

6.3.2. Dynamic Client Registration

AccessMatrix UAM supports Dynamic Client Registration (DCR) with the AccessMatrix OAuth Proxy. DCR enables an OAuth client to be dynamically registered with the authorization server. UAM also supports DCR management, where the client can view and edit client configuration details after registering with the authorization server. DCR is available for both OAuth 2.0 and OAuth 2.1 flows.

DCR is supported for the following OAuth 2.0 flows:

- Authorization Code flow
- Implicit flow
- Client Credentials flow

DCR is supported for the following OAuth 2.1 flows:

- Hybrid With PKCE flow
- Authorization Code with PKCE flow

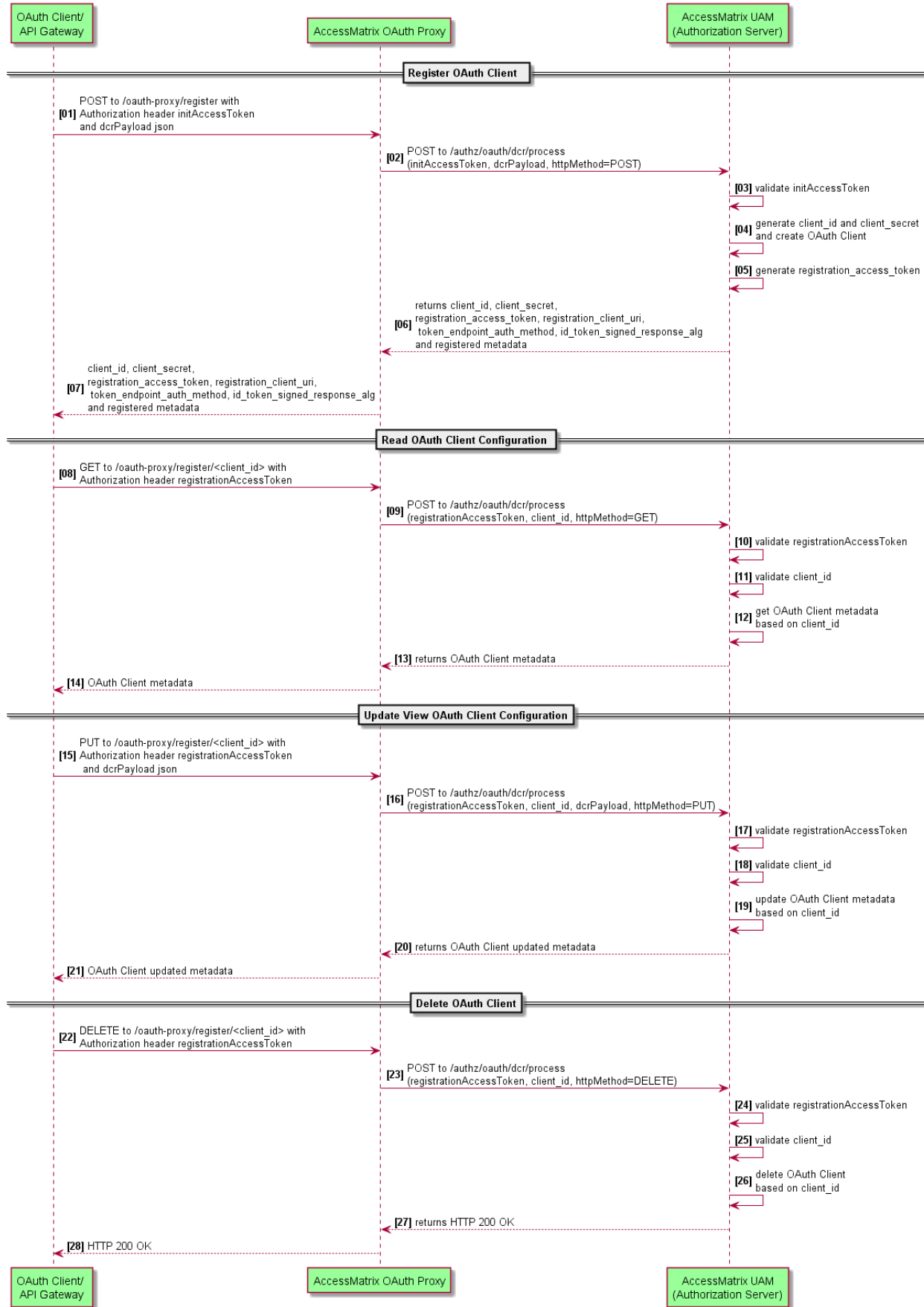


Note: If the grant type (flow) is unspecified, the OAuth client is created based on the Authorization Code flow requirements.

Sequence Diagram

The following diagram provides an overview of the typical DCR flow:

Dynamic Client Registration with AccessMatrix Authorization Server:



**Notes:**

- Initial access token refers to the access token issued to the OAuth client to allow it to initiate the DCR flow. This token does not expire.
- Registration access token refers to the access token issued to the OAuth client that allows it to access the client configuration endpoint to view and edit client configuration details. This token does not expire.

Register OAuth Client

Before initiating the DCR flow, the OAuth client is issued with an initial access token that is automatically generated by AccessMatrix.



Note: This value can be accessed in the Admin Console on the Dynamic Client Registration page by clicking the **Initial Access Token** button.

1. The Client sends an **HTTP POST** message to the `/register` or `/v2/register` endpoint of the OAuth Proxy with the initial access token and the relevant client metadata parameters needed for registration.



Note: The `/register` endpoint is used for OAuth 2.0 flows, while the `/v2/register` endpoint is used for OAuth 2.1 flows. For more details on the most appropriate endpoint to use, refer to [/v2/register vs /register](#).

2. The OAuth Proxy sends the request to the authorization server.
3. The authorization server validates the initial access token.
4. Upon successful validation, the authorization server issues the client a registration access token, and assigns the client a unique client identifier, as well as the associated metadata provided in the registration request with the issued client identifier. Optionally, a client secret will also be issued.
5. Upon successful validation, the authorization server issues and assigns the client with the following:
 - Registration access token
 - Unique client identifier
 - Associated metadata provided in the registration request with the issued client identifier.
 - (OPTIONAL) Client secret

Read OAuth Client

1. The OAuth client makes an **HTTP GET** request to the `/register` endpoint of the OAuth Proxy to read the current registration state of the client. The request includes the registration access token.
2. The OAuth Proxy sends the request to the authorization server.
3. The authorization server validates the registration access token and returns all of the relevant OAuth client metadata to the Client.

Update OAuth Client

1. The OAuth client makes an **HTTP PUT** request to the `/register` or `/v2/register` endpoint of the OAuth Proxy to update the registered client. The request includes the registration access token, along with all of the relevant OAuth client metadata.



Note: The `/v2/register` endpoint is used for updating OAuth clients from OAuth 2.0 to OAuth 2.1. This feature is only available to OAuth clients using Authorization Code Flow. For more details, refer to [Updating OAuth 2.0 to OAuth 2.1](#).

2. The OAuth Proxy sends the request to the authorization server.
3. The authorization server validates the registration access token and replaces all existing OAuth client metadata with the metadata provided in the update request.



Note: Any fields that are left unspecified in the request are treated as null values. This means that the values of the corresponding OAuth client attributes are replaced with null values.

4. The authorization server returns the updated OAuth client metadata to the Client.

Delete OAuth Client

1. The OAuth client makes an **HTTP DELETE** request to the `/register` endpoint of the OAuth Proxy to delete itself. The request includes the registration access token.
 2. The OAuth Proxy sends the request to the authorization server.
 3. The authorization server validates the registration access token and deletes the OAuth client. The client Id, client secret and registration access token will be invalidated.
-

Enable DCR

Before using any of the endpoints detailed in this page, DCR must first be enabled in the Admin Console.

Perform the following:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > Dynamic Client Registration**.
2. Click the **Initial Access Token** button and note down the value displayed.
3. Click the **Edit** button.
4. Set the value of the **Enable Dynamic Client Registration** attribute to "true".
5. Specify a value for the mandatory attributes ***Value to return as ID Token Subject (sub) value** and ***Registration URL**.
6. Specify the remaining attributes as needed.



Note: For more information on each attribute, refer to [Configurable DCR Attributes](#).

7. Click the **Save** button to save the changes.

Configurable DCR Attributes

Attribute Name	Description
Enable Dynamic Client Registration	Determines if DCR will be enabled. Default: false
Enable Initial Access Token for Dynamic Client Registration Management	Determines if the initial access token is used for Dynamic Client Registration management. • If set to "false", the authorization server verifies the OAuth client against the registration access token for Dynamic Client Registration management. • If set to "true", the authorization server first attempts to verify the client against the registration access token for Dynamic Client Registration management. If this fails, the server verifies the client against the initial access token instead. Default: false
Enable Regeneration of	Determines if registration access token regeneration is enabled.

Attribute Name	Description
Registration Access Token	- If set to "true", a new registration access token is generated for an OAuth client each time the OAuth client is updated via DCR. The old registration access token is invalidated. - If set to "false", an OAuth client's existing registration access token will never be regenerated. It will remain the same even after the OAuth client is updated via DCR. Default: false
Default Value to set to OAuth Client when created via DCR	
*Value to return as ID Token Subject (sub) value	This setting is used to resolve an end user identity for the OAuth client when generating the ID token.
User Response to return as ID Token Subject (sub) value	This attribute is only required if the Value to return as ID Token Subject (sub) value attribute is set to "User Response". This allows other alternatives to retrieve a user identity (e.g. using the user response's predefined field/attributes, or by customization of the user response).
Access Token Expiry (in seconds)	Sets the access token expiry time, in seconds. Value should be from 600 to 7200 seconds. It should not be less than 600.
Refresh Token Expiry (in minutes)	Sets the refresh token expiry time, in minutes. Value should be from 10 to 525600. It should not be less than 10. Default: 1440
User Consent Rule	Determines if the OAuth client can be trusted without a consent prompt.
*Registration URL	Client configuration endpoint. Example: https://i-sprint.com/oidc/register
Allow All Users to Access	Determines whether all users are authorized to access the OAuth client application by default. When set to "false", each user must be specifically authorized to access the application. Default: true

Create OAuth Client

HTTP POST to the /register API to create an OAuth client in the UAM server.



Note: HTTP POST to the /v2/register API to create an OAuth 2.1 client in the UAM server.

HTTP Method: POST

Path: \${yourOAuthProxyURL}/register

Header: Authorization Bearer <Initial_Access_Token>

To view the sample requests and responses for creating an OAuth client with a specific flow, you may refer to the following links:

- [Create - OAuth 2.0: Authorization Code](#)
- [Create - OAuth 2.0: Implicit](#)
- [Create - OAuth 2.0 : Client Credentials](#)
- [Create - OAuth 2.0 : Authorization Code with Client Credentials](#)
- [Create - OAuth 2.1: Mobile Native App](#)
- [Create - OAuth 2.1: Authorization Code with PKCE](#)
- [Create - OAuth 2.1: Hybrid with PKCE](#)

/v2/register vs /register

The /v2/register API endpoint is designed to distinguish between OAuth 2.0 and OAuth 2.1 for the Authorization code flow grant type. Please refer to the following table for the most appropriate URL to be used depending on the registration method and app type in use:

Registration Method	URL	Grant Type	App Type
Via grantType	/register	authorization_code	• Server Side App
Via grantType	/v2/register	authorization_code	• Mobile Native App • Authorization Code with PKCE • Hybrid With PKCE Note: If token_endpoint_auth_method contains "none", app_type "Mobile Native App" will be selected. If response_types contain "code id_token", app_type "Hybrid With PKCE" will be selected.

Registration Method	URL	Grant Type	App Type
Via appType	/register or /v2/register	N/A	• Server Side App • Mobile Native App • Authorization Code with PKCE • Hybrid With PKCE
i Note: When using appType as the registration method, either the /register or /v2/register endpoints can be used. The appType determines the flow.			

Request Parameters

The following parameters can be posted as an application/json media type to the API.

i Note: The listed parameters are applicable for any OAuth version, unless indicated otherwise in the description.				
Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
app_type	String	Optional	All	The app type of the OAuth client. Specify the app type corresponding to the grant type that the OAuth client should use: • "ServerSideApp" - authorization_code • "HybridWithPKCEFlow" - authorization_code • "AuthorizationCodePKCEFlow" - authorization_code • "MobileNativeApp" - authorization_code • "SinglePageApp" - implicit • "ServerToServerApp" - client_credentials Note: Either the app_type or grant_types parameter can be used to specify the grant type. If

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				both have been defined, app_type will take precedence.
grant_types	Array	Optional	All	The grant type of the OAuth client. Specify the grant type that the OAuth client should use: "authorization_code" "implicit" "client_credentials" "refresh_token" Note: It is possible to specify both "client_credentials" and "authorization_code" for a single OAuth client. The client will be able to process authorization requests of both grant types. Either the grant_types or the app_type parameter can be used to specify the grant type. If both are provided, app_type will take precedence. If both app_type and grant_types are omitted, the default is "authorization_code". The generated app type will be dependent on the endpoint specified: "ServerSideApp" with API "/register" "AuthorizationCodePKCEFlow" with API "/v2/register".
client_name	String	Optional	All	The name of the OAuth client.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
client_secret	String	Optional	All	The OAuth client's client secret. This parameter can be used to specify the value of the client secret if needed. If omitted, the AccessMatrix server will generate a random client secret value.
status	String	Optional	All	The OAuth client status. It can either be "Activated" or "Disabled". If omitted, the default is Activated.
allow_introspect_other_clients	Boolean	Optional	client_credentials	When enabled, this allows the OAuth client to introspect the access token or refresh token for other OAuth clients. If omitted, the default value is false. Note: This is only available for OAuth clients using the Client Credentials grant. It is not available for OAuth clients using the Authorization Code with Client Credentials grant.
token_endpoint_auth_method	String	Optional	authorization_code	The requested client authentication method for the Token endpoint. Values: • "client_secret_post" • "client_secret_basic" • "private_key_jwt" • "none" Note: When using the /v2/register endpoint with the grant type specified as "authorization_code" and the token_endpoint_auth_method specified as "none," an OAuth

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				client is created with the "MobileNativeApp" app type.
pkce_client_auth_method	String	Optional	authorization_code	The requested client authentication method in token introspection, revocation, and refresh processes. Values: "private_key_jwt" "none" Note: This is only available for public clients using the Authorization Code grant type with "/v2/register" API. This parameter is only available for OAuth 2.1 flows.
redirect_uris	Array	Optional	authorization_code, implicit	The callback locations where the user-agent will be redirected to, along with the code or access token.
js_web_origin_uris	Array	Optional	authorization_code, implicit	The web origin URIs that are used to allow AccessMatrix UAM (the authorization server) to validate the origin of the call to ensure it is trusted and whitelisted.
required_authn_level	Integer	Optional	authorization_code, implicit	This setting indicates the minimum authentication level of the authenticated user required by this OAuth client.
scope	String	Optional	All	A space-separated list of scope values used in the application to specify what access privileges are being requested for access tokens. Scope values need to be a subset of the OAuth Authz Server supported scopes.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
is_session_token_embedded	Boolean	Optional	authorization_code, implicit	Embed the session token in the authorization code, refresh token or ID token if applicable. If this attribute is enabled: - The expiry of the ID token, access token, and refresh token will be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting. - For the refresh token, the expiry is Session Idle Time minus 60 seconds.
allow_all_users_to_access	Boolean	Required	authorization_code, implicit	Enabling this feature allows all authenticated users to access this OAuth client. If omitted, the default value is true.
allow_multiple_profiles	Boolean	Optional	authorization_code, implicit	Enabling this feature allows a user to have multiple login profiles (secondary account) for the same application. If omitted, the default value is false.
logo_uri	String	Optional	authorization_code, implicit	The URL for the logo for the OAuth client application.
jwks_uri	String	Optional	authorization_code, implicit	The URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.
jwks	String	Optional	authorization_code, implicit	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. The jwks parameter takes precedence over the jwks_uri parameter if both are specified.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
response_types	Array	Optional	authorization_code, implicit	The type of response the client expects. Values: "code id_token" Note: When using the /v2/register endpoint with the grant type specified as "authorization_code", and response_types specified as "code id_token", an OAuth client is created with the "HybridWithPKCEFlow" app type.
sso_url	Array	Optional	authorization_code	A URL that initiates OIDC authentication or SSO operation. It is a client application URL that generate a one time use authentication URL (together with its payloads) where it should trigger browser redirect to OP OIDC OP authentication endpoint (/auth) for OIDC authentication flow. Note: It is required only by SSO portal or custom IDP portal that lists user's entitled OIDC RP application(s).
authz_code_format	String	Optional	authorization_code	Specifies the format for the authorization Code. If specified as "JWT", the authorization code will be returned as JWT, where additional custom claims can be embedded based on the authz_code_user_response_id parameter. Values:

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				"Opaque" "JWT" Default: Opaque Note: The JWT will be returned in the <code>code</code> parameter, which includes the <code>am_code</code> parameter containing the authorization code.
authz_code_user_response_id	String	Optional	authorization_code	User response to be embedded in the authorization code. The <code>authz_code_format</code> parameter must be specified as "JWT".
Request Object				
request_object_sign_algorithm	String	Optional	authorization_code, implicit	Specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is specified, the OAuth client is allowed to use any of the supported algorithms for Request Object signing. Values: "RS256" "RS384" "RS512" "ES256" "ES384" "ES512"
Access Token				
access_token_expiry	Integer	Optional	All	Sets the access token expiry time in seconds. Value should be from 600 to 7200.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
access_token_format	String	Optional	All	Specifies the format of the access token. Values: "Opaque" "JWT" "JWE" (not applicable for client_credentials grant type) For JWE format, ensure that: - The jwe_algorithm and jwe_enc_alg parameters are configured. - The jws_uri parameter is configured. If omitted, the default is Opaque.
access_token_user_response_id	String	Optional	authorization_code, implicit	User response to be returned in access token. Applicable for the JWT and JWE formats.
JSON Web Encryption (JWE)				
jwe_algorithm	String	Optional	authorization_code, implicit	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Values: "RSA-OAEP-256" "ECDH-ES+A256KW"
jwe_enc_algorithm	String	Optional	authorization_code, implicit	Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Values: "A256CBC-HS512" "A256GCM"

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
Refresh Token				
refresh_token_expiry_in_mins	Long	Optional	authorization_code	Specifies the validity period of the refresh token. Value should be from 10 to 525600. If omitted, the default is 1440.
refresh_token_lifetime_in_hours	Long	Optional	authorization_code	The lifetime expiry for the refresh token, starting from when it is first generated. Refresh token can be renewed up until this expiry. "0" means no lifetime expiry. Value should be from 0 to 1440. If omitted, the default is 168.
refresh_token_integration_date	Boolean	Optional	authorization_code	Refer to the Refresh Token Integration Date attribute description in Configure the OAuth Client for the details of this parameter . If omitted, the default is false.
refresh_token_format	String	Optional	authorization_code	Specifies the format for the refresh token. If specified as "JWT", the refresh token will be returned as JWT, where additional custom claims can be embedded based on the refresh_token_user_response_id parameter. Values: "Opaque" "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_rt parameter containing the refresh token.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
refresh_token_user_response_id	String	Optional	authorization_code	User response to be embedded in the refresh token. The refresh_token_format parameter must be specified as "JWT".
OIDC Tokens and User Session				
oAuth_session_concurrency	Boolean	Optional	authorization_code, implicit	If this parameter is set to "true", then the invalidation of a user session also invalidates the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited. If omitted, the default is false.
revoke_token_invalid_session	Boolean	Optional	authorization_code	If this parameter is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. If omitted, the default is false.
Client Secret				
client_secret_validity_period_in_mins	Integer	Optional	authorization_code, client_credentials	The validity period of the previous client secret after a client secret reset is triggered. During this period, both the previous client secret and the new client secret will be usable. Set the value to "0" to immediately invalidate the previous client secret upon a client secret reset. Value should be from 0 to 525600. Default: 0
OpenID Connect				
default_max_age	Integer	Optional	authorization_code, implicit	The default allowable elapsed time since the last time the end user was actively authenticated by AccessMatrix UAM. This value is in

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				seconds. If omitted, no default is specified.
subject_value	String	Optional	authorization_code, implicit	This setting indicates where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this OAuth client. It is used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. Values: • "Id" • "Name" • "DN" • "Email" • "Secondary Account" • "User Response"
subject_value_response_config	String	Optional	authorization_code, implicit	This parameter is only required if subject_value is configured with "User Response" as its value. This allows other alternatives to retrieve a user identity (using the user response's predefined field/attributes, or by customization of user response).
attr_claim_value_response_config	String	Optional	authorization_code, implicit	This attribute value has to be a user response object to include additional claims of user in ID token without user consent.
id_token_encrypt	Boolean	Optional	authorization_code, implicit	Setting this parameter to "true" will return the ID token as a JWE token. For the JWE format, ensure that:

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				- jwe_algorithm and jwe_enc_algorithm are configured. - Either jwks_uri or jwks is specified. Default: false
User Consent				
consent_rule	String	Optional	authorization_code, implicit	This is to indicate if the OAuth Client Application can be trusted without consent prompt.
remove_scope_upon_token_revoke	String	Optional	authorization_code, implicit	If enabled, this removes the consented scope(s) of the user upon refresh token revocation.
tos_uri	String	Optional	authorization_code, implicit	The URL for the OAuth client terms of service page.
policy_uri	String	Optional	authorization_code, implicit	The URL for the OAuth client data policy page.
Policy				
App_Policy_Id	String	Optional	authorization_code	The application policy that will be applied to the OAuth client. This allows the OAuth client to whitelist specific authentication or reauthentication methods for users. This setting may be required if: Enable User Token Revocation with Security Events attribute has been set to "true" in the OAuth Authz Server. - CLP is required to enforce authentication using only

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				whitelisted authentication mechanisms configured in the policy. Note: For more details on how to configure the policy, refer to the UAM Administration Guide - Application Policy Configuration.

Response Properties

A successful response returns the HTTP "201 Created" status code, along with all registered metadata (refer to [Request Parameters](#)) about the OAuth client, including any fields provisioned by UAM Server.

Field	Data Type	Description
client_id	String	The unique client identifier generated by the UAM Server.
client_secret	String	The confidential secret generated by the UAM Server. Alternatively, if specified in the API request, the confidential secret specified by the API caller.
client_secret_expires_at	Integer	The time at which the client secret will expire. Client secret expiry is not supported by the UAM Server. A value of "0" is always returned to indicate that the client secret will not expire.
registration_access_token	String	The registration access token that can be used at the Client Configuration endpoint to perform subsequent operations after Client registration.
registration_client_uri	String	The location of the Client Configuration endpoint where the registration access token can be used to perform subsequent operations after Client registration.
error	String	OAuth error code, e.g. invalid_client_metadata.
error_description	String	A detailed description of the OAuth client registration error.

Error Codes and Possible Causes

The following table illustrates some of the errors that may arise, as well as the possible causes. For an example of the sample codes or headers that are returned, refer to the links under the Status Code column.

Status Code	Error	Possible Causes
400: Bad Request	invalid_client_metadata	An unsupported scope was specified in the request.
		The value of one of the client metadata fields is invalid.
	access_denied	OAuth clients exceed the license limitations.
401: Unauthorized	invalid_token	No initial access token was provided.
		The initial access token provided is invalid.
500: Internal Server Error	-	Configuration error in the Dynamic Client Registration configuration page of the Admin Console, where the Enable Dynamic Client Registration attribute is set to "false".

Update OAuth Client

HTTP PUT to the /register API to update the metadata of the OAuth client or to reset the client secret.

i **Note:** HTTP PUT to the /v2/register API to update the metadata of the OAuth 2.1 client. Updating OAuth 2.0 to OAuth 2.1 only applies to Authorization Code Flow.

HTTP Method: PUT

Path: \${yourOAuthProxyURL}/register/<client_id>

Header: Authorization Bearer <Registration_Access-Token>

To view the sample requests and responses for updating an OAuth client with a specific flow, you may refer to the following links:

- [Update - OAuth 2.0: Authorization Code](#)
- [Update - OAuth 2.0: Implicit](#)

-
- [Update - OAuth 2.0: Client Credentials](#)
 - [Update - OAuth 2.0: Authorization Code with Client Credentials](#)
 - [Update - OAuth 2.0: Mobile Native App](#)
 - [Update - OAuth 2.1: Authorization Code with PKCE](#)
 - [Update - OAuth 2.1: Hybrid with PKCE](#)

Updating OAuth 2.0 to OAuth 2.1

The `/v2/register` endpoint allows for updating existing clients to newer OAuth 2.1 flows. However, this feature is available only to clients that are using Server Side App. If the user attempts to update any other OAuth 2.0 app type, the update request will be ignored and the existing flow will continue to be used. The following table illustrates the OAuth 2.0 app types that are allowed to be updated to OAuth 2.1 using this endpoint:

App type (OAuth 2.0)	Grant Type	Allowed to Update to OAuth 2.1	Description
ServerSide App	authorization_code	Yes	Can be updated to OAuth 2.1 using API <code>/v2/register</code> .
ServerToServerApp	client_credentials	No	Not allowed to update to OAuth 2.1. The existing flow will continue to be used.
SinglePageApp	implicit	No	Not allowed to update to OAuth 2.1. The existing flow will continue to be used.

Request Parameters

The following parameters can be put as an application/json media type to the API.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
client_id	String	Required	All	Unique string using as the Id for the OAuth client.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
client_name	String	Optional	All	The name of the OAuth client.
client_secret	String	Optional	All	The OAuth client's client secret. This parameter can be used to update the client secret to the specified value, if needed.
status	String	Optional	All	The OAuth client status. It can either be "Activated" or "Disabled". If omitted, the default is Activated.
allow_introspect_other_clients	Boolean	Optional	client_credentials	When enabled, this allows the OAuth client to introspect the access token or refresh token for other OAuth clients. If omitted, the default value is false. Note: This is only available for OAuth clients using the Client Credentials grant. It is not available for OAuth clients using the Authorization Code with Client Credentials grant.
token_endpoint_auth_method	String	Optional	authorization_code	The requested client authentication method for the Token endpoint. Values: "client_secret_post" "client_secret_basic" "private_key_jwt" "none" Note: When using the /v2/register endpoint with grant type specified as "authorization_code" and the token_endpoint_auth_method specified as "none," the OAuth client will be updated to the "MobileNativeApp" app type.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
pkce_client_auth_method	String	Optional	authorization_code	The requested client authentication method in token introspection, revocation, and refresh processes. Values: "private_key_jwt" "none" Note: This is only available for public client using Authorization Code grant with "/v2/register" API. This parameter is only applicable to OAuth 2.1 flows.
redirect_uris	Array	Optional	authorization_code, implicit	The callback locations where the user-agent will be redirected to, along with the code or access token.
js_web_origin_uris	Array	Optional	implicit	The web origin URIs are used to allow AccessMatrix UAM (the authorization server) to validate the origin of the call to ensure it is trusted and whitelisted.
required_authn_level	Integer	Optional	authorization_code, implicit	This setting indicates the minimum authentication level of the authenticated user required by this OAuth client.
scope	String	Optional	All	A space-separated list of scope values used in the application to specify what access privileges are being requested for access tokens. Scope values need to be a subset of the OAuth Authz Server supported scopes.
is_session_token_embedded	Boolean	Optional	authorization_code, implicit	Embed the session token in the authorization code, refresh token or ID token if applicable. If this attribute is enabled:

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				- The expiry of the ID token, access token, and refresh token will be based on the Session Idle Time defined in the realm-based session policy of the authentication realm or in Global Setting. - For the refresh token, the expiry is Session Idle Time minus 60 seconds.
allow_all_users_to_access	Boolean	Required	authorization_code, implicit	Enabling this feature allows all authenticated users to access this OAuth client. If omitted, the default value is true.
allow_multiple_profiles	Boolean	Optional	authorization_code, implicit	Enabling this feature allows a user to have multiple login profiles (secondary account) for the same application. If omitted, the default value is false.
logo_uri	String	Optional	authorization_code, implicit	The URL for the logo for the OAuth client application.
jwks_uri	String	Optional	authorization_code, implicit	The URL of the Relying Party's JWKS (JSON Web Key Set) endpoint.
jwks	String	Optional	authorization_code, implicit	A JSON object that represents a set of JWKS (JSON Web Key Set) published by the Relying Party. The jwks parameter takes precedence over the jwks_uri parameter if both are specified.
sso_url	Array	Optional	authorization_code	A URL that initiates OIDC authentication or SSO operation. It is a client application URL that generate a one time use authentication URL (together

Parameter	Data Type	Required/ Optional	Applicable Grant Type	Description
				with its payloads) where it should trigger browser redirect to OP OIDC OP authentication endpoint (/auth) for OIDC authentication flow. Note: It is required only by SSO portal or custom IDP portal that lists user's entitled OIDC RP application(s)
response_types	Array	Optional	authorization_code, implicit	The type of response the client expects. Values: "code id_token" Note: This parameter is only available for OAuth 2.1 flows. When using the /v2/register endpoint with the grant type specified as "authorization_code" and response_types contain "code id_token", the OAuth client will be updated to the "HybridWithPKCEFlow" app type.
app_type	String	Optional	authorization_code, implicit	The app type of the OAuth client. Specify the app type corresponding to the grant type that the OAuth client should use: "HybridWithPKCEFlow" "AuthorizationCodePKCEFlow" "MobileNativeApp" Note: Either the app_type or the grant_types parameter can be used to specify the grant type. If both are provided, app_type will take precedence. This parameter is only available for OAuth 2.1 flows.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
grant_types	Array	Optional	authorization_code	The grant type of the OAuth client. Specify the grant type that the OAuth client should use: "authorization_code" "refresh_token" Note: Either the grant_types or app_type parameter can be used to specify the grant type. If both have been specified, app_type will take precedence.
authz_code_format	String	Optional	authorization_code	Specifies the format for the authorization Code. If specified as "JWT", the authorization code will be returned as JWT, where additional custom claims can be embedded based on the authz_code_user_response_id parameter. Values: "Opaque" "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_code parameter containing the authorization code.
authz_code_user_response_id	String	Optional	authorization_code	User response to be embedded in the authorization code. The authz_code_format parameter must be specified as "JWT".
Request Object				

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
request_object_sign_algorithm	String	Optional	authorization_code, implicit	Specifies the JWS algorithm to be used for signing Request Objects sent to the OpenID Provider. If no algorithm is specified, the OAuth client is allowed to use any of the supported algorithms for Request Object signing. Values: • "RS256" • "RS384" • "RS512" • "ES256" • "ES384" • "ES512"
Access Token				
access_token_expiry	Integer	Optional	All	Sets the access token expiry time in seconds. Value should be from 600 to 7200.
access_token_format	String	Optional	All	Specifies the format of the access token. Values: • "Opaque" • "JWT" • "JWE" (not applicable for client_credentials grant type) For JWE format, ensure that: • The jwe_algorithm and jwe_enc_alg parameters are configured. • The jws_uri parameter is configured.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				If omitted, the default is Opaque.
access_token_user_response_id	String	Optional	authorization_code, implicit	User response to be returned in access token. Applicable for the JWT and JWE formats.
JSON Web Encryption (JWE)				
jwe_algorithm	String	Optional	authorization_code, implicit	Cryptographic algorithm used to encrypt or determine the value of the CEK (Content Encryption Key). Required for JWE (JSON Web Encryption). Values: "RSA-OAEP-256" "ECDH-ES+A256KW"
jwe_enc_algorithm	String	Optional	authorization_code, implicit	JWE Encryption Algorithm Encryption algorithm used to perform authenticated encryption. Required for JWE (JSON Web Encryption). Values: "A256CBC-HS512" "A256GCM"
Refresh Token				
refresh_token_expiry_in_mins	Long	Optional	authorization_code, implicit	Specifies the validity period of the refresh token. Value should be from 10 to 525600. If omitted, the default is 1440.
refresh_token_lifetime_in_hours	Integer	Optional	authorization_code, implicit	The lifetime expiry for the refresh token, starting from when it is first generated. Refresh token can be renewed up until this expiry. "0" means no lifetime expiry. Value should be from 0 to 1440. If omitted, the default is 168.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
refresh_token_integration_date	Boolean	Optional	authorization_code	Refer to the Refresh Token Integration Date attribute description in Configure the OAuth Client for the details of this parameter. If omitted, the default is false.
refresh_token_format	String	Optional	authorization_code	Specifies the format for the refresh token. If specified as "JWT", the refresh token will be returned as JWT, where additional custom claims can be embedded based on the refresh_token_user_response_id parameter. Values: "Opaque" "JWT" Default: Opaque Note: The JWT will be returned in the code parameter, which includes the am_rt parameter containing the refresh token.
refresh_token_user_response_id	String	Optional	authorization_code	User response to be embedded in the refresh token. The refresh_token_format parameter must be specified as "JWT".
OIDC Tokens and User Session				
oAuth_session_concurrency	Boolean	Optional	authorization_code, implicit	If this parameter is set to "true", then the invalidation of a user session also invalidates the OIDC tokens linked to the session. This only works when the Session Concurrency control is limited. If omitted, the default is false.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
revoke_token_invalid_session	Boolean	Optional	authorization_code	If this parameter is set to "true", then the revocation of an access or refresh token invalidates the user session used to generate the token. If omitted, the default is false.
Client Secret				
client_secret_validity_period_in_mins	Integer	Optional	authorization_code, client_credentials	The validity period of the previous client secret after a client secret reset is triggered. During this period, both the previous client secret and the new client secret will be usable. Set the value to "0" to immediately invalidate the previous client secret upon a client secret reset. Value should be from 0 to 525600. Default: 0
OpenID Connect				
default_max_age	Integer	Optional	authorization_code, implicit	The default allowable elapsed time since the last time the end user was actively authenticated by AccessMatrix UAM. This value is in seconds. If omitted, no default is specified.
subject_value	String	Optional	authorization_code, implicit	This setting indicates where to retrieve the user identity (based on the authenticated AccessMatrix user identity of the session token) for this OAuth client. It is used to resolve an end-user identity for the OAuth client when generating the OIDC tokens. Values: "Id" "Name"

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
				"DN" "Email" "Secondary Account" "User Response"
subject_value_response_config	String	Optional	authorization_code, implicit	This parameter is only required if subject_value is configured with "User Response" as its value. This allows other alternatives to retrieve a user identity (using the user response's predefined field/attributes, or by customization of user response).
attr_claim_value_response_config	String	Optional	authorization_code, implicit	This attribute value has to be a user response object to include additional claims of user in ID token without user consent.
id_token_encrypt	Boolean	Optional	authorization_code, implicit	Setting this parameter to "true" will return the ID token as a JWE token. For the JWE format, ensure that: - jwe_algorithm and jwe_enc_algorithm are configured. - Either jwks_uri or jwks is specified. Default: false
User Consent				
consent_rule	String	Optional	authorization_code, implicit	This is to indicate if the OAuth Client Application can be trusted without consent prompt.
remove_scope_upon_token_revoke	String	Optional	authorization_code, implicit	Actions on consented scope(s) of user upon Refresh Token revocation.
tos_uri	String	Optional	authorization_code, implicit	The URL for the OAuth client terms of service page.

Parameter	Data Type	Required/Optional	Applicable Grant Type	Description
policy_uri	String	Optional	authorization_code, implicit	The URL for the OAuth client data policy page.
Policy				
App_Policy_Id	String	Optional	authorization_code	The application policy that will be applied to the OAuth client. This allows the OAuth client to whitelist specific authentication or reauthentication methods for users. This setting may be required if: • Enable User Token Revocation with Security Events attribute has been set to "true" in the OAuth Authz Server. • CLP is required to enforce authentication using only whitelisted authentication mechanisms configured in the policy. Note: For more details on how to configure the policy, refer to the UAM Administration Guide - Application Policy Configuration.

Response Properties

A successful response returns the HTTP "200 OK" status code, client id and client secret, along with all metadata (refer to the Request Parameters) about the OAuth client.

Field	Data Type	Description
client_id	String	The unique Client identifier generated by UAM Server.

Field	Data Type	Description
clientSecret	String	The confidential secret generated by UAM Server. If reset secret is chosen, a newly generated secret is returned. Alternatively, if specified in the API request, the confidential secret specified by the API caller.
client_secret_expires_at	Integer	The time at which the client secret will expire. Client secret expiry is not supported by the UAM Server. A value of "0" is always returned to indicate that the client secret will not expire.
error	String	OAuth error code, e.g. invalid_client_metadata.
error_description	String	A detailed description of the OAuth client registration error.

Error Codes and Possible Causes

The following table illustrates some of the errors that may arise, as well as the possible causes. For an example of the sample codes or headers that are returned, refer to the links under the Status Code column.

Status Code	Error	Possible Causes
400: Bad Request	invalid_client_metadata	An unsupported scope was specified in the request.
		The value of one of the client metadata fields is invalid.
		The client Id value is empty in the request.
		The client Id specified in the request is different from the client Id in the /register API URL.
401: Unauthorized	unauthorized_client	No client Id was provided in the /register API URL.
		The client Id specified in the /register API URL is invalid.
		The client whose Id is provided in the /register API URL is not authorized to update the client metadata.
	invalid_token	No initial access token was provided.
		The initial access token specified is invalid.
		Configuration error in the Dynamic Client Registration configuration page of the Admin Console, where the

Status Code	Error	Possible Causes
		Enable Initial Access Token for Dynamic Client Registration Management attribute is set to "false".
500: Internal Server Error	-	Configuration error in the Dynamic Client Registration configuration page of the Admin Console, where the Enable Dynamic Client Registration attribute is set to "false".

Reset Client Secret

HTTP PUT to the `/register/<client_id>/resetsecret` API to reset the specified OAuth Client's client secret.

HTTP Method: PUT

Path: `${yourOAuthProxyURL}/register/<client_id>/resetsecret`

Header: Authorization Bearer `<Registration_Access_Token>`



Note: Only the client_secret is reset. The OAuth client metadata is still returned in the response.

To view the sample requests and response for creating an OAuth client with a specific flow, you may refer to the following links:

- [Reset - OAuth 2.0: Authorization Code](#)
- [Reset - OAuth 2.0: Implicit](#)
- [Reset - OAuth 2.0: Client Credentials](#)
- [Reset - OAuth 2.0: Authorization Code with Client Credentials](#)
- [Reset - OAuth 2.1: Authorization Code with PKCE](#)
- [Reset - OAuth 2.1: Hybrid with PKCE](#)

Request Parameters

No parameters are needed for this **HTTP PUT** API call.

Response Properties

A successful response returns the HTTP "200 OK" status code, client id and client secret, along with all metadata (refer to the Request Parameters of the [previous section](#)) about the OAuth client.

Field	Data Type	Description
client_id	String	The unique Client identifier generated by UAM Server.
clientSecret	String	The confidential secret generated by UAM Server. If reset secret is chosen, a newly generated secret is returned.
client_secret_expires_at	Integer	The time at which the client secret will expire. Client secret expiry is not supported by the UAM Server. A value of "0" is always returned to indicate that the client secret will not expire.
error	String	OAuth error code, e.g. invalid_client_metadata.
error_description	String	A detailed description of the OAuth client registration error.

Error Codes and Possible Errors

The following table illustrates some of the errors that may arise, as well as the possible causes. For an example of the sample codes or headers that are returned, refer to the links under the Status Code column.

Status Code	Error	Possible Causes
400: Bad request	invalid_request	Client specified does not support client secret.
401: Unauthorized	unauthorized_client	No client Id was provided in the /register API URL.
		The client Id specified in the /register API URL is invalid.
		The client whose Id is provided in the /register API URL is not authorized to update the client metadata.
	invalid_token	No initial access token was provided.
		The initial access token specified is invalid.
		Configuration error in the Dynamic Client Registration configuration page of the Admin Console, where the Enable Initial Access Token for Dynamic Client Registration Management attribute is set to "false".
500: Internal Server Error	-	Configuration error in the Dynamic Client Registration configuration page of the Admin Console, where the Enable Dynamic Client Registration attribute is set to "false".

Read OAuth Client Metadata

HTTP GET to the /register/<client_id> API to get the registered metadata of the OAuth client.

HTTP Method: GET

Path: \${yourOAuthProxyURL}/register/<client_id>

Header: Authorization Bearer <Registration_Access-Token>

To view the sample requests and responses for reading an OAuth client's metadata, you may refer to the following links:

- [Read - OAuth 2.0: Authorization Code](#)
- [Read - OAuth 2.0: Implicit](#)
- [Read - OAuth 2.0: Client Credentials](#)
- [Read - OAuth 2.0: Authorization Code with Client Credentials](#)
- [Read - OAuth 2.0: Hybrid with PKCE](#)
- [Read - OAuth 2.1: Authorization Code with PKCE](#)
- [Read - OAuth 2.1: Hybrid with PKCE](#)

Request Parameters

No parameters are needed for this **HTTP GET** API call.

Response Properties

A successful response returns the HTTP "200 OK" status code, client id and client secret, along with all metadata about the OAuth client.

Field	Data Type	Description
client_id	String	The unique Client identifier generated by the UAM Server.
clientSecret	String	The confidential secret generated by the UAM Server. Alternatively, if specified in the API request to create/update the OAuth client, the confidential secret specified by the API caller.

Field	Data Type	Description
client_secret_expires_at	Integer	The time at which the client secret will expire. Client secret expiry is not supported by the UAM Server. A value of "0" is always returned to indicate that the client secret will not expire.
error	String	OAuth error code, e.g. invalid_client_metadata.
error_description	String	A detailed description of the OAuth client registration error.

Error Codes and Possible Causes

The following table illustrates some of the errors that may arise, as well as the possible causes. For an example of the sample codes or headers that are returned, refer to the links under the Status Code column.

Status Code	Error	Possible Causes
401: Unauthorized	unauthorized_client	No client Id was provided in the /register API URL.
		The client Id specified in the /register API URL is invalid.
		The client whose Id is provided in the /register API URL is not authorized to read the client metadata.
	invalid_token	No initial access token was provided.
		The initial access token specified is invalid.
500: Internal Server Error	-	Configuration error in the Dynamic Client Registration configuration page of the Admin Console, where the Enable Dynamic Client Registration attribute is set to "false".

Delete OAuth Client

HTTP DELETE to the /register/<client_id> API to delete an OAuth client.

HTTP Method: DELETE

Path: \${yourOAuthProxyURL}/register/<client_id>

Header: Authorization Bearer <Registration_Access_Token>

Request Parameters

No parameters are needed for this **HTTP DELETE** API call.

Response Properties

A successful response returns the HTTP "204 No Content" success status code.

Error Codes and Possible Causes

The following table illustrates some of the errors that may arise, as well as the possible causes. For an example of the sample codes or headers that are returned, refer to the links under the Status Code column.

Status Code	Error	Possible Causes
401: Unauthorized	unauthorized_client	No client Id was provided in the /register API URL.
		The client Id specified in the /register API URL is invalid.
		The client whose Id is provided in the /register API URL is not authorized to delete the client metadata.
	invalid_token	No initial access token was provided.
		The initial access token specified is invalid.
500: Internal Server Error	-	Configuration error in the Dynamic Client Registration configuration page of the Admin Console, where the Enable Dynamic Client Registration attribute is set to "false".

6.4. Access Control

There are two ways of enforcing access control through the OAuth client:

- [Access Privilege](#)
- [Enforce Minimum Authentication Level](#)

Access Privilege

You can add access control to the OAuth Client application via **Group OAuth Client privilege** or **User's OAuth Client Privilege** .

To enforce such restrictions, you must first configure the OAuth client to disable all user access to the application using the admin console. Perform the following steps:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Select "false" using the drop-down list for the **Allow All Users to Access** attribute.
5. Click the **Save** button to save the changes.

Configure Group Access Privileges

Access privileges can be restricted to a specific group. To configure this, perform the following steps:

1. Navigate to **Operation Dashboard > User & Segment > Group**.
2. Search for the group.
3. Under the **View Group** panel, select **OAuth Client Privilege** tab and click the **Edit** button.
4. Click **Add** button, **Select OAuth Client** dialog box will appear.
5. Choose the OAuth Client and click **OK**.
6. Click the **Save** button to save the changes.

Configure User Access Privileges

Access privileges can be restricted to a specific user. To configure this, perform the following steps:

-
1. Navigate to **Operation Dashboard > User & Segment > User**.
 2. Search for the user.
 3. Under the **View User** panel, select **OAuth Client Privilege** tab and click the **Edit** button.
 4. Click **Add** button, **Select OAuth Client** dialog box will appear.
 5. Choose the OAuth Client and click **OK**.
 6. Click the **Save** button to save the changes.

Enforce Minimum Authentication Level

Alternatively, if the application should only be accessed by highly privileged accounts, access can be further restricted by mandating a minimum authentication level for the application. This is configured using the login module. To enable this setting, perform the following steps:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Set the **Required Authentication Level** attribute to a value greater than 1.



Note: Ensure the privileged end user has been assigned a login module with an authentication level equal to, or higher than this value.

5. Click the **Save** button to save the changes.
-

6.5. Deprecated Flows

AccessMatrix Universal Access Management (UAM) supports various OAuth/OIDC flows. The flows listed below have been deprecated as of the latest OAuth 2.1 specification. They must not be used for new applications, and should only be used for legacy applications.

- [Authorization Code Flow](#)
- [Implicit Flow](#)
- [Resource Owner Password Credentials Flow](#)
- [Hybrid Flow](#)

7. OAuth Client Authentication

Client authentication is the process by which an OAuth client application proves its identity to the OAuth authorization server. This ensures that only legitimate clients can access protected resources. By verifying the client's identity, OAuth prevents unauthorized or malicious applications from accessing sensitive data.

The following pages provide details on client authentication-related configurations:

- [Private Key JWT](#)
- [Client Secret](#)
- [PKCE \(only for token exchange\)](#)

7.1. Private Key JWT

The Private Key JWT is a method used to authenticate the OAuth client. Through this method, the client creates and signs a JWT using their own private key. Subsequently, the authorization server can authenticate the client by verifying the signature and payload of the assertion.

This method should be used whenever possible due to the following benefits for client authentication:

- The client is not required to share a secret with the authorization server. Instead, the client proves its identity by demonstrating possession of a private key through a digital signature.
- Private key-based authentication is more secure, and reduces the risk of client impersonation.



Note: A sample request payload is illustrated in [Authorization Code flow with PKCE](#).

7.2. Client Secret

OAuth client authentication with a client secret is a method used to authenticate the client application to the authorization server. By using the client secret, it prevents client impersonation whereby malicious actors pretend to be a legitimate client to get an access token from the resource owner under false pretenses.

This client authentication method is typically used in confidential clients, such as web server applications, where the `client_secret` can be securely stored.



Note: It is important to note that the `client_secret` should never be included in public (mobile or browser-based) clients.

The client secret can be sent using 2 authentication methods: Basic and POST.

Basic

Using the basic authentication scheme, the client secret is sent as an authorization header. An example is shown below.

Code

```
POST /oauth-proxy/token HTTP/1.1
Host: authzserver
Content-Length: 307
Content-Type: application/x-www-form-urlencoded
Authorization: Basic dGVzdC5jb206ZiUzRDJnYW9qd2Nv
grant_type=authorization_code&code_verifier=ragda6agja&redirect_uri=http%
3A%2F%2Flocalhost&code=ddg5g7mdxx...truncated...CSq
```

POST

Using the POST method, the client secret is enclosed in the HTTP request body as a parameter. An example is shown below.

Code

```
POST /oauth-proxy/token HTTP/1.1
Host: authzserver
Content-Length: 307
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=authorization_code&client_secret=zj24KhNs1lqp&code_verifier=ra  
gda6agja&redirect_uri=http%3A%2F%2Flocalhost&code=ddg5g7mdxx...truncated.  
..CSq
```

7.3. PKCE (only for token exchange)

In OAuth 2.0, authorization flows are done with entirely public information. As such, public clients are not authenticated. To address this security concern, the Proof Key for Code Exchange (PKCE) was introduced as an additional security feature for the authorization code flow, primarily targeting mobile applications. PKCE provides an alternative to the implicit flow, ensuring secure authentication even when the client cannot keep a secret.

It is important to note that PKCE is not a form of client authentication, but a way to ensure that the application starting the authentication flow is the same one that finishes it. It does so by using `code_challenge` and `code_verifier` to associate the app that starts the flow is the one that completes it. In doing so, the client's identity is verified. This process ensures that even if an attacker intercepts the authorization code, they cannot exchange it for an access token without the `code_verifier`.

i **Note:** The `code_verifier` is generated by the client for each OAuth 2.0 Authorization Request. It is a high-entropy cryptographic random string of unreserved characters with a minimum length of 43 characters and a maximum of 128 characters. The `code_challenge` is then created when the client hashes the `code_verifier` using the `code_challenge_method` (SHA-256).

This is particularly useful for public clients, such as mobile applications or single-page applications, which cannot securely store a client secret. PKCE effectively closes the gap left by the absence of a client secret, enhancing the security of the OAuth 2.0 flow.

8. Refresh Token

Standard Use Case

In general, refresh token is used to get new OIDC tokens. The refresh token expiry can be changed by configuring the OAuth client.

By default, **Embed Session Token** is set to "false" in the OAuth client. The default expiry periods will then be set for the following attributes:

- Session token idle time - 600 seconds
- Access token - 3600 seconds
- ID token - 60 minutes
- Refresh token - 1440 minutes
- Refresh token lifetime - 168 hours

Configuration rules:

- The session token is set when a user is first authenticated using /auth endpoint. The session token lifetime idle time expiry setting should be configured accordingly to your business requirements.:
- Generally, the **Refresh Token Expiry** should be longer than ID token validity period and access token expiry time duration.

Advanced Use Case

Under special circumstances, there are configurations that allow the refresh token to embed session tokens. This feature is used to allow an OAuth client application to extract session token from the OIDC token for AM REST API calls.



Important Note: This use case is not recommended as a session token represents a privileged session. If it is obtained by illegitimate parties, it could pose serious security implications. Such requirements are uncommon and must not be enabled if it is not necessary.

If **Embed Session Token** is set to "true" in OAuth client, then the following default expiry will be set:

- Session token idle time is defined in the realm-based session policy of the authentication realm or in global setting.
-

-
- The expiry of ID token, access token, and refresh token would be based on the **Session Idle Time** defined in the realm-based session policy of the authentication realm or in global setting.
 - For refresh token, the expiry is **Session Idle Time** minus 60 seconds.

Configuration rules:

- There is no need to configure for ID token and access token expiry/validity when session token is embedded.
- Server-side application must refresh tokens using refresh token latest by 60 seconds before session token idle time out.

Other Use Cases

Revoke OIDC Token when OP Session is Invalidated

Alternatively, Single Logout (SLO) can be triggered from the OpenID Provider (OP). In the event of OP session logout, OIDC tokens belonging to all other SSO OAuth clients will be invalidated.

This behaviour is configurable in OAuth Client configuration under the **Revoke Token Upon User Session Invalidation** attribute.

For example:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click **Edit**.
4. Specify a value in the **Revoke Token Upon User Session Invalidation** attribute.
5. Click **Save**.

Invalidate OP Session when OIDC Token is Invalidated

SLO is achieved by presenting a refresh token to the OIDC revocation endpoint. This causes all AM5 sessions currently tied to the refresh token to be logged out.

In general, this behavior is used in conjunction with session concurrency control (with 1 session per user max setting) where the user's sessions persist at the AM5 server side.

The OAuth Client can log out all of its sessions at the OP by calling the token revocation endpoint. This mechanism performs the following actions concurrently:

- Invalidates the user's OIDC token(s)
- Logs out the user's current session at the OpenID Provider (OP)

This behavior is configurable in OAuth Client configuration under the **Invalidate User Session Upon Token Revocation** attribute.

For example:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Specify a value in the **Invalidate User Session Upon Token Revocation** attribute.
5. Click the **Save** button to save the changes.

9. Common Login Page

This page briefly details how to use the AccessMatrix Common Login Page (CLP) as the OIDC OP authentication/consent front end. For more details regarding OIDC integration with CLP in general, refer to [AccessMatrix Common Login Page Deployment Guide - CLP as OIDC OpenID Provider](#).

Configure Consent Page

CLP can be configured to Customize Display Content of Scope(s) for OIDC User Consent Page for user friendly names.

Refer to [Customize Display Content of Scope\(s\) for OIDC User Consent Page](#).

Configure Login Page of /auth Endpoint

Refer to [AccessMatrix Common Deployment Guide - AccessMatrix CLP Authentication Methods](#) for more details on how to configure realm selection in CLP.

Configure CORS for OIDC Endpoints

Refer to [Cross-Origin Resource Sharing \(CORS\) for OIDC Endpoints](#) for more details on the steps required.

Configure CLP as Identity Broker

CLP can be configured to work as an identity broker to provide external OIDC OP authentication.

Refer to [AccessMatrix Common Login Page Deployment Guide - External OP Authentication via OIDC](#) for more details on how to configure realm selection in CLP.

10. Custom Login / Consent Page and OAuth Proxy Integration



Note: If you are using the default deployment with the AccessMatrix Common Login Page (CLP) instead of a custom authentication front end/back end, skip this page and its related sections. You can refer to [Common Login Page](#) for CLP-related configurations.

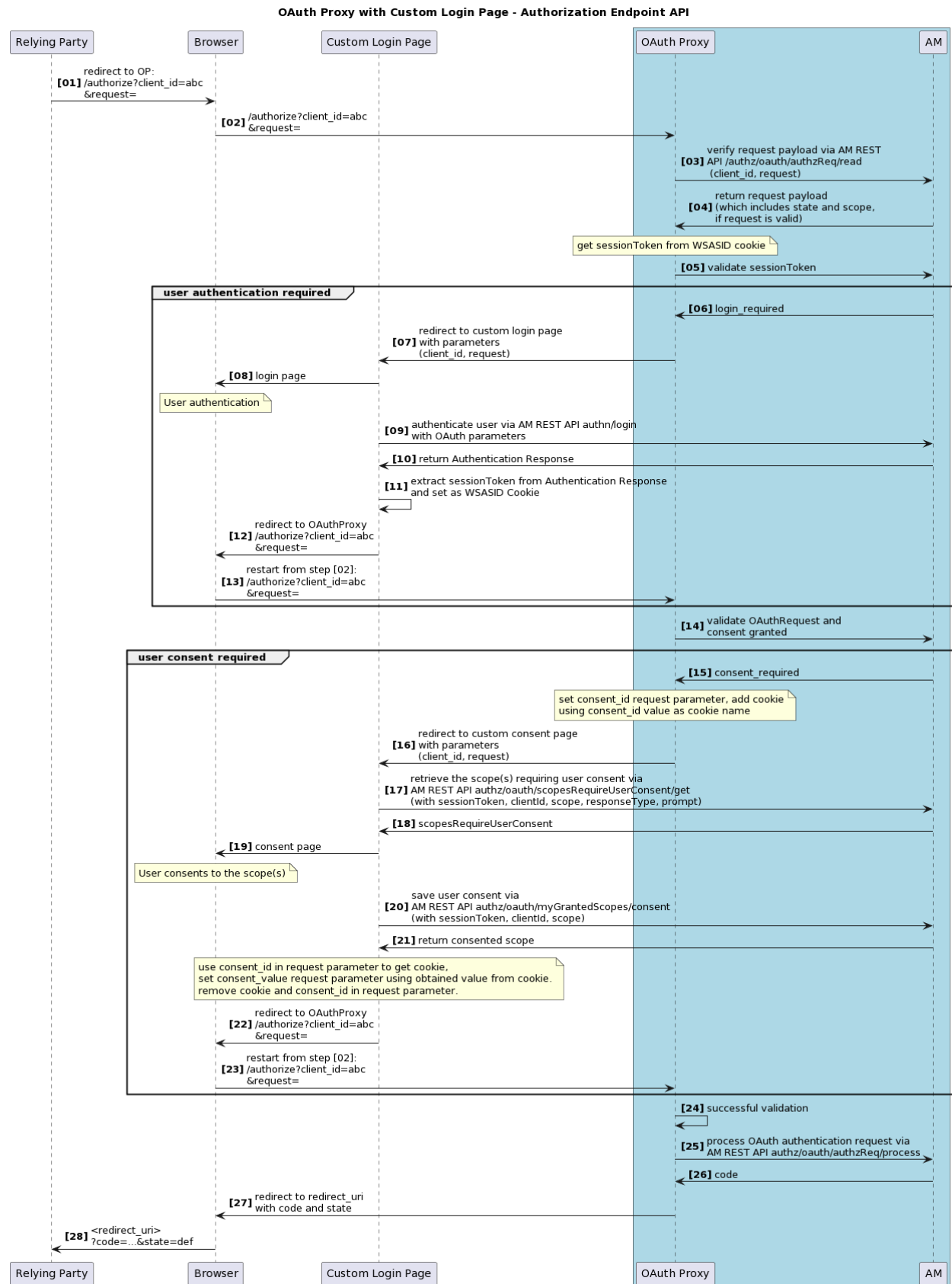
Instead of using the bundled AccessMatrix Common Login Page (CLP), an external custom authentication front end and back end can be used for OAuth flows. This document refers to this component as the custom login page. The custom login page is expected to comprise the following:

- **Front end**
 - Provides the login page for user logins
 - Provides the consent page for user consent (if user consent is required)
- **Back end**
 - Processes user authentication by calling the AccessMatrix Authentication API
 - Handles consent management by calling the AccessMatrix User Consent API (if user consent is required)

OAuth Proxy (or AccessMatrix CLP) and the AccessMatrix server (also referred to as the UAM server) collectively act as the OIDC OpenID Provider. Refer to the sequence diagram in the next section for more details on how they work together in a typical OAuth flow.

Sequence Diagram

The following diagram illustrates how the OAuth Proxy, AccessMatrix server, and the custom login page work together, using the Authorization Code flow as an example.



In this diagram, it is assumed that the end user has not logged in to a Relying Party (RP) application and triggered a request to allow it to retrieve protected resources on the user's behalf.

-
- Step 1-2: The RP sends the request via the proxy GET or POST Method to the OAuth Proxy's /auth endpoint.
 - Step 3-4: The OAuth Proxy sends a request to AccessMatrix to do the preliminary check and validation on the OAuth request.
 - Step 5-6: Upon successful validation, since the user is not yet authenticated, the OAuth Proxy redirects them to the Custom Login Page to log in.
 - Step 7-8: The Custom Login Page is required to call the AccessMatrix REST API /authz/oauth/authzReq/read to validate the authenticity of the incoming request. The request and validity of the request are checked. If the request is deemed valid, the response payload, which includes the state and the scope (for the consent page to prompt the user for consent), is returned.
 - Step 9-11: The user is authenticated and, if successful, the Custom Login Page receives an authentication response with the session token. The Custom Login Page extracts the session token from the authentication response and sets it as the WSASID cookie.
 - Step 12-13: This is followed by having the same oauth request redirected back to the OAuth Proxy's /authorize endpoint for use consent authorization validation.
 - Step 15-16: If user consent is required, OAuth Proxy will redirect the browser back to the Custom Login Page.
 - Step 17-18: Custom Login Page is required to call AccessMatrix REST API authz/oauth/scopesRequireUserConsent/get to check and retrieve if there is any scope would require user consent.
 - Step 19-21: User would be prompted with consent page. Once the user has granted the consent, the Custom Login Page calls the AccessMatrix REST API to save the user consent via authz/oauth/myGrantedScopes/consent. The Custom Login Page receives the consented scopes as the response.
 - Step 22-25: The Custom Login Page redirects the user to the OAuth Proxy's /auth endpoint again. This time, as the user has been authenticated, the OAuth authorization is validated and processed.
 - Step 26-28: For a successful use case, the authorization code is returned from AccessMatrix, and then redirected back to the RP.
-

Implementation Planning

i **Note:** Before designing the custom login page, it is important to consider the security and performance of your OAuth and OIDC implementation. For more details, refer to [Implementation Considerations](#).

Generally, the custom login page can function as:

1. Login page only
2. Login page and consent page only
3. OAuth /auth endpoint, login page and consent page
4. OAuth /auth endpoint only

The following table illustrates when AccessMatrix CLP or the OAuth Proxy should be deployed depending on your customization requirements:

Customization	AccessMatrix CLP	OAuth Proxy
Login page only	If CLP standard consent page is required as well	If only the login page is customized
Login page and consent page only	Not required	Required
OAuth /auth endpoint, login page	Not required	Required, since other OAuth/ OIDC endpoints are still required.
OAuth /auth endpoint only	Required if consent page and/or login page of AccessMatrix CLP and other OAuth/ OIDC endpoints are required.	Required if and only if OAuth/OIDC endpoints are required

Integration Considerations

Managing User Login

Click [here](#) to find out more about implementation of login page.

Saving of User's Consent

Click [here](#) to find out more about implementation of consent page.

Implementation Considerations

Browser Redirection

If the custom implementation flow requires browser redirection of an OAuth request to the OAuth Proxy /auth endpoint, it is highly recommended to ensure the browser redirection is done without going through the internet. This can be achieved using local network configurations via internal DNS settings, server-side handling, internal proxy via reverse proxy or application-level redirection.

Browser redirection via the internet can have security implications, its safety depends on several factors. Some considerations are illustrated below:

- **Man-in-the-Middle (MitM) Attacks:** If the redirection goes through the internet and the communication is not encrypted (i.e., using HTTP instead of HTTPS), it is vulnerable to MitM attacks. An attacker could intercept and manipulate the traffic.
- **Performance Issues:** Redirecting through the internet might introduce latency and performance issues, especially if the internet connection is slow or unreliable.



Note: Read [here](#) to find out more about the other security and performance considerations before your implementation.

11. UAM OAuth Proxy

The following pages provide the OAuth Proxy deployment and configuration steps:

- [UAM OAuth Proxy Deployment](#)
- [UAM OAuth Proxy Configuration](#)

11.1. UAM OAuth Proxy Deployment

The UAM OAuth Proxy can be deployed by installing the Web Archive (WAR) package. To deploy:

1. Secure the OAuth Proxy WAR file from the AccessMatrix CD-Image. The filename format is **AccessMatrix-x.x.xxxx-GA-oauth-proxy.war**, e.g. **AccessMatrix-5.7.5.7535-GA-oauth-proxy.war**.
2. (Optional) For easier reference, rename the WAR file to **oauth-proxy.war**.
3. The OAuth Proxy is a stateless web service, which means you can deploy the WAR file (**oauth-proxy.war**) to any web application container supported by AccessMatrix core products version 5.7.2 and up (i.e. Apache Tomcat and JBoss Web Server).
4. If using Tomcat, create an *oauth-proxy* folder under */opt/acmatrix/tomcat/webapps/*.
5. Extract the contents of **oauth-proxy.war** to this folder.

i **Note:** In this guide, *<PROXY_HOME>* is used to refer to the installation directory of the OAuth Proxy components, e.g. */opt/acmatrix/tomcat/webapps/oauth-proxy/*.

UAM OAuth Proxy Components

Below is the list of the folders and sub-folders inside the OAuth Proxy WAR file:

Folder/File Name	Sub-folder/File Name	Description
WEB-INF	classes\log4j2.properties	This file contains the configurations of log level and log output file location. Refer to the Configure OAuth Proxy Log Properties section for configuration details.
	lib	The libraries used for the OAuth Proxy.
	web.xml	This file contains the configurations of the AccessMatrix REST API URL, cookie settings, connection pool configuration, and filter mapping URL pattern.
oautherror.jsp	-	The page to display the OAuth Proxy error message.

11.2. UAM OAuth Proxy Configuration

The following sections contain the configurations required in the OAuth Proxy to integrate with AccessMatrix UAM system.

Configure OAuth Proxy to Connect to the AccessMatrix Server

The configurations below establish the connection between the OAuth Proxy and AccessMatrix server.

To configure:

1. Navigate to `<PROXY_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the `restAPIURL` element and modify the value to refer to the relevant AccessMatrix server as shown below:

XML

```
<context-param>
  <param-name>restAPIURL</param-name>
  <param-value>http://localhost:8080/am5/rest</param-
value>
</context-param>
```

To support client-authenticated SSL (a mandatory configuration for production environments), the following settings should be configured:

- a. Modify the `restAPIURL` value to use a secure connection to the AccessMatrix server as shown below:

XML

```
<context-param>
  <param-name>restAPIURL</param-name>
  <param-value>http://localhost:8443/am5/rest</param-
value>
</context-param>
```

- b. Search for the Connection Pool Configuration section.
- c. Uncomment the section and add the following client keystore properties:

XML

```
<context-param>
  <param-name>am.hc4.ssl.keystore</param-name>
  <param-value>/opt/acmatrix/tomcat/conf/clp-
```

```
client.keystore</param-value>
</context-param>
<context-param>
  <param-name>am.hc4.ssl.keystore-passphrase</param-name>
  <param-
value>AesEncryptor:pWtZZ0N6bfIvSlBBF0zXC0ZwS1Kny/GexGVFpaqWI1Q
=</param-value>
</context-param>
```



Note: Refer to [AccessMatrix Common Login Page Deployment Guide - List of Configurable Parameters](#) for the description of the properties.

3. Save the changes.
4. Restart the OAuth Proxy for changes to take effect.

Configure Login and Consent Page Redirect URL

1. Navigate to `<PROXY_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the `loginRedirectURL` element, as shown below:

XML

```
<init-param>
  <param-name>loginRedirectURL</param-name>
  <param-value>/oauth2login</param-value>
</init-param>
```

3. Modify the existing value (i.e. `/oauth2login`), to the actual login redirect URI value of the custom login page. For example, `"https://<webapp_domain>/login"`.



Note: It is assumed that the user login and consent are both handled via a single redirect URI. If this is not the case, a `consentRedirectURL` parameter is also provided in the same section of the configuration file. Update the value of this parameter accordingly.

4. Save the changes.
5. Restart the OAuth Proxy for changes to take effect.



Note: If your project makes use of the `prompt` request parameter with `"login"` as the parameter value, your custom login page needs to set the `oauthPromptLoginDone` request parameter (with any value) after the user has been authenticated or reauthenticated by your custom login page. This `oauthPromptLoginDone` request parameter is returned via `/auth` to the UAM's OAuth Proxy via browser redirection.

Configure Session Token Cookie Domain and Path

Upon user login, an AccessMatrix session token cookie (also known as WSASID cookie) will be generated. The session cookie is used to verify the user's identity and session validity. It can be set to a domain and context path.



Note: To learn more about OP session management and best practices, refer to [OP Session Management](#).

To configure:

1. Navigate to `<PROXY_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the cookie-related elements as shown below:

XML

```
<context-param>
  <param-name>cookiePath</param-name>
  <param-value>/</param-value>
</context-param>
<context-param>
  <!--Allow cookie to set in this domain. If you
  using Internet Explorer, this value is required to be
  set. Default is empty-->
  <param-name>cookieDomain</param-name>
  <param-value></param-value>
</context-param>
<context-param>
  <param-name>cookieSameSite</param-name>
  <param-value></param-value> <!-- Default is not set
  for SameSite -->
</context-param>
```

3. Specify the domain value to allow the session cookie be set in the specified domain. By default, domain value is empty, thus setting the session cookie in the default domain.



Note: You are required to specify the domain value if you are using Internet Explorer.

4. Specify the context path value. By default, path is set to `/`, the default value is not safe as this will make available the session cookie to all the context paths in the current domain, . Please restrict it based on the context path of your OAuth Proxy deployment.
 5. Specify the `cookieSameSite` value to `"Strict"`. By default, `cookieSameSite` is empty, thus it would depend on browser's default behavior.
-



Note: The purpose of `cookieSameSite` is to avoid CSRF (Cross-Site Request Forgery).

6. Save the changes made.
7. Restart the OAuth Proxy for changes to take effect.

Configure Error Page URL

The OAuth Proxy Authorization endpoint is triggered to facilitate the OP's user authentication and consent. In the event of OAuth-related errors, the OP forwards the user to an error page instead.

oautherror.jsp is a simple error page included in the OAuth Proxy WAR that displays an error reference Id for troubleshooting use. You may customize and modify the error page URL to redirect users to a different error page instead.

To do so, perform the following configurations:

1. Navigate to `<PROXY_HOME>/WEB-INF/` and open **web.xml**.
2. Look for the `errorPageUrl` param-name, as shown below:

XML

```
<init-param>
  <param-name>errorPageUrl</param-name>
  <param-value>/oautherror.jsp</param-value>
</init-param>
```

3. Change the existing `/oautherror.jsp` param-value accordingly.



Note: Your error page must still be a local file. It cannot be an external URL.

4. Save the changes made.
5. Restart the OAuth Proxy for changes to take effect.

Configure OAuth Proxy Log Properties (Optional)

The OAuth Proxy uses Apache Log4j 2 as the logging utility. It defines the log level and log output file location.

To configure:

1. Navigate to `<PROXY_HOME>/WEB-INF/classes/` and open **log4j2.properties**.
-

-
2. Modify the log level, if necessary.
 3. Save the changes.
 4. Restart the OAuth Proxy for changes to take effect..

Configure Connection Pool Settings to AccessMatrix Server (Optional)

Connection pooling refers to the method of creating a pool of connections and caching those connections so that it can be reused again. It improves the response time of a web-based application. This setting is used to fine tune the connection to AccessMatrix server.

To configure:

1. Navigate to `<PROXY_HOME>/WEB-INF/` and open **web.xml**.
2. Search for the Connection Pool Configuration section.
3. Set the connection timeout value of the OAuth Proxy to AccessMatrix server by uncommenting and configuring the following parameter values:

XML

```
<context-param>
  <param-name>am.hc4.timeout.conn</param-name>
  <param-value>20000</param-value>
</context-param>
<context-param>
  <param-name>am.hc4.pool.maxconn_per_route</param-
name>
  <param-value>50</param-value>
</context-param>
```

4. You may add more connection pool parameters by referring to [AccessMatrix Common Login Page Deployment Guide - List of Configurable Parameters](#) section.
5. Save the changes.
6. Restart the OAuth Proxy for changes to take effect.

Configuring Cross-Origin Resource Sharing (CORS) for OIDC Endpoints

Refer to [Cross-Origin Resource Sharing \(CORS\) for OIDC Endpoints](#).

12. OpenID Connect Endpoints

The OpenID Connect endpoints are deployed in AccessMatrix OAuth Proxy WAR file (for custom login page deployment) or in AccessMatrix Common Login Page(CLP) WAR file (for default deployment using AccessMatrix CLP). These endpoint URLs need to be configured in the **OAuth Authz Server** configuration page in the Admin Console so that it is listed in the OpenID Connect Provider metadata when it is exported from Admin Console or retrieved from the `/.well-known/openid-configuration` endpoint which returns OpenID Connect metadata related to the specified authorization server.

Perform the following steps to configure the endpoints for the OpenID Connect Provider metadata or discovery:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Authz Server** and click **OpenID Connect Discovery** tab.
2. To avoid security threats, HTTPS is the recommended protocol for endpoint access. Specify the attribute values for the following endpoints:

Attribute Name	Description
Authorization Endpoint	Interact with the resource owner and obtain an authorization grant. For CLP/OAuthProxy: <CLP/OAuthProxy URL>/auth
Token Endpoint	Obtain access and/or ID token by presenting an authorization grant or refresh token. For CLP/OAuthProxy: <CLP/OAuthProxy URL>/token
User Info Endpoint	Returns claims about the authenticated end-user. For CLP/OAuthProxy: <CLP/OAuthProxy URL>/userinfo
JSON Web Keys (JWKS) URI	Returns the public keys used to sign the responses. For CLP/OAuthProxy: <CLP/OAuthProxy URL>/jwks.json
Revocation Endpoint	Revokes access or refresh token. For CLP/OAuthProxy: <CLP/OAuthProxy URL>/revoke
Introspection Endpoint	Checks the validity of access token or refresh token. For CLP/OAuthProxy: <CLP/OAuthProxy URL>/introspect
Backchannel Authentication Endpoint (only available in beta version)	Out-of-band authentication of the end-user from the Client to the OpenID Provider directly. For CLP/OAuthProxy: <CLP/OAuthProxy URL>/bc-auth

3. Save the changes to complete the setup.



Note: These refer to the endpoints from:

- **oauth-proxy.war** - if the project deployment is using custom login page

- **clp.war** (AccessMatrix Common Login Page) - if the project is used as the default authentication front end.

/auth

HTTP Method: GET/POST

Path: \${yourOAuthProxyURL}/auth

The OpenID Connect flows that are using HTTP browser redirect such as Authorization Code flow with PKCE, Implicit and Authorization Code flows will begin with this endpoint. This request authenticates the user and returns the tokens along with an authorization grant to the client application.

Request Parameters

Parameter	Data Type	Description	Mandatory
client_id	String	Obtained during OAuth client configuration creation. It identifies the client and it must match the value pre-registered in Admin Console.	TRUE
code_challenge	String	A challenge of PKCE. The challenge is verified in the access token request. Note: For Authorization Code flow, either state or code_challenge value must be specified. For more details, refer to PKCE (only for token exchange) .	TRUE (Only applicable when using Authorization Code flow with PKCE)
code_challenge_method	String	The method used to derive the code challenge for PKCE. Valid value: S256	FALSE (Only applicable when using Authorization Code flow with PKCE)
display	String	Specify on how to display the authentication and consent UI. Valid value:	FALSE
max_age	String	Allowable elapsed time (in seconds), since the last time the end-user was actively authenticated by AccessMatrix UAM.	FALSE
nonce	String	A value that will be returned in the ID token. It is used to mitigate replay attacks.	TRUE

Parameter	Data Type	Description	Mandatory
prompt	String	Valid values: none, consent, login, or consent and login in any order. See Parameter Details for more information.	FALSE
redirect_uri	String	The callback location where the authorization code or tokens should be sent. It must match the OAuth redirect URIs value preregistered in OAuth client configuration in the Admin Console during client registration. This value should be provided and it can be specified either in this parameter or in the form of request object, or in both parameters. To know more about request objects, check out the details of the request parameter.	TRUE
response_type	String	Values can be "code", "token", "id_token", or a combination of "token" and "id_token". The combination determines the flow.	TRUE
scope	String	openid is required for authentication requests. Other scopes may also be included.	TRUE
state	String	A value to be returned in the token. The client application can use it to remember the state of its interaction with the end-user at the time of the authentication call. It can contain alphanumeric, comma, period, underscore and hyphen characters. See Parameter Details for more information. Note: - For Authorization Code flow, either state or code_challenge value must be specified. - For Implicit flow, state value must be specified.	FALSE
acr_values	String	The requested Authentication Context Class Reference values. The space-separated string that specifies the acr values that the authorization server is being requested to use for processing the authentication request, with the values appearing in order of preference. The acr value is equivalent to the authentication level of the login module	FALSE

Parameter	Data Type	Description	Mandatory
		in AccessMatrix UAS server. The authentication level satisfied by the authentication performed is returned as the acr claim value.	
id_token_hint	String	The ID token previously issued by the authorization server being passed as a hint about the End-User's current or past authenticated session with the client.	FALSE
login_hint	String	A hint to the authorization server about the login identifier the end-user might use to log in. The parameter may have one of these values based on the format specified: "am_<browser_state>" - use this format to specify the browser_state value. "amsegid_<segmentId>" - use this format to specify the segment Id of the user. If the value is supplied, then the user is not asked to enter segment Id in CLP. Instead, the segment Id is automatically populated based on the specified segment Id value. Note: For CLP use only.	FALSE
request	String	This parameter expects a JWT string value which contains all the OIDC authentication request parameters. This JWT is called the request object. OpenID Connect request parameter values contained in the request object supersede those values that are passed in via OAuth 2.0 query parameters. Refer to Request Object for more details.	FALSE
response_mode	String	The mechanism to be used for returning Authorization Response parameters from the Authorization Endpoint. When specified, Authorization Response may be sent to redirect URI as part of query parameters, fragments, or POST parameters.	FALSE

Parameter Details

Parameter	Description
prompt	If no prompt parameter is specified, the standard behavior occurs: • If an AccessMatrix session already exists, the user is silently authenticated. Otherwise, the user is prompted to authenticate. • If scopes are requested and require consent, the user is then prompted to give consent if the consent has yet to be granted by the authentication user. There are three possible values for this parameter: • "none" - Do not prompt for authentication or consent. If an AccessMatrix session already exists, the user is silently authenticated. Otherwise, an error is returned. • "login" - Always prompt the user for authentication, regardless of whether they have an AccessMatrix session. • "consent" - Display the CLP consent dialog, even if the user has already given consent.
state	AccessMatrix UAM requires the OAuth 2.0 state parameter on all requests to the /authorize endpoint in order to prevent cross-site request forgery (CSRF). The OAuth 2.0 specification requires that clients protect their redirect URIs against CSRF by sending a value in the authorize request which binds the request to the user-agent's authenticated state. Using the state parameter is also a countermeasure to several other known attacks as outlined in OAuth 2.0 Threat Model and Security Considerations.

Response Result

The response will be redirected via the `redirect_uri` except when the request fails due to a missing or invalid redirect URI, or if the error is 'invalid_client' or 'unauthorized_client'.

In the event of such errors, the OP forwards the user to the configured error page instead. Refer to the [Configure Error Page URL](#) for configuration.

Field	Data Type	Description
access_token	String	An access token. This field is only returned if <code>response_type</code> includes "token".
code	String	An authorization code. This field is only returned if <code>response_type</code> includes "code".

Field	Data Type	Description
error	String	Error code.
error_description	String	Description of the error.
expires_in	String	Number of seconds until the <code>access_token</code> expires. This field is only returned if the response included an <code>access_token</code> .
id_token	String	An ID token. This field is returned if the <code>response_type</code> includes <code>id_token</code> .
state	String	The unmodified <code>state</code> value populated from the request.
token_type	String	The token type is always "Bearer" and is returned only when "token" is specified as a <code>response_type</code> .
browser_state	String	A random value to indicate the user login state. This value is only returned if the <code>response_type</code> includes "code".

Possible Errors

These errors are compliant with the [OpenID Connect and OAuth 2.0 Error Response Specifications](#).

Error	Details
access_denied	The server denied the request.
invalid_client	The specified OAuth client Id is invalid.
invalid_request	The request is missing a necessary parameter or the parameter has an invalid value.
server_error	The server encountered an internal error.
unsupported_response_type	The authorization server does not support obtaining an authorization code using this method.
login_required	Login is required either due to user authentication is expired or idle time out. In other cases, ID token hint may be mismatched with the active session, or when the request is specified with <code>prompt=none</code> , where the user should not expect any prompt and yet the user is still to be authenticated.
consent_required	User might have skipped the consent page or refused to grant consent when consent is required. In other cases, when the request is specified with <code>prompt=none</code> , where the user should

Error	Details
	not expect any prompt and yet the scope parameter specified in the authorization request requires the user to grant consent for certain scope(s).
insufficient_scope	User does not contain the necessary scopes to access the resource.

Examples

Authorization Code flow

Request Example		
This request initiates the Authorization Code flow as signalled by <code>response_type=code</code>. The request returns an authorization code that can be used as the <code>code</code> parameter in a token request.		
<table> <tr> <th>Code</th></tr> <tr> <td> <pre>https://{yourOAuthProxyDomain}/auth?client_id=dddbucvy c38HLL1ef0dde&response_type=code&scope=openid&redirect_uri=http%3A%2F%2Flocal host%3A8080&state=state-296bc9a0-a2a2-4a57-bela- d0e2fd9bb601&nonce=g5ly497e8ps'</pre> </td></tr> </table>	Code	<pre>https://{yourOAuthProxyDomain}/auth?client_id=dddbucvy c38HLL1ef0dde&response_type=code&scope=openid&redirect_uri=http%3A%2F%2Flocal host%3A8080&state=state-296bc9a0-a2a2-4a57-bela- d0e2fd9bb601&nonce=g5ly497e8ps'</pre>
Code		
<pre>https://{yourOAuthProxyDomain}/auth?client_id=dddbucvy c38HLL1ef0dde&response_type=code&scope=openid&redirect_uri=http%3A%2F%2Flocal host%3A8080&state=state-296bc9a0-a2a2-4a57-bela- d0e2fd9bb601&nonce=g5ly497e8ps'</pre>		
Response Example (Success)		
Result is a redirection URI with parameters in query component in "application/x-www-form-urlencoded" format:		
<table> <tr> <th>Code</th></tr> <tr> <td> <pre>HTTP 302 http://redirect_uri/?state=...&code=...</pre> </td></tr> </table>	Code	<pre>HTTP 302 http://redirect_uri/?state=...&code=...</pre>
Code		
<pre>HTTP 302 http://redirect_uri/?state=...&code=...</pre>		
Response Example (Error)		
The client is not valid:		
<table> <tr> <th>Code</th></tr> <tr> <td> <pre>http://redirect_uri/?error=invalid_client&state=...&error_description=client Id%2C+w0RSAACK_-dqW7lu5v1EnAa%2C+cannot+be+found</pre> </td></tr> </table>	Code	<pre>http://redirect_uri/?error=invalid_client&state=...&error_description=client Id%2C+w0RSAACK_-dqW7lu5v1EnAa%2C+cannot+be+found</pre>
Code		
<pre>http://redirect_uri/?error=invalid_client&state=...&error_description=client Id%2C+w0RSAACK_-dqW7lu5v1EnAa%2C+cannot+be+found</pre>		

Implicit flow

Request Example		
This request initiates the Implicit flow which gets an ID token and access token from the authorization server without the code exchange step. The same request is used as the first example but with <code>response_type=id_token</code> token: <table><tr><th>Code</th></tr><tr><td><pre>https://{yourOAuthProxyDomain}/auth?client_id=eeev6kx4qq6f adg0heff7&response_type=id_token token&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3 A8080&state=state-296bc9a0-a2a2-4a57-be1a- d0e2fd9bb601&nonce=foo</pre></td></tr></table>	Code	<pre>https://{yourOAuthProxyDomain}/auth?client_id=eeev6kx4qq6f adg0heff7&response_type=id_token token&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3 A8080&state=state-296bc9a0-a2a2-4a57-be1a- d0e2fd9bb601&nonce=foo</pre>
Code		
<pre>https://{yourOAuthProxyDomain}/auth?client_id=eeev6kx4qq6f adg0heff7&response_type=id_token token&scope=openid&redirect_uri=http%3A%2F%2Flocalhost%3 A8080&state=state-296bc9a0-a2a2-4a57-be1a- d0e2fd9bb601&nonce=foo</pre>		
Response Example (Success)		
Result is a redirection URI with parameters in fragment component in "application/x-www-form-urlencoded" format (the part of URL after the # is not sent to the server side): <table><tr><th>Code</th></tr><tr><td><pre>HTTP 302 http://redirect_uri/#state=...&expires_in=600&access_token=....</pre></td></tr></table>	Code	<pre>HTTP 302 http://redirect_uri/#state=...&expires_in=600&access_token=....</pre>
Code		
<pre>HTTP 302 http://redirect_uri/#state=...&expires_in=600&access_token=....</pre>		
Response Example (Error)		
The client is not valid: <table><tr><th>Code</th></tr><tr><td><pre>http://redirect_uri/?error=invalid_client&state=...&error_description=client Id%2C+w0RSAACK_-dqW7lu5vlEnAa%2C+cannot+be+found</pre></td></tr></table>	Code	<pre>http://redirect_uri/?error=invalid_client&state=...&error_description=client Id%2C+w0RSAACK_-dqW7lu5vlEnAa%2C+cannot+be+found</pre>
Code		
<pre>http://redirect_uri/?error=invalid_client&state=...&error_description=client Id%2C+w0RSAACK_-dqW7lu5vlEnAa%2C+cannot+be+found</pre>		

Request Object

Request object is a way to send OIDC Authentication Request in JWT format. All of the OIDC parameters that are normally sent to authorization endpoint can be packed inside the JWT as claims. The advantage of using request object is to add an authenticity that the request is coming from the said Relying Party as JWT is signed using the Relying Party's private key.



Note: For more details about Request Objects, refer to [Secure Authorization Request](#).

/token

HTTP Method: POST

Path: \${yourOAuthProxyURL}/token

This endpoint returns access tokens, ID tokens, and refresh tokens based on the request parameters. Calling this endpoint is the only step for resource owner password credentials (ROPC) flow or to refresh token. For the Authorization Code flow, calling this endpoint is the second step of the flow.

Request Parameters

The following parameters can be posted as a part of the URL-encoded form values to the API.

The call to the /token endpoint requires authentication. Authentication can be done via basic authentication of the client ID and secret or via the private key JWT authentication method. You can find the client ID and secret under the OAuth client basic information section.

Parameter	Data Type	Description	Mandatory
client_id	String	Obtained during OAuth client configuration creation. It identifies the client and must match the value preregistered in Admin Console. It is required if client does not have a secret, or if client has a secret (and Token endpoint authentication method is set to "ClientSecret") and client credentials are not provided in the authorization header. This field is required for the Client Credential flow where "client_credentials" is the grant_type.	FALSE
client_secret	String	Obtained during OAuth client configuration creation. It identifies the client and must match the value preregistered in Admin Console. It is required if client has a secret (and Token endpoint authentication method is set to "ClientSecret") and client credentials are not provided in the authorization header. This field is required for the Client Credential flow where "client_credentials" is the grant_type.	FALSE
client_assertion	String	Client-generated JWT signed with an asymmetric key to authenticate the client.	FALSE

Parameter	Data Type	Description	Mandatory
		Refer to OpenID Connect specification for the claim values contained in the JWT. This parameter is required for the private_key_jwt authentication method.	
client_assertion_type	String	An absolute URI that defines the format of the assertion. The value must be "urn:ietf:params:oauth:client-assertion-type:jwt-bearer". This parameter is required for the private_key_jwt authentication method.	FALSE
code	String	Required if grant_type is "authorization_code". The value specified here is obtained from the /auth endpoint. The code is only for one time use and has a life time of 10 minutes.	FALSE
code_verifier	String	Required if grant_type is "authorization_code" and "code_challenge" was specified in the original /authorize request. This value is the code verifier for PKCE. It is used to recompute the "code_challenge" and verify it against the original "code_challenge" in the authorization request.	TRUE (Only applicable for Authorization Code flow with PKCE)
grant_type	String	It can be one of the following values: "authorization_code" for the Authorization Code flow or Authorization Code flow with PKCE "password for Resource Owner Password Credential flow "client_credentials" for the Client Credential flow "refresh_token" for token refresh by the Authorization Code flow, Authorization Code flow with PKCE, Resource Owner Password Credential flow, or Hybrid flow.	FALSE

Parameter	Data Type	Description	Mandatory
		This parameter determines the flow and the protocol used by AccessMatrix UAM to create tokens.	
username	String	Required if the grant_type is "password".	FALSE
password	String	Required if the grant_type is "password".	FALSE
redirect_uri	String	Required if grant_type is "authorization_code". This is to specify the callback location where the authorization was sent. This value must match the redirect_uri used to generate the original "authorization_code".	FALSE
refresh_token	String	Required if grant_type is "refresh_token". This value is the refresh token that was returned from this endpoint previously.	FALSE
integrationDate	Date	Required if Use Refresh Token Integration Date attribute is set to "true" in OAuth client configuration of Authorization Code Flow for Server-side (Web) Application. This value is the date of the first account link-up. When the Use Refresh Token Integration Date attribute is set to "true", this value is used as the grant time of the token (where grant time refers to the starting time of the refresh token's validity). Note: This is used for SGFinDex.	FALSE
scope	String	Required if grant_type is "password". This is a list of scopes which clients wanted to include in the access token.	FALSE

Response Result

Based on the scopes requested, the scopes specified in a request are included in the access token in the response. The result is always JSON (based on standard) including error response.

HTTP response with a 200 (OK) status code for successful token response, 400 (Bad Request) / 401 (Unauthorized) for token error response.

Field	Data Type	Description
access_token	String	An access token. This field is only returned if response_type includes token.
expires_in	String	The number of seconds until the access_token expires. This field is only returned if the response included an access_token.
id_token	String	An ID token. This field is returned if openid scope is specified.
token_type	String	The audience of the token. The token type is always "Bearer".
refresh_token	String	An opaque refresh token. This is returned if the server side configuration of the OAuth client in the Admin Console is configured to allow the return of the refresh token for the Authorization Code flow and Resource Owner Password Credentials flow. For the refresh token grant, a refresh token will always be returned by default.

Possible Errors

These errors are compliant with the [OpenID Connect and OAuth 2.0 Error Response Specifications](#).

Error	Details
invalid_client	The specified OAuth client Id is invalid.
invalid_request	The request is missing a necessary parameter or the parameter has an invalid value.
invalid_grant	The provided authorization grant (e.g., authorization code, resource owner password credentials) or refresh token is invalid, expired, revoked, does not match the redirection URI used in the authorization request, or was issued to another client.
server_error	The server encountered an internal error.
unsupported_grant_type	The authorization grant type is not supported by the authorization server.
access_denied	The server denied the request due to invalid session given.
insufficient_scope	User does not contain the necessary scopes to access the resource.

Examples

Authorization Code flow with PKCE

Request Example (client secret authentication method)	
HTTP POST to <i>https://authzserver/oauth-proxy/token</i> with the following parameters: • grant_type=authorization_code • redirect_uri • code • code_verifier Both the client_id and password credentials are encoded and sent in the basic authorization header.	
Code	<pre>POST /oauth-proxy/token HTTP/1.1 Host: authzserver Content-Length: 307 Content-Type: application/x-www-form-urlencoded Authorization: Basic czZC..... grant_type=authorization_code&code_verifier=ragda6agja&redirect_uri=http%3A% 2F%2Flocalhost&code=ddg5g7mdxx...truncated...CSq</pre>
Request Example (private key JWT authentication method)	
HTTP POST to <i>https://authzserver/oauth-proxy/token</i> with the following parameters: • grant_type=authorization_code • code • scope=openid • redirect_uri • client_assertion_type (for private_key_jwt authentication method) • client_assertion (for private_key_jwt authentication method) • code_verifier	
Code	<pre>POST /oauth-proxy/token HTTP/1.1 Host: authzserver Content-Type: application/x-www-form-urlencoded grant_type=authorization_code& code=n0esc3NRze7LTCu7iYzS6a5acc3f0ogp4& code_verifier=grqtg993fag&</pre>

Request Example (client secret authentication method)	
	<pre>scope=openid& redirect_uri=http%3A%2F%2Flocalhost& client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3A client-assertion-type%3Ajwt-bearer& client_assertion=eyJhbGciOiJSUzI1NiIsImtpZCI6IjIyIn0. eyJpc3Mi[...omitted for brevity...]. cC4hiUPo[...omitted for brevity...]</pre>
Request Example (Success)	
Code	
	<pre>Result: HTTP 200 {"expires_in":600,"access_token":"04F1D7Dv3gmR-iHyyoxNarIo- oGHL21Q1O3Thk7VOYzMqNldCohf08AvXLLcThw15NHXkXuPocbKruww3ZuLzA48Dz94XJ_spwX9r SjG_V0AMgTw4-imdEB-Nd7YOXoO4Sus8","token_type":"Bearer"}</pre>
Response Example (Error)	
Code	
	<pre>HTTP 400 Bad Request {"error": "invalid_request"}</pre>

Refresh Token

Request Example (client secret authentication method)	
HTTP POST to <i>https://<authzserver>/oauth-proxy/token</i> with the following parameters: - grant_type=refresh_token - scope=openid - refresh_token Both the client_id and password credentials are encoded and sent in the basic authorization header.	
Code	
	<pre>POST /oauth-proxy/token HTTP/1.1 Host: authzserver Content-Length: 307 Content-Type: application/x-www-form-urlencoded</pre>

Request Example (client secret authentication method)

```
Authorization: Basic czZC.....
grant_type=refresh_token&scope=openid&refresh_token=Pgdgmdxx...truncated...C
Sq
```

Request Example (private key JWT authentication method)

HTTP POST to *https://authzserver/oauth-proxy/token* with the following parameters:

- grant_type=refresh_token
- refresh_token
- scope=openid
- client_assertion_type (for private_key_jwt authentication method)
- client_assertion (for private_key_jwt authentication method)

- grant_type=refresh_token
- refresh_token
- scope=openid
- client_assertion_type (for private_key_jwt authentication method)
- client_assertion (for private_key_jwt authentication method)

Code

```
POST /oauth-proxy/token HTTP/1.1
Host: authzserver
Content-Type: application/x-www-form-urlencoded
grant_type=refresh_token&
refresh_token=ggtrhdeha5akkh0ogp4&
scope=openid&
client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3A
client-assertion-type%3Ajwt-bearer&
client_assertion=eyJhbGciOiJSUzI1NiIsImtpZCI6IjIyIn0.
eyJpc3Mi[...omitted for brevity...].
cC4hiUPo[...omitted for brevity...]
```

Response Example (Success)

Code

```
Result: HTTP 200
{"expires_in":600,"access_token":...,"refresh_token":...,"token_type":"Bearer" }
```

Response Example (Error)

Code

```
HTTP 400 Bad Request
{"error": "invalid_request"}
```

/jwks.json

HTTP Method: GET

Path: \${yourOAuthProxyURL}/jwks.json

Returns a JSON Web Key Set (JWKS), which contains the public keys that can be used to verify the signatures of tokens that you received from the authorization server. Any of the two or three keys listed are used to sign tokens. The order of keys in the result does not indicate which keys are used. These keys can be used to locally validate JWTs returned by OAuth Proxy Endpoints. Standard open-source libraries are available for every major language to perform JSON Web Signature (JWS) validation.



Notes:

- Always retrieve keys dynamically with the JWKS published in the discovery/OAuth server metadata document.
- It is also recommended to do caching or persisting these keys to improve performance. If you cache signing keys, be aware that verification will fail when JWK keys are rotated manually. Clients that cache keys should periodically check the JWKS for updated signing keys.

JWK Properties

Property	Data Type	Description
alg	String	The algorithm used with the key. Valid value: RS256
e	String	RSA key value (exponent) for key blinding.
kid	String	The certificate's key ID.
key	String	Cryptographic algorithm family for the certificate's key pair. Valid value: RSA
n	String	RSA public modulus.
use	String	Refers to how the key is used. Valid value: sig

Examples

Response Example

```
JSON
```

```
{
  "keys": [
    {
      "kty": "RSA",
      "n": "sBRS8EIybdvqR7SoBTd7V3I_4F-
xycN3nlo8JYTfBvEnDjLMdZK7qBffvVLC8VvG6NDZ3hQDx1M-
PjjMD1TtOlPJOsS8tnTl83BKSGLxlFnVsWxeZ3x32Zi6YSXpF_Wf5lh0_gq6rC-
X6dWSm6rC_rCAw7U5i5cFsoL2c5uSKGAZDUFT_fwC7q0xPaS5lGb1L8_xa46Derbd-
ffOIcvDKLib5NJsrZWLJfrY3l9otgkgxM148sjutoBheFzXfJy93Ar3j5RzP0ped5
HeHyfliCfQHvTVKLlIbkU4HMPYIMITobKw4gIg2q7GmOn1xLbIXyfylJ2oh2F1B3VpYhhKlMw"
,
      "alg": "RS256",
      "e": "AQAB",
      "kid": "OAS-n-yp-V2yH8I",
      "use": "sig"
    },
    {
      "kty": "RSA",
      "n": "wkt5gsG2ZjKAeYmppPrUuLoIVxcCO_64z-
1Fmmta3Wm6u4o4SvDULuKP8Br5zadLk9tftToEyZfGIlQW
cq5Vr3StpJNMw6NcPMYN95KsOex4-
V3Ulr18bRHOSMIQoRPH_F3ZiYa4VjrEXoikkfognSCjkPdLvzK
N84qVjrIcvgmhuwCMYGLmV-G4szVx06JJTtb8kLKL-
c66zrtRROKE6NXO6Q2912vsM0ZxhIuDa5nb
_rJaw0MZnVukueaFaCU_v9h_AVsSk1-
RJAYm_uCeBHWKkG2zJ9VYBh4676H0kMln0sbwob3c6BwXaHE
BZ1W0ghXbiHvJ4PfeaE8pMOcK5w",
      "alg": "RS256",
      "e": "AQAB",
      "kid": "OAS-yljr82Jks6g",
      "use": "sig"
    },
    {
      "kty": "RSA",
      "n": "nwFoBNXdN1TsSlBUCRQ_46DDGi5LqrOoG027zRf6jD0UqA5sd6IvVOc-g-
0bXvTfa-jS7hi7faWIX6tV-
8kRyJUOPXvuvQkzvN81_EvEgen6UJzgwld0JjlMLEqaZ5FTx5VPe3iAUeUxxeCjGjSgw5yDkf
CW1EVSpY
8jq0OY88GUeO2FlCbQbDHcup6sKtad4HiBhEyPyOQIpJaRBBAQGsmMF1Bw9XIOEyL03B7XykJ
wbq1ZZkd
_xPfMo40QA1Xp52S0eQadJgZwbSGqlrc6YlziJ_vE6-
aHNKLz0unltrsYFaWFjUDYer_NLefnheRF9Zhr
tOrq8yweFon7W0rmeQ",
      "alg": "RS256",
      "e": "AQAB",
      "kid": "URBLlaxXwzyAceP352mXPK6ixUmGOeAQUDgJpKb6L-s",
      "use": "sig"
    }
  ],
  "version": 90
}
```

Key Rotation

Key Rotation only works if you configure to use OAuth authorization server to dynamically generate Json Web Token keys for you.



Note: Key Rotation is not supported for keystore configuration of JWKs keys which is a legacy practice. It is recommended to use **System generated key pair for signing in ID Token (JWT)** instead if you are currently using Key Store for your JWK keys.

The keys that are used to sign tokens can be manually changed. This function is located under **OAuth Authz Server** configuration in Admin Console.

To manually rotate key:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Authz Server**.
 2. Click **Generate OpenID Connect Signing Key Pair** to rotate signing key pair.
- If this is the first time you configure to use **System generated key pair for signing in ID Token (JWT)**, you would need to configure the following:
 - **RSA Key Length** - the RSA key length used to generate new RSA key pair for the signing of ID token. Default: 2048
 - **Wrapping Key** - this is required to generate OpenID Connect signing key pair. This key is used to protect system-generated signing key pair. Refer to [AccessMatrix Common Administration Guide - Configure Encryption Key](#) to know how to create a wrapping key.
 - **No. of public keys to remember for verification** - Value should not be less than 2. Default: 2

/userinfo



Note: Due to the possibility of token substitution attacks, the UserInfo Response is not guaranteed to be about the end-user identified by the sub (subject) element of the ID token. The sub claim in the UserInfo Response must be verified to exactly match the sub claim in the ID token. If it does not match, the UserInfo Response values must not be used.

HTTP Method: POST / GET

Path: \${yourOAuthProxyURL}/userinfo

Returns claims for the user based on the access token. Access token needs to be set with bearer header or with GET and bearer body.

Examples

Request Example	
Access token with bearer header:	
Code	
<pre>Authorization: Bearer \${access_token} https://{yourOAuthProxyURL}/userinfo</pre>	
Response Example (Success)	
JSON	
<pre>{ "sub": "00uid4BxXw6I6TV4m0g3", "name" : "John Doe", "email": "john.doe@example.com", "email_verified": true, "address" : { "street_address": "123 Hollywood Blvd.", "locality": "Los Angeles", "region": "CA", "postal_code": "90210", "country": "US" }, "phone_number": "+1 4255551212" }</pre>	

`/.well-known/openid-configuration`

HTTP Method: POST / GET

Path: `${yourOAuthProxyURL}/.well-known/openid-configuration`

Returns OpenID Connect metadata about your authorization server. Custom scopes and custom claims aren't returned. This API does not require any authentication.

Examples

Request Example

Code

```
https://{yourOAuthProxyDomain}/.well-known/openid-configuration
```

Response Result

Field	Data Type	Description
authorization_endpoint	String	URL of the authorization server's authorization endpoint.
claims_supported	Array	A list of the claims supported by this authorization server.
claim_types_supported	Array	A list of the claim types that the OpenID Connect Provider supports.
grant_types_supported	Array	JSON array containing a list of the grant type values that this authorization server supports.
issuer	String	The authorization server's issuer identifier. In the context of this document, this is your authorization server's base URL.
jwks_uri	String	URL of the authorization server's JSON Web Key Set document.
request_parameter_supported	Boolean	Setting to "true" will indicate those request parameters are supported by this authorization server.
response_modes_supported	Array	A JSON array containing a list of the <code>response_mode</code> values that this authorization server supports.
response_types_supported	Array	A JSON array containing a list of the <code>response_type</code> values that this authorization server supports. It can be a combination of "code", "token", and "id_token".

Field	Data Type	Description
scopes_supported	Array	A JSON array containing a list of the scope values that this authorization server supports.
subject_types_supported	Array	A JSON array containing a list of the Subject Identifier types that this authorization server supports. Valid types include pairwise and public.
token_endpoint	String	URL of the authorization server's token endpoint.
token_endpoint_auth_methods_supported	Array	A JSON array containing a list of client authentication methods supported by this token endpoint.
userinfo_endpoint	String	URL of the authorization server's user info endpoint.
id_token_signing_alg_values_supported	String	A list of the JSON Web Signature (JWS) signing algorithms (alg values) supported by the OpenID Connect Provider for the ID token to encode the claims in a JSON Web Token (JWT).

/revoke

HTTP Method: POST

Path: \${yourOAuthProxyURL}/revoke

The API takes an access or refresh token and revokes it. A client may only revoke its own tokens.

Request Parameters

The following parameters can be posted as a part of the URL-encoded form values to the API.

The call to the /revoke endpoint requires authentication. Authentication can be done via, basic authentication of the client ID and secret, or private key JWT. You can find the client ID and secret under the OAuth Client basic information section.

Parameter	Data Type	Description	Mandatory
client_id	String	The client ID generated as a part of client registration. This parameter is required for authentication using client_secret.	FALSE
client_secret	String	Required if the client has a secret and client credentials are not provided in the authorization header, and if the client_assertion_type isn't specified. For the Authorization Code with PKCE flow, this parameter is not required.	FALSE
client_assertion	String	Client-generated JWT signed with an asymmetric key to authenticate the client. Refer to OpenID Connect specification for the claim values contained in the JWT. This parameter is required for the private_key_jwt authentication method.	FALSE
client_assertion_type	String	An absolute URI that defines the format of the assertion. The value must be "urn:ietf:params:oauth:client-assertion-type:jwt-bearer". This parameter is required for the private_key_jwt authentication method.	FALSE
token	String	An access or refresh token.	TRUE
token_type_hint	String	Type of token. Valid values are access_token and refresh_token.	FALSE

Response Properties

A successful revocation is denoted by an HTTP 200 OK response.

Possible Errors

Error	Details
invalid_client	The specified OAuth client ID is invalid.
invalid_request	The request is missing a necessary parameter or the parameter has an invalid value.
server_error	The server encountered an internal error.
invalid_token	The access token, authorization code or refresh token is invalid.

/introspect

HTTP Method: POST

Path: \${yourOAuthProxyURL}/introspect

This endpoint provides a way for authorized component/server to check the validity of access token or refresh token. This means this endpoint can be used if there is a revocation of the access token or refresh token. The endpoint returns the active attribute that has the value of "true" when the access/refresh token is valid for the authorized server or "false", otherwise. For active=true, this endpoint also returns the information of the provided token, such as the user's identity (subject) and expiry.

The introspection endpoint is meant to be used by the server-side component; thus, it requires authentication.

Authentication can be done using one of these methods:

- **client_secret_basic:** client ID and client secret in Authorization header with basic authentication
- **client_secret_post:** client ID and client secret in POST request parameters
- **private_key_jwt:** JWT in POST request parameter. The JWT is signed with a private key that corresponds to the registered public key.
- **Bearer token:** client credential access token as a bearer token in the Authorization header

Introspection Use Cases

- **Own Use :** The introspection endpoint can be called by the same OAuth client whom token is issued to.

-
- **API Gateway Use:** Optionally, it can also be called by the Introspection OAuth client on behalf of other OAuth client . It is useful for API Gateway to be an Introspection OAuth client where it can call introspection endpoint on behalf of the application (other OAuth client).

Below are the steps to achieve that:

1. Create Server-to-server OAuth client for API Gateway with the **Allowed to call introspection for other OAuth clients** attribute set to "true". Click [here](#) for more details.
2. API Gateway obtain an introspection access token using client credential authorization request. Click [here](#) for more details.
3. When calling the introspection endpoint on behalf of other OAuth client, API Gateway needs to use the corresponding introspection access token as its client authentication's bearer token.

i **Note:** If a bearer token is used to call the introspection endpoint on behalf of another OAuth client, the configured client authentication method will be disregarded. OAuth client authentication will pass if the bearer token is valid and **Allowed to call introspection for other OAuth clients** is set to "true" in the Introspection OAuth client.

Request Parameters

The following parameters can be posted as a part of the URL-encoded form values to the API.

Parameter	Data Type	Description	Mandatory
token	String	An access or refresh token.	TRUE
token_type_hint	String	A hint about the type of the token submitted for introspection, either refresh_token or access_token.	FALSE
client_id	String	Obtained during OAuth client configuration creation. It identifies the client and it must match the value preregistered in Admin Console. This parameter is required in general, unless gateway's bearer token is used. This is optional for the private_key_jwt authentication method.	FALSE

Parameter	Data Type	Description	Mandatory
client_secret	String	Obtained from OAuth client configuration. For the Authorization Code with PKCE flow and private_key_jwt authentication method, this parameter is not required.	FALSE
client_assertion	String	Client-generated JWT signed with an asymmetric key to authenticate the client. Refer to OpenID Connect specification for the claim values contain in JWT. This parameter is required for the private_key_jwt authentication method.	FALSE
client_assertion_type	String	An absolute URI that defines the format of the assertion. The value must be "urn:ietf:params:oauth:client-assertion-type:jwt-bearer". This parameter is required for the private_key_jwt authentication method.	FALSE

Response Result

Field	Data Type	Description
active	Boolean	Indicate whether or not the presented token is currently active.
scope	String	Contains a space-separated list of scopes associated with this token.
client_id	String	Client identifier for the OAuth 2.0 client that requested this token.
username	String	Human-readable identifier for the resource owner who authorized this token.
token_type	String	Type of the token, either access_token or refresh_token.

Field	Data Type	Description
exp	Integer	Integer timestamp, measured in the number of seconds since January 1 1970 UTC, indicating when this token will expire.
iat	Integer	Integer timestamp, measured in the number of seconds since January 1 1970 UTC, indicating when this token was originally issued.
jti	String	Identifier for the token.
sub	String	Subject of the token, as defined in JWT [RFC7519]. Usually a machine-readable identifier of the resource owner who authorized this token.
aud	String	Service-specific string identifier or list of string identifiers representing the intended audience for this token. This value is configured with OAuth client Id authorized by resource owner for now.
iss	String	Issuer of the token.
error	String	OAuth error code, e.g. invalid_request, invalid_client.
error_description	String	A detailed description of the OAuth error.

Possible Errors

These errors are compliant with the [OpenID Connect and OAuth 2.0 Error Response Specifications](#).

Error	Details
invalid_client	The specified OAuth client Id is invalid.
invalid_request	The request is missing a necessary parameter or the parameter has an invalid value.
invalid_grant	The provided authorization grant (e.g., authorization code, resource owner password credentials) or refresh token is invalid, expired, revoked, does not match the redirection URI used in the authorization request, or was issued to another client.
server_error	The server encountered an internal error.
unsupported_grant_type	The authorization grant type is not supported by the authorization server.

Error	Details
access_denied	The server denied the request due to invalid session given.
insufficient_scope	User does not contain the necessary scopes to access the resource.

Examples

Client Authentication using client ID and client secret in URL-encoded POST parameters	
Code	<pre>POST /oauth-proxy/introspect HTTP/1.1 Host: {yourDefaultOAuthProxyDomain}.com Content-Length: 307 Content-Type: application/x-www-form-urlencoded client_id=kU9kFJTTRicPqL- 12k8imvg&client_secret=nCYPSeU0btJbkZzrUogfsQTb&token=060001...truncated ...Ez9c&token_type_hint=access_token</pre>
Client Authentication using client ID and client secret in Authorization header with basic authentication	
Code	<pre>POST /oauth-proxy/introspect HTTP/1.1 Host: {yourDefaultOAuthProxyDomain}.com Content-Length: 307 Content-Type: application/x-www-form-urlencoded Authorization: Basic a1U5a0ZKVFJJY1BxTC1sMms4aW12ZzpuQ1lQU2VVMGJ0SmJrWnpyVW9nZnNRVGI= token=060001...truncated...Ez9c&token_type_hint=access_token</pre>
Client Authentication using private key JWT in POST parameters	
Code	<pre>POST /oauth-proxy/introspect HTTP/1.1 Host: {yourDefaultOAuthProxyDomain}.com Content-Length: 307 Content-Type: application/x-www-form-urlencoded client_assertion=kU9kFJTTRicPqL- 12k8imvg&client_assertion_type=urn:ietf:params:oauth:client- assertion-type:jwt-bearer&token=060001...truncated ...Ez9c&token_type_hint=access_token</pre>

Client Authentication using client ID and client secret in URL-encoded POST parameters	
Client Authentication using bearer token in the Authorization header	
Code	<pre>POST /oauth-proxy/introspect HTTP/1.1 Host: {yourDefaultOAuthProxyDomain}.com Content-Length: 307 Content-Type: application/x-www-form-urlencoded Authorization: bearer 0600019fM7uP7phuAfMvX6bFkqVqHik0CHbr...truncated...aKqxwsmXkvDfz0lYjFi6KXHTZ Ns token_type_hint=access_token&token=060001LPhEgUO-...truncated...- JYS1UYvYYe7QypBafFhlv68Pfs5n4s2YIP25Xk</pre>
Response Example (Success)	
JSON	<pre>{ "aud": "kU9kFJTRICpQL-l2k8imvg", "sub": "demouser", "iss": "i-sprint.com", "active": true, "token_type": "access token", "client_id": "kU9kFJTRICpQL-l2k8imvg", "jti": "pUMxpd5au7gDZWSjYyxSjQ", "username": "Demo User" "scope": "openid" "exp": "1449627626" "iat": "1449624026" }</pre>
Response Example (Error)	
JSON	<pre>HTTP 400 Bad Request {"error": "invalid_request"}</pre>

Optimizing introspection of Access Token in JWT Format

Access token, generated by AccessMatrix as OpenID Provider, can be generated in three formats depending on the configuration: Proprietary, JWT (JSON Web Token), and JWE (JSON Web Encryption).

Validation of Access Token format in JWT involves some cryptography operation using asymmetric encryption, the JWT validation requires significant amount of computing power from AccessMatrix

server. In order to make this process more scalable, OAuth Proxy will validate the JWT locally whereas the opaque access token would be validated by the AccessMatrix server instead (and also to retrieve the required information).

This behavior (to have OAuth Proxy validating JWT locally at the introspect endpoint) is enabled by default. If for some security reasons (e.g. OAuth Proxy hardening), JWT validation processing by OAuth Proxy is deemed not required or needs to be disabled, navigate to `<CLP-DIR>\WEB-INF\` and open the **web.xml** file to modify the configuration as shown below:

XML

```
<context-param>
  <param-name>access-token.jwt.validation</param-
name>
  <param-value>false</param-value>
</context-param>
```



Notes:

- If OAuth Authorization Server's issuer is modified in AccessMatrix Server, OAuth Proxy needs to be restarted. The issuer value is cached as it is not common to modify issuer.
- Access token in JWE (JSON Web Encryption) cannot be introspected as it would require Relying Party's private key to decrypt it. If Relying Party wishes to introspect the access token, the JWE needs to be decrypted first. The am_at claim value, which is the proprietary access token, can then be sent to the introspection endpoint.

13. Advanced Features

How to Achieve Single Logout (SLO) in OIDC

AccessMatrix Universal Access Management (UAM) supports the Single Logout (SLO) feature in OIDC. With SLO, it allows users to invalidate all other active sessions the moment a logout has been initiated. This provides added convenience and security by ensuring that other malicious third parties cannot misuse the remaining active sessions. This feature can be triggered from either the OAuth Client or the OpenID Provider (OP).



Note: For more information on how to configure SLO, you may refer to [Revoke OIDC token when OP session is invalidated](#) and [Invalidate OP session when OIDC token is invalidated](#).

Re-authentication for Sensitive Operation

For sensitive or high-risk operations where additional security is required, re-authentication can be used to provide an additional layer of protection against bad actors. Re-authentication is a process to ensure that the user validates their identity and presence again, even though the initial authentication has been successful.

In OIDC, this is achieved by passing either of the following parameters in every authorization request to the /auth endpoint.



Note: To prevent hackers from spoofing relying party (RP) using the old tokens (e.g. ID token or/and access token not yet expired), RP must check nonce and authentication time upon receiving its response from the OP.

- `prompt=login`

By using this parameter, the user will always be prompted to login regardless of existing active sessions.

- `max_age=<value>`



Note: Value is specified in seconds.

By using this parameter, the OP will re-authenticate the end user if current time - last authentication time is less than the value specified in the `max_age` parameter.

In other contexts, you may wish to enforce multi-factor authentication for sensitive operations. This can be done by configuring the:

-
- **Required Authentication Level** attribute in OAuth Client using the Admin Console.

This enforces the required authentication level for all the users of the corresponding OAuth client.



Note: This setting is not selectively applied based on a target user group or rule. For more information on how to configure this attribute and other configurations, refer to [Access Control](#).

- **acr_values** parameter

This parameter functions similarly to a login module's **Authentication Level** attribute in the Admin Console. It enforces the specified authentication level upon every authorization request. This ensures that the user is authenticated with the corresponding authentication level. If multiple values have been configured for this parameter, then the user will be authenticated using the lowest authentication level required.



Note: The **acr_values** parameter does not overwrite the **Required Authentication Level** attribute in the OAuth client configurations.

14. Customizations

Login Page of /auth Endpoint

Refer to [Login Page](#).

Consent Page of /auth Endpoint

Refer to [Consent Page](#).

Retrieving User Info or Custom Claims

AccessMatrix provides many alternatives of returning user info back to the OAuth Client. Generally, User Info or custom claims can be retrieved from the User Response.



Note: For more details about User Response, refer to [UAM Administration Guide - User Response Configuration](#).

The following options allow for returning user info or custom claims:

- **Retrieval via User Info endpoint**



Note: For more details about the User Info endpoint, refer to [/userinfo](#). This option is dependent on the requested scope, which can be configured to indicate if consent is required.

- via [scope](#)

- **Retrieval from ID Token (JWT or JWE format)**

- via [scope](#)
- via [user response](#)

- **Retrieval from Access Token (JWT or JWE format)**

- via [scope](#)
- via [user response](#)

The following options allow for returning custom claims only:

- **Retrieval from Authorization Code (JWT format)**

- via [user response](#)

- **Retrieval from Refresh Token (JWT format)**

- via [user response](#)
-

Embed Additional User Info in Access Token

AccessMatrix provides a way for additional information to be sent to the Replying Party without the use of scopes.

Example use-cases:

- If the additional information is only required/desired to a specific Relying Party and it is not desirable (or safe) to be made available and configurable (as scopes) for other clients.
- If mandatory info is to be returned and does not require user consent where the format of the token could be in JWT (not confidential) or in JWE (confidential and security sensitive).

Perform the following steps to embed more info to access token for the outlier client:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Verify that the **Access Token Format** attribute is configured either with "JWT" or "JWE".
5. Select the user response to be embedded using the **Access Token User Response** attribute.



Note: This field can be empty (i.e. specified as "-"). For more details, refer to [UAM Administration Guide - User Response Configuration](#)

6. Click **Save**.

Embed Additional User Info in ID Token

AccessMatrix provides a way for additional information to be sent to the Replying Party without the use of scopes. Example use-case could be that the additional information is only required/desired to a specific Relying Party and it is not desirable (or safe) to be made available and configurable (as scopes) for other clients access.

Perform the following steps to embed more info to ID token for the outlier client:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
 2. Click on the relevant OAuth client under the **OAuth Client Id** column.
 3. Click the **Edit** button and select the **OpenID Connect** tab.
-

-
4. Select the user response to be embedded using the **Attribute Claim Value Response Configuration** attribute.



Note: This field can be empty (i.e. specified as "-"). For more details, refer to [UAM Administration Guide - User Response Configuration](#).

5. Click the **Save** button to save the changes.

Embed Additional Custom Claims in Authorization Code

AccessMatrix allows for additional information to be sent with the authorization code by using JWTs.



Note: This feature is only available for flows that use authorization code. For more information on the configurable attributes, refer to the following links:

- Authorization Code with PKCE Flow
- Hybrid with PKCE Flow
- Authorization Code Flow (Deprecated)
- Hybrid Flow (Deprecated)

Example use cases:

- If there is a need for the OP to distinguish between two siloed deployments of UAM OP with separate databases but share the same OP issuer Id. By adding custom claims, the API gateway or proxy server can route the request to the appropriate UAM server (OAuth Proxy / Common Login Page) for processing.

Perform the following steps to embed more info in the authorization code:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
 2. Click on the relevant OAuth client under the **OAuth Client Id** column.
 3. Click the **Edit** button.
 4. Select "JWT" using the drop-down list corresponding to the **Authorization Code Format** attribute.
 5. Select the user response to be embedded using the **Authorization Code User Response** attribute.
-

i **Note:** This field can be empty (i.e. specified as "-"). For more details, refer to [UAM Administration Guide - User Response Configuration](#).

6. Click the **Save** button to save the changes.

Embed Additional Custom Claims in Refresh Token

AccessMatrix allows for additional information to be sent with the refresh token by using JWTs.

i **Note:** This feature is only available for flows that use authorization code. For more information on the configurable attributes, refer to the following links:

- Authorization Code with PKCE Flow
- Hybrid with PKCE Flow
- Authorization Code Flow (Deprecated)
- Hybrid Flow (Deprecated)

Example use cases:

- If there is a need to distinguish between two siloed deployments of UAM with separate databases but share the same OP issuer Id. By adding custom claims, the API gateway or proxy server can route the request to the appropriate UAM server (OAuth Proxy / Common Login Page) for processing.

Perform the following steps to embed more info in the refresh token:

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Federated SSO > OAuth 2.0 & OpenID Connect > OAuth Client**.
2. Click on the relevant OAuth client under the **OAuth Client Id** column.
3. Click the **Edit** button.
4. Select "JWT" using the drop-down list corresponding to the **Refresh Token Format** attribute.
5. Select the user response to be embedded using the **Refresh Token User Response** attribute.

i **Note:** This field can be empty (i.e. specified as "-"). For more details, refer to [UAM Administration Guide - User Response Configuration](#).

6. Click the **Save** button to save the changes.
-

Customize User Info Using User Response Plugin

Alternatively, you may create a custom plugin Java class for the retrieval of user info. To learn more about how this can be configured and deployed using the User Response interface, refer to [AccessMatrix Common Administration Guide - User Response Configuration - User Custom Attributes \(Custom Plug-In Class\)](#).

Other Endpoints (Not recommended)

Below are the endpoint equivalents of the REST APIs available for different customization uses. For example, they can be used to customize the OIDC endpoints or to directly call certain REST APIs.

/auth

- Refer to authz/oauth/authzReq/read of the [AccessMatrix REST API Specification](#) to validate the authentication request.
- Refer to authz/oauth/authzReq/process of the [AccessMatrix REST API Specification](#).

/token

- Refer to authz/oauth/accessTokenReq/process of the [AccessMatrix REST API Specification](#) to process the Authorization Code flow of the the token endpoint.
- Refer to authz/oauth/refreshTokenReq/process of the [AccessMatrix REST API Specification](#) to process the refresh token flow of the the token endpoint.
- Refer to authz/oauth/ROPCReq/process of the [AccessMatrix REST API Specification](#) to process the Resource Owner Password Credential flow of the the token endpoint.
 - This method expects end user has already been authenticated accordingly and session token is obtain for this API call.

/introspect

- Refer to authz/oauth/accessToken/verify of the [AccessMatrix REST API Specification](#).



Note: Currently, this method only verifies access token of end user.

- Refer to authz/oauth/token/introspect of the [AccessMatrix REST API Specification](#).

/revoke

- Refer to authz/oauth/revokeTokenReq/process of the [AccessMatrix REST API Specification](#).
-

/userinfo

- Refer to authz/oauth/userinfoReq/process of the [AccessMatrix REST API Specification](#).

/keys

- JWKS can be obtained from Admin console via `${YourAM5URL}/am5/wpe/jwks.json`.

/.well-known/openid-configuration

- OpenID Connect Provider metadata can be obtained from Admin console via `${YourAM5URL}/am5/wpe/.well-known/openid-configuration`.

Customize Refresh Token to Embed OP Session



Note: If performance is an important consideration, it is recommended that you do not embed the session token.

If **Embed Session Token** is set to "true" in OAuth client, then the following default expiry will be set:

- Session token idle time is defined in the realm-based session policy of the authentication realm or in global setting.
- The expiry of ID token, access token, and refresh token would be based on the **Session Idle Time** defined in the realm-based session policy of the authentication realm or in global setting.
- For refresh token, the expiry is **Session Idle Time** minus 60 seconds.

Configuration rules:

- There is no need to configure for ID token and access token expiry/validity when session token is embedded.
 - Server-side application must refresh tokens using refresh token latest by 60 seconds before session token idle time out.
-

15. Appendices

A. FAQ

What type of OAuth Client for API Gateway that can introspect tokens is recommended?

Client Credentials Flow OAuth Client is recommended for backend server to OP where the API Gateway is requesting access to protected resources based on an authorization previously arranged with the authorization server. The API Gateway can also introspect on behalf of other OAuth Clients if by configuring the attribute "**Allowed to call introspection for other OAuth Clients**" to "**true**". Refer to [Configure the OAuth Client - Client Credentials](#).

How to return additional information in the OIDC token?

Additional information can be configured to be returned as claims in the OIDC token. This can be done via different methods to suit different needs. Please refer to [Customizations - Access Token](#), [Customizations - ID token](#), [Customizations - User Info claim](#).

Auditing for UAM OIDC Events

The AccessMatrix UAM authorization server audits important OpenID Connect (OIDC) completed actions, such as the retrieval of a user's identity (for example, the user's phone number, email, address, profile, and more), the generation of OIDC tokens, and others. Thereafter, the administrator can create a report to view the audited events.

Currently, AccessMatrix UAM supports the following audit events:

Event Type	Description	Fields
OAuth-Consent	This audit event is raised when the user grants consent to a Relying Party to access their identity or any other scopes.	client_id
		scopes
OAuth-ConsentRemoved	This audit event is raised when the user revokes consent given to a Relying Party to access their scopes.	client_id
		removed_scopes
		clientUUID

Event Type	Description	Fields
OAuth-Token	This audit event is raised at the token exchange step of the authorization request flow.	client_id
		authzCode
		accessToken
		refreshToken
		realmId
OAuth-Authz	This audit event is raised at the end of the authorization request flow when the authorization code and/or access token and/or ID token is returned.	client_id
		accessTokenId
		idTokenId
		realmId
		authzCode
		accessToken
		response_type
OAuth-RevokeToken	This audit event is raised when an access token or refresh token is revoked.	client_id
		token
		realmId
		token_type
		accessTokenId
		refreshTokenId
OAuth-Introspect	This audit event is raised when an access token or refresh token is introspected.	client_id
		token
		token_type
		realmId
OAuth-Refresh	This audit event is raised when an access token is refreshed.	client_id
		token
		accessToken

Event Type	Description	Fields
		refreshToken
		realmId
OAuth-UserInfo	This audit event is raised when claims about an authenticated end-user are requested by a Relying Party.	client_id
		token
		realmId



Note: Refer to [AccessMatrix Common Administration Guide - Audit Policy](#) for more information.

By default, the event types "OAuth-Authz", "OAuth-RevokeToken", and "OAuth-Introspect" are excluded in the audit policy. To ensure they are included in the generated report, perform the following steps in the Admin Console:

1. Navigate to **Administration Dashboard > Security Policy > Admin & Audit > Audit Policy** and click **Edit**.
2. Locate the **Filter out the following event categories and types without logging** attribute under the **Audit events that will not be recorded** header.
3. Select the OAuth event types you want to include and click the **Remove** button.
4. Save your changes and exit.

Next, to view the above OAuth audit events, generate either an Events Report or User Activities Report in the Admin Console by following the steps below:

1. Navigate to either:
 - a. Events Report - **Report Dashboard > System & Admin > Historical > Events**.
 - b. User Activities Report - **Report Dashboard > Segment & User > Historical > User Activities**.
2. For **Event Category**, ensure **All Event Categories** is selected.
 - a. Alternatively, click **Select Event Category** and select **OAuth**.
3. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

Auditing for OAuth Client Events

The AccessMatrix UAM authorization server audits events relating to OAuth clients, such as the creation and modification of an OAuth client. Thereafter, the administrator can create a report to view the audited events.

These events are audited under the following categories:

Event Type	Description
Modification-Add	In the context of OAuth clients, this audit event is raised when a new OAuth client is created.
Modification-Delete	In the context of OAuth clients, this audit event is raised when an OAuth client is deleted.
Modification-Modify	In the context of OAuth clients, this audit event is raised when an OAuth client is modified. This includes events such as the reset/modification of the OAuth client's client secret.

1. Navigate to either:
 - a. Events Report - **Report Dashboard > System & Admin > Historical > Events.**
 - b. User Activities Report - **Report Dashboard > Segment & User > Historical > User Activities.**
2. For **Event Category**, ensure **All Event Categories** is selected.
 - a. Alternatively, click **Select Event Category** and select **Modification.**
3. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

Auditing for User/ User Group OIDC Privileges

The AccessMatrix UAM authorization server audits the granting/removal of OIDC client privileges to/from users and user groups. The administrator can create a report to view the audited events, which are as follows:

Event Type	Description	Fields
Principal-AddOAuthClientPrivilege	This audit event is raised when a user/user group is granted an OAuth client privilege.	client_id
Principal-RemoveOAuthClientPrivilege	This audit event is raised when a user's/user group's OAuth client privilege is removed.	client_id

To view the above audit events, generate either an Events Report or User Activities Report in the Admin Console by following the steps below:

1. Navigate to either:
 - a. Events Report - **Report Dashboard > System & Admin > Historical > Events.**
 - b. User Activities Report - **Report Dashboard > Segment & User > Historical > User Activities.**
2. For **Event Category**, ensure **All Event Categories** is selected.
 - a. Alternatively, click **Select Event Category** and select **Principal**.
3. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

Auditing for Expired OAuth Token Housekeeping

The AccessMatrix UAM server regularly persists OAuth tokens (e.g. refresh token or access token) as part of the OIDC flows.

The server removes OAuth tokens that have expired, as well as all tokens associated with deleted OAuth clients, at a fixed time each day via a backend scheduler. These events are audited under the following categories:

Event Type	Description	Fields
ExpiredOAuthTokenCleaner-Initialized	This audit event is raised when the	nextExecTime

Event Type	Description	Fields
	Expired OAuth Token Cleaner is initialized.	
ExpiredOAuthTokenCleaner-Started	This audit event is raised when the Expired OAuth Token Cleaner begins purging expired OAuth tokens.	startTime
		nextExecTime
ExpiredOAuthTokenCleaner-Completed	This audit event is raised when the Expired OAuth Token Cleaner has completed the purging of expired OAuth tokens.	expiredOAuthTokenDetail
		startTime
		endTime
		elapsedTime
		nextExecTime

1. Navigate to either:
 - a. Events Report - **Report Dashboard > System & Admin > Historical > Events.**
 - b. User Activities Report - **Report Dashboard > Segment & User > Historical > User Activities.**
2. For **Event Category**, ensure **All Event Categories** is selected.
 - a. Alternatively, click **Select Event Category** and select **ExpiredOAuthTokenCleaner**.
3. Input the report criteria, select the report format (i.e. "HTML", "PDF" or "CSV") and click the **Generate** button.

B. UAM OAuth and OIDC Error Messages

This section describes some of the OAuth-related errors as defined by OAuth RFC 6749. The errors are returned via the redirection URL during authorization request and it is also returned in response body (JSON format) during token request.

Note: These errors correspond to the error code 200.

Error Class Name	Error Description
invalid_request	The request is missing a required parameter; it includes an invalid parameter value; it includes a parameter more than once; or is otherwise malformed.

Error Class Name	Error Description
invalid_client	The client authentication failed (e.g. unknown client; no client authentication included; or unsupported authentication method).
invalid_grant	The provided authorization grant (e.g. authorization code, resource owner credentials) or refresh token is invalid, expired, or revoked and does not match the redirection URI used in the authorization request, or it was issued to another client.
unauthorized_client	The client is not authorized to request an authorization code using this method.
access_denied	The resource owner or authorization server denied the request.
unsupported_response_type	The authorization server does not support obtaining an authorization code using this method.
invalid_scope	The requested scope is invalid, unknown, or malformed. Solutions: • Ensure that all server times are in sync if there are multiple OAuth proxy servers that are tied to the OAuth proxy domain URL (e.g. using a load balancer). • This could also be a sign of a replay attack where expired consent token is used for the OAuth flow. To mitigate this attack, configure the CLP Consent Validity Period (in seconds) attribute in the OAuth Authz Server. Do not set the tolerance window to be too wide, as this process occurs quickly and might compromise the security of the flow.
server_error	The authorization server encountered an unexpected condition that prevented it from fulfilling the request. (This error code is needed because a 500 Internal Server Error HTTP status code cannot be returned to the client via an HTTP redirect)
login_required	Silent authentication requires active user session. Login is required when the user is not logged in.
consent_required	Silent authentication requires user consent.

C.Validating JWTs

Perform the following steps to validate the JWT:

i **Note:** For more information, refer to their respective sections.

-
1. Retrieve and parse the AccessMatrix UAM JSON Web Keys (JWK).
 2. Decode JWT.
 3. Verify the signature used to sign the JWT.
 4. Verify the claims found inside the JWT.

Refer [here](#) for the sample code. The sample code provided verifies the `at_hash` and `nonce` claims.

Retrieve JSON Web Keys

The JSON Web Key Set (JWKS) can be retrieved from the AccessMatrix UAM OAuth Proxy endpoint (`/jwks.json`). Alternatively, OAuth authorization server's metadata endpoint (`/.well-known/openid-configuration`) contains the `jwks_uri`, which can be used to obtain the JWKS.

Verify JWT Signature

Verify the JWT's signature by matching the key that was used to sign with one of the keys retrieved from the AccessMatrix UAM OAuth Proxy's JWK endpoint. Specifically, each public key is identified by a `kid` attribute, which corresponds with the `kid` claim in the ID token header.

If the `kid` claim does not match, it is possible that the signing keys have changed. Check the `jwks_uri` value in the OAuth authorization server metadata and try retrieving the keys again from the JWK endpoint.



Note:

- Key rotation of the token signing key has to be done manually.
- To save the network's round-trip, the app should cache the `jwks_uri` response locally. The standard HTTP caching headers are used and should be respected.
- Keys used to sign tokens manually rotate and should always be resolved dynamically against the published JWKS. The app might fail if public keys are hardcoded in the applications. Be sure to include key rollover in your implementation.
- If the application cannot retrieve keys dynamically, the administrator can use the static signing key by configuring keystore of the signing key in OAuth authorization server.

Verify the Claims

For access tokens, you may need to verify the following claims:

-
- The `iss` (issuer) claim matches the Issuer of the AccessMatrix UAM Authorization Server configuration in Admin Console.
 - The `aud` (audience) claim must match the Client ID that you used to request the JWT. This will be the Client ID for the application created in Admin Console
 - The `client_id` claim should be similar to `aud` (audience) value.
 - The `iat` (issued at time) claim indicates when the JWT was issued, expressed in Unix time.
 - The `exp` (expiry time) claim indicates the time that the JWT will expire, expressed in Unix time. Ensure it has not been passed.

For ID token JWT, you may need to verify the following claims:

- The `iss` (issuer) claim matches the Issuer of the AccessMatrix UAM Authorization Server configuration in Admin Console.
- The `aud` (audience) claim must match the Client ID that you used to request the JWT. This will be the Client ID for the application created in Admin Console.
- The `sub` (subject) claim must match the unique identifier for the authenticated user.
- The `iat` (issued at time) claim indicates when the JWT was issued, expressed in Unix time.
- The `exp` (expiry time) claim indicates the time that the JWT will expire, expressed in Unix time. Ensure it has not been passed.
- The `nonce` claim value should match whatever was passed during the JWT request.
- The `at_hash` claim refers to the access token's hash value. It should not be used when the access token is not issued. The `at_hash` value is a case-sensitive string that represents the base64url encoding of the left-most half of the hash of the octets of the ASCII representation of the access token value, where the hash algorithm used is SHA-256. In other words, the `access_token` value is hashed with SHA-256, and the left-most 128 bits are base64url encoded.

For authorization code or refresh token JWTs, you may need to verify the following claims:

- The `iss` (issuer) claim matches the Issuer of the AccessMatrix UAM Authorization Server configuration in Admin Console.
 - The `exp` (expiry time) claim indicates the time that the JWT will expire, expressed in Unix time. Ensure it has not been passed.
 - The `iat` (issued at time) claim indicates when the JWT was issued, expressed in Unix time.
-

Sample Code for Validating JWTs (Java)

Java

```
package test;

import java.io.FileInputStream;
import java.io.InputStream;
import java.io.UnsupportedEncodingException;
import java.net.MalformedURLException;
import java.net.URL;
import java.security.Key;
import java.util.Map;

import org.jose4j.jwt.JwtClaims;
import org.jose4j.jwt.consumer.InvalidJwtException;
import org.jose4j.jwt.consumer.JwtConsumer;
import org.jose4j.jwt.consumer.JwtConsumerBuilder;
import org.json.JSONObject;

public class SampleProgramValidateJWT {
    private static final String CLIENT_ID = "w0RSAACK_-
dqW7lu5vlEnA";

    public void validateJWT(String JWT) throws Exception
    {

        // OpenID Connect, sample on reading JWT using
jose4j library, no call to AM server is required
        // Other libraries are available
        // Read OpenID Connect for specification on
id_token validation,
        // jose4j only provides validation on JWT, not
OpenID Connect id_token validation
        HttpsJwks httpsJwks = new
HttpsJwks("https://youroauthProxyDomain/clp/jwks.json");
        VerificationKeyResolver jwksResolver = new
HttpsJwksVerificationKeyResolver(httpsJwks);

        JwtConsumer jwtc = new JwtConsumerBuilder()
            .setRequireExpirationTime()
            .setAllowedClockSkewInSeconds(30)
            .setRequireSubject()
            .setExpectedIssuer("i-sprint.com") // OAuth
Authorization Server's issuer id
            .setExpectedAudience(CLIENT_ID) // OAuth
Client's client_id is the intended audience
            .setVerificationKeyResolver(jwksResolver)
            .build();
        JwtClaims jwtClaims = null;
        try {
            jwtClaims = jwtc.processToClaims(JWT);
        } catch (InvalidJwtException e) {
            e.printStackTrace();
        }
    }
}
```

```
        System.out.println("Exception:
"+e.getMessage());
        return;
    }
}
```

D. About PKCE

The Proof Key for Code Exchange (PKCE), as defined in [RFC 7636](#), is an extension of the Authorization Code flow that is meant to overcome the vulnerability of the exposed client secret and mitigate authorization code interception attacks.

The PKCE extension works as follows:

1. The OAuth client application generates a cryptographically random key called the `code_verifier` (on every authorization request).
2. The `code_verifier` is used to derive a transformed value called the `code_challenge`, which is sent to authorization server's Authorize endpoint and used to retrieve the `authorization_code`.
3. The OAuth client application, upon receiving the `authorization_code`, sends the `authorization_code` and the `code_verifier` to AccessMatrix UAM's Token endpoint to exchange them for the requested tokens.
4. AccessMatrix UAM's Token endpoint recomputes the `code_challenge` from the `code_verifier` using an agreed-upon hash algorithm and then compare them. If the `code_challenge` and `code_verifier` match, it is able to confirm that both requests were sent by the same client.

A rogue application can only intercept the `authorization_code`, but it cannot have access to the `code_challenge` or the `code_verifier`, as they are both sent via HTTPS.

Sample Payload for Dynamic Client Registration

This section provides examples of the sample request and response for each endpoint.

Create OAuth Client

This section is organized into the various flows that can

Create - OAuth 2.0: Authorization Code

Request Example

JSON

```
{
  "app_type": "ServerSideApp",
  "token_endpoint_auth_method": "client_secret_post",
  "client_name": "authzCodeClient",
  "client_secret": "PCpZjQbnDVUbhvlp193zyZTE",
  "status": "Activated",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "required_authn_level": 1,
  "scope": "phone email",
  "is_session_token_embedded": true,
  "allow_all_users_access": false,
  "allow_multiple_profiles": true,
  "logo_uri": "http://logo.com",
  "jwks_uri": "http://jwksURI.com",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9iSMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpKlfyhGt17qZwO6pGFhgVHWywbVtTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "sso_url": "http://ssoURL.com",
  "request_object_sign_algorithm": "RS256",
  "access_token_expiry": 620,
  "access_token_format": "Opaque",
  "access_token_user_response_id": "OIDC_email",
  "jwe_algorithm": "RSA-OAEP-256",
  "jwe_enc_algorithm": "A256CBC-HS512",
  "refresh_token_expiry_in_mins": 1440,
  "refresh_token_lifetime_in_hours": 168,
  "refresh_token_integration_date": false,
  "oAuth_session_concurrency": false,
  "revoke_token_invalid_session": false,
  "default_max_age": 620,
  "subject_value_response_config": "OIDC_email",
  "attr_claim_value_response_config": "OIDC_email",
  "subject_value": "Id",
  "id_token_encrypt": false,
  "consent_rule": "NOT_TRUSTED",
  "remove_scope_upon_token_revoke": "Keep",
  "tos_uri": "http://tos.com",
  "policy_uri": "http://policy.com"
}
```

Response Example (Success)

JSON

Request Example

```
{
  "app_type": "ServerSideApp",
  "default_max_age": "620",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhIJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9i sMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAI0mRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWyWbVtTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "sso_url": "http://ssoURL.com",
  "logo_uri": "http://logo.com",
  "access_token_user_response_id": "OIDC_email",
  "access_token_expiry": "620",
  "registration_client_uri":
"http://localhost:8080/clp/register/tUY0vPHpBVVq8AvYjfrbjg",
  "attr_claim_value_response_config": "OIDC_email",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "registration_access_token": "Xqij99FYERn6TeJNgg4mHCrU",
  "client_id": "tUY0vPHpBVVq8AvYjfrbjg",
  "token_endpoint_auth_method": "client_secret_basic",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "request_object_sign_algorithm": "RS256",
  "scope": "phone email",
  "allow_all_users_access": "false",
  "client_secret": "PCpZjQbnDVUbhvlp193zyZTE",
  "client_name": "authzCodeClient",
  "policy_uri": "http://policy.com",
  "subject_value_response_config": "OIDC_email",
  "allow_multiple_profiles": "true",
  "grant_types": [
    "authorization_code"
  ],
  "jwe_algorithm": "RSA-OAEP-256",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "refresh_token_integration_date": "false",
  "refresh_token_expiry_in_mins": "1440",
  "jwe_enc_algorithm": "A256CBC-HS512",
  "subject_value": "Id",
  "is_session_token_embedded": "true",
  "client_secret_expires_at": "0",
  "jwks_uri": "http://jwksURI.com",
  "tos_uri": "http://tos.com",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated"
}
```


Create - OAuth 2.0: Implicit

Request Example

JSON

```
{
  "app_type": "SinglePageApp",
  "client_name": "implicitClient",
  "client_secret": "vopGdcOLrWbillLgcm4RLeMAH",
  "status": "Activated",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "js_web_origin_uris": [
    "https://jswebOriginuris"
  ],
  "required_authn_level": 1,
  "scope": "phone",
  "is_session_token_embedded": true,
  "allow_all_users_access": false,
  "allow_multiple_profiles": true,
  "logo_uri": "http://logol.com",
  "jwks_uri": "http://www.test1.com/jwks",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_dl_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9iSMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAI0mRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpKlfyhGt17qZwO6pGFhgVHWyWbVtTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "request_object_sign_algorithm": "RS256",
  "access_token_expiry": 620,
  "access_token_format": "Opaque",
  "access_token_user_response_id": "OIDC_email",
  "jwe_algorithm": "RSA-OAEP-256",
  "jwe_enc_algorithm": "A256CBC-HS512",
  "oAuth_session_concurrency": false,
  "default_max_age": 620,
  "subject_value": "Id",
  "subject_value_response_config": "OIDC_email",
  "attr_claim_value_response_config": "OIDC_email",
  "id_token_encrypt": false,
  "consent_rule": "NOT_TRUSTED",
  "remove_scope_upon_token_revoke": "Keep",
  "tos_uri": "http://tos.com",
  "policy_uri": "http://policy.com"
}
```

Response Example (Success)

JSON

```
{
  "app_type": "SinglePageApp",
  "default_max_age": "620",
```

Request Example

```
"jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs631-  
g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9i  
sMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-  
cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-  
udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-  
9RxcdpKlfyhGt17qZwO6pGFhgVHWyWbVTTPSHhao-I2tnCoapuremrcIG56g-w' }",  
  "logo_uri": "http://logo1.com",  
  "access_token_user_response_id": "OIDC_email",  
  "access_token_expiry": "620",  
  "registration_client_uri":  
"http://localhost:8080/clp/register/qEYCONOviCJiSiOUx5Ibrw",  
  "attr_claim_value_response_config": "OIDC_email",  
  "access_token_format": "Opaque",  
  "id_token_encrypt": "false",  
  "registration_access_token": "T-1XRXYrtqFYppitWgU4zVDC",  
  "client_id": "qEYCONOviCJiSiOUx5Ibrw",  
  "consent_rule": "NOT_TRUSTED",  
  "request_object_sign_algorithm": "RS256",  
  "scope": "phone",  
  "js_web_origin_uris": [  
    "https://jswebOriginuris"  
  ],  
  "allow_all_users_access": "false",  
  "client_secret": "vopGdcOLrWbilLgcm4RLeMAH",  
  "client_name": "implicitClient",  
  "policy_uri": "http://policy.com",  
  "subject_value_response_config": "OIDC_email",  
  "allow_multiple_profiles": "true",  
  "grant_types": [  
    "implicit"  
  ],  
  "jwe_algorithm": "RSA-OAEP-256",  
  "redirect_uris": [  
    "http://localhost:8080/clp/register"  
  ],  
  "jwe_enc_algorithm": "A256CBC-HS512",  
  "subject_value": "Id",  
  "is_session_token_embedded": "true",  
  "client_secret_expires_at": "0",  
  "jwks_uri": "http://www.test1.com/jwks",  
  "tos_uri": "http://tos.com",  
  "oAuth_session_concurrency": "false",  
  "remove_scope_upon_token_revoke": "Keep",  
  "required_authn_level": "1",  
  "response_types": [  
    "token",  
    " id_token",  
    " token id_token"  
  ],  
  "status": "Activated"  
}
```

Create - OAuth 2.0: Client Credentials

Request Example	
JSON	<pre>{ "app_type": "ServerToServerApp", "client_name": "clientCredentialClient", "client_secret": "FsbTrc-DczrBGsZ-40D6ArA_", "status": "Activated", "access_token_format": "JWT", "access_token_expiry": 7200, "scope": "email phone", "allow_introspect_other_clients": true }</pre>
Response Example (Success)	
JSON	<pre>{ "grant_types": ["client_credentials"], "allow_introspect_other_clients": "true", "client_secret_expires_at": "0", "scope": "phone email", "access_token_expiry": "7200", "registration_client_uri": "http://localhost:8080/clp/register/DE3dJg5uxCrHhZ1bGnykmQ", "client_secret": "FsbTrc-DczrBGsZ-40D6ArA_", "access_token_format": "JWT", "registration_access_token": "7iFmKcLEuiuqkpTNq8nld5Jm", "client_name": "clientCredentialClient", "client_id": "DE3dJg5uxCrHhZ1bGnykmQ", "status": "Activated" }</pre>

Create - OAuth 2.0: Authorization Code with Client Credentials

Request Example	
JSON	<pre>{ "grant_types": ["authorization_code", "client_credentials"], </pre>

Request Example

```
{
  "client_name": "authzCodeClientWithClientCredential",
  "status": "Activated",
  "access_token_format": "JWT",
  "access_token_expiry": 7200,
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ]
}
```

Response Example (Success)

JSON

```
{
  "app_type": "ServerSideApp",
  "access_token_expiry": "3600",
  "registration_client_uri":
"http://localhost:8080/clp/register/b07HcFdnyCweXjrHwgY4mw",
  "refresh_token_expiry_in_mins": "1440",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "registration_access_token": "PHtdK0KkHjAFRCRwWvLh7dVy",
  "client_id": "b07HcFdnyCweXjrHwgY4mw",
  "token_endpoint_auth_method": "client_secret_basic",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "client_secret": "_7TdbaBKhnYFY2lsQJ6uEeqK",
  "client_name": "authzCodeClientWithClientCredential",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code",
    "client_credentials"
  ],
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "refresh_token_integration_date": "false",
  "subject_value": "Id",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Create - OAuth2.1: Mobile Native App

Request Example

JSON

```
{
  "grant_types": [
    "authorization_code"
  ],
  "token_endpoint_auth_method": "none",
  "pkce_client_auth_method": "private_key_jwt",
  "jwks_uri": "http://172.16.40.37:8080/am5/wpe/jwks.json",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs631-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9i sMZ1QyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWyWbVtTpSHhao-I2tnCoapuremrcIG56g-w' }"
```

Response Example (Success)

JSON

```
{
  "app_type": "MobileNativeApp",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs631-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9i sMZ1QyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWyWbVtTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "pkce_client_auth_method": "private_key_jwt",
  "access_token_expiry": "3600",
  "registration_client_uri":
"http://localhost:8080/clp/register/i0RHnbqna7r84aQwcIf4gQ",
  "refresh_token_expiry_in_mins": "1440",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "registration_access_token": "Nop3qgWgxBhRPveMYBbzqkos",
  "client_id": "i0RHnbqna7r84aQwcIf4gQ",
  "token_endpoint_auth_method": "none",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "client_name": "i0RHnbqna7r84aQwcIf4gQ",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code"
  ],
  "refresh_token_integration_date": "false",
  "subject_value": "Id",
  "is session token embedded": "false",
```

Request Example

```
{
  "jwks_uri": "http://172.16.40.37:8080/am5/wpe/jwks.json",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated"
}
```

Create - OAuth 2.1: Authorization Code with PKCE

Request Example

JSON

```
{
  "app_type": "AuthorizationCodePKCEFlow",
  "token_endpoint_auth_method": "private_key_jwt",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs631-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9iSMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxx3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWywbVtTPSHhao-I2tnCoapuremrcIG56g-w' }"
}
```

Response Example (Success)

JSON

```
{
  "app_type": "AuthorizationCodePKCEFlow",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs631-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9iSMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxx3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWywbVtTPSHhao-I2tnCoapuremrcIG56g-w' }",
  "pkce_client_auth_method": "private_key_jwt",
  "access_token_expiry": "3600",
  "registration_client_uri":
"http://localhost:8080/clp/register/vU1PCvA8qSgtleW74Cz7jw",
  "refresh_token_expiry_in_mins": "1440",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "registration_access_token": "MeE0ywdOUdEJLYvC2T24DdJ_",
  "client_id": "vU1PCvA8qSgtleW74Cz7jw",
  "token_endpoint_auth_method": "private_key_jwt",
  "consent_rule": "NOT TRUSTED",
}
```

Request Example

```
{
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "client_secret": "SpoilCiFJYmqMF2eqpG0snhT",
  "client_name": "vU1PCvA8qSgtleW74Cz7jw",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code"
  ],
  "refresh_token_integration_date": "false",
  "subject_value": "Id",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Create - OAuth 2.1: Hybrid with PKCE

Request Example

JSON

```
{
  "grant_types": [
    "authorization_code"
  ],
  "response_types": [
    "code id_token"
  ],
  "token_endpoint_auth_method": "private_key_jwt",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9iSMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWyBvTTPSHhao-I2tnCoapuremrcIG56g-w' }",
  "status": "Activated",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ]
}
```

Response Example (Success)

Request Example

JSON

```
{
  "app_type": "HybridWithPKCEFlow",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9iSMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWywbVtTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "pkce_client_auth_method": "private_key_jwt",
  "access_token_expiry": "3600",
  "registration_client_uri":
"http://localhost:8080/clp/register/aEcMCUZUb0ra2nqtX7c7hQ",
  "refresh_token_expiry_in_mins": "1440",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "registration_access_token": "vsMrEdmgnJoEmhf0Bfmrz_uH",
  "client_id": "aEcMCUZUb0ra2nqtX7c7hQ",
  "token_endpoint_auth_method": "private_key_jwt",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "client_secret": "L1ZWCxu7rZnofqzdW_BCdK9Y",
  "client_name": "aEcMCUZUb0ra2nqtX7c7hQ",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code"
  ],
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "refresh_token_integration_date": "false",
  "subject_value": "Id",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code id_token"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Update OAuth Client

Update - OAuth 2.0: Authorization Code

Request Example

Request Example

JSON

```
{
  "client_id": "SEV1GcDuYGVHpAWTq6R3h1",
  "token_endpoint_auth_method": "client_secret_post",
  "client_name": "authzCodeClient1",
  "client_secret": "PCpZjQbnDVUbhvlp193zyZTE",
  "status": "Disabled",
  "redirect_uris": [
    "http://localhost:8080/clp/register",
    "http://localhost:8080/clp/register2"
  ],
  "required_authn_level": 3,
  "scope": "phone",
  "is_session_token_embedded": false,
  "allow_all_users_access": true,
  "allow_multiple_profiles": false,
  "logo_uri": "http://logol.com",
  "jwks_uri": "http://jwksURI1.com",
  "sso_url": "http://ssoURL.com",
  "request_object_sign_algorithm": "RS384",
  "access_token_expiry": 630,
  "access_token_format": "JWE",
  "access_token_user_response_id": "OIDC_phone",
  "jwe_algorithm": "ECDH-ES+A256KW",
  "jwe_enc_algorithm": "A256GCM",
  "refresh_token_expiry_in_mins": 1440,
  "refresh_token_lifetime_in_hours": 210,
  "refresh_token_integration_date": true,
  "oAuth_session_concurrency": true,
  "revoke_token_invalid_session": true,
  "default_max_age": 600,
  "subject_value": "Name",
  "subject_value_response_config": "OIDC_phone",
  "attr_claim_value_response_config": "OIDC_phone",
  "id_token_encrypt": true,
  "consent_rule": "TRUSTED",
  "remove_scope_upon_token_revoke": "Revoke All",
  "tos_uri": "http://tos1.com",
  "policy_uri": "http://policy1.com",
}
```

Response Example (Success)

JSON

```
{
  "app_type": "ServerSideApp",
  "default_max_age": "600",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhIJ47qWs631-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9ismZ1QyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNSDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-
```

Request Example

```
udbHElA3tQuSmI88aDbZrxTu4sKpoZGUlsSAyJjujF20-
9RxcdpK1fyhGt17qZwO6pGFhgVHWyBvTTPSHhao-I2tnCoapuremrcIG56g-w' }",
  "sso_url": "http://ssoURL.com",
  "logo_uri": "http://logo1.com",
  "access_token_user_response_id": "OIDC_phone",
  "access_token_expiry": "630",
  "attr_claim_value_response_config": "OIDC_phone",
  "access_token_format": "JWE",
  "id_token_encrypt": "true",
  "client_id": "SEV1GcDuYGVHpAWTq6R3h1",
  "token_endpoint_auth_method": "client_secret_basic",
  "consent_rule": "TRUSTED",
  "refresh_token_lifetime_in_hours": "210",
  "revoke_token_invalid_session": "true",
  "request_object_sign_algorithm": "RS384",
  "scope": "phone",
  "allow_all_users_access": "true",
  "client_secret": "PCpZjQbnDVUbhvp193zyZTE",
  "client_name": "authzCodeClient1",
  "policy_uri": "http://policy1.com",
  "subject_value_response_config": "OIDC_phone",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code",
    "refresh_token"
  ],
  "jwe_algorithm": "ECDH-ES+A256KW",
  "redirect_uris": [
    "http://localhost:8080/clp/register2",
    "http://localhost:8080/clp/register"
  ],
  "refresh_token_integration_date": "true",
  "refresh_token_expiry_in_mins": "1440",
  "jwe_enc_algorithm": "A256GCM",
  "subject_value": "Name",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "jwks_uri": "http://jwksURI1.com",
  "tos_uri": "http://tos1.com",
  "oAuth_session_concurrency": "true",
  "remove_scope_upon_token_revoke": "Revoke All",
  "required_authn_level": "3",
  "response_types": [
    "code"
  ],
  "status": "Disabled"
}
```

Update - OAuth 2.0: Implicit

Request Example

Request Example

JSON

```
{
  "client_id": "qEYCONOviCJiSIoUx5Ibrw",
  "client_name": "implicitClient2",
  "client_secret": "vopGdcOLrWbillLgcm4RLeMAH",
  "status": "Activated",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "js_web_origin_uris": [
    "https://jswebOriginuris1"
  ],
  "required_authn_level": 3,
  "scope": "phone",
  "is_session_token_embedded": false,
  "allow_all_users_access": true,
  "allow_multiple_profiles": false,
  "logo_uri": "http://logo2.com",
  "jwks_uri": "http://www.test2.com/jwks",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9ismZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpozGUlsSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWyWbVTTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "request_object_sign_algorithm": "RS256",
  "access_token_expiry": 640,
  "access_token_format": "JWE",
  "access_token_user_response_id": "OIDC_phone",
  "jwe_algorithm": "RSA-OAEP-256",
  "jwe_enc_algorithm": "A256CBC-HS512",
  "oAuth_session_concurrency": false,
  "default_max_age": 640,
  "subject_value": "Id",
  "subject_value_response_config": "OIDC_phone",
  "attr_claim_value_response_config": "OIDC_phone",
  "id_token_encrypt": true,
  "consent_rule": "TRUSTED",
  "remove_scope_upon_token_revoke": "Keep",
  "tos_uri": "http://tos1.com",
  "policy_uri": "http://policy1.com"
}
```

Response Example (Success)

JSON

```
{
  "app_type": "SinglePageApp",
  "default_max_age": "640",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9i"
}
```

Request Example

```
SMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-
cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-
udbHElA3tQuSmI88aDbZrxtu4sKpozGUlsSAyJjujF20-
9RxcdpKlfyhGt17qZwO6pGFhgVHWywbVtTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "logo_uri": "http://logo2.com",
  "access_token_user_response_id": "OIDC_phone",
  "access_token_expiry": "640",
  "attr_claim_value_response_config": "OIDC_phone",
  "access_token_format": "JWE",
  "id_token_encrypt": "true",
  "client_id": "qEYCONOviCJiSiOUx5Ibrw",
  "consent_rule": "TRUSTED",
  "request_object_sign_algorithm": "RS256",
  "scope": "phone",
  "js_web_origin_uris": [
    "https://jswebOriginuris1"
  ],
  "allow_all_users_access": "true",
  "client_secret": "vopGdcOLrWbillGcm4RLeMAH",
  "client_name": "implicitClient2",
  "policy_uri": "http://policy1.com",
  "subject_value_response_config": "OIDC_phone",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "implicit"
  ],
  "jwe_algorithm": "RSA-OAEP-256",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "jwe_enc_algorithm": "A256CBC-HS512",
  "subject_value": "Id",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "jwks_uri": "http://www.test2.com/jwks",
  "tos_uri": "http://tos1.com",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "3",
  "response_types": [
    "token",
    "id_token",
    "token id_token"
  ],
  "status": "Activated"
}
```

Update - OAuth 2.0: Client Credentials

Request Example

Request Example	
JSON	<pre>{ "client_id": "DE3dJg5uxCrHhZ1bGnykmQ", "client_name": "clientCredentialClient2", "client_secret": "FsbTrc-DczrBGsZ-40D6ArA_", "status": "Disabled", "access_token_expiry": 3200, "allow_introspect_other_clients": false, "access_token_format": "Opaque", "scope": "email" }</pre>
Response Example (Success)	
JSON	<pre>{ "grant_types": ["client_credentials"], "allow_introspect_other_clients": "false", "client_secret_expires_at": "0", "scope": "email", "access_token_expiry": "3200", "registration_client_uri": "http://localhost:8080/clp/register/DE3dJg5uxCrHhZ1bGnykmQ", "client_secret": "FsbTrc-DczrBGsZ-40D6ArA_", "access_token_format": "Opaque", "registration_access_token": "mWmuYUq9zPl0iqcFUCup_j4b", "client_name": "clientCredentialClient2", "client_id": "DE3dJg5uxCrHhZ1bGnykmQ", "status": "Disabled" }</pre>

Update - OAuth 2.0: Authorization Code with Client Credentials

Request Example	
JSON	<pre>{ "client_id": "b07HcFdnyCweXjrHwgY4mw", "token_endpoint_auth_method": "client_secret_post", "client_name": "authzCodeClientWithClientCredential1", }</pre>

Request Example	
Response Example (Success)	
JSON	<pre> { "app_type": "ServerSideApp", "access_token_expiry": "3600", "registration_client_uri": "http://localhost:8080/clp/register/b07HcFdnyCweXjrHwgY4mw", "refresh_token_expiry_in_mins": "1440", "access_token_format": "Opaque", "id_token_encrypt": "false", "registration_access_token": "KDDZtfEWCAA3PK-NbYa2jf1v", "client_id": "b07HcFdnyCweXjrHwgY4mw", "token_endpoint_auth_method": "client_secret_basic", "consent_rule": "NOT_TRUSTED", "refresh_token_lifetime_in_hours": "168", "revoke_token_invalid_session": "false", "allow_all_users_access": "true", "client_secret": "_7TdbaBKhnYFY2lsQJ6uEeqK", "client_name": "authzCodeClientWithClientCredential1", "allow_multiple_profiles": "false", "grant_types": ["authorization_code", "client_credentials"], "refresh_token_integration_date": "false", "subject_value": "Id", "is_session_token_embedded": "false", "client_secret_expires_at": "0", "oAuth_session_concurrency": "false", "remove_scope_upon_token_revoke": "Keep", "required_authn_level": "1", "response_types": ["code"], "status": "Activated", "client_secret_validity_period_in_mins": "0" } </pre>

Update - OAuth 2.0: Mobile Native App

Request Example	
JSON	<pre> { "client_id": "b07HcFdnyCweXjrHwgY4mw", "client_name": "authzCodeClientWithMobileNativeApp", } </pre>

Request Example
Response Example (Success)
JSON <pre> { "app_type": "MobileNativeApp", "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9iSMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAIOMRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpozGULsSAyJjujF20-9RxcdpKlfyhGt17qZwO6pGFhgVHWywBvTtpSHhao-I2tnCoapuremrcIG56g-w' }", "pkce_client_auth_method": "private_key_jwt", "access_token_expiry": "3600", "refresh_token_expiry_in_mins": "1440", "access_token_format": "Opaque", "id_token_encrypt": "false", "client_id": "b07HcFdnyCweXjrHwgY4mw", "token_endpoint_auth_method": "none", "consent_rule": "NOT_TRUSTED", "refresh_token_lifetime_in_hours": "168", "revoke_token_invalid_session": "false", "allow_all_users_access": "true", "client_name": "authzCodeClientWithMobileNativeApp", "allow_multiple_profiles": "false", "grant_types": ["authorization_code"], "redirect_uris": ["http://localhost:8080/clp/register"], "refresh_token_integration_date": "false", "subject_value": "Id", "is_session_token_embedded": "false", "jwks_uri": "http://jwksURI.com", "oAuth_session_concurrency": "false", "remove_scope_upon_token_revoke": "Keep", "required_authn_level": "1", "response_types": ["code"], "status": "Activated" }</pre>

Update - OAuth 2.1: Authorization Code with PKCE

Request Example
JSON

Request Example

```
{
  "client_id": "b07HcFdnyCweXjrHwgY4mw",
  "client_name":
"authzCodeClientWithAuthorizationCodePKCEFlow",
}
```

Response Example (Success)

JSON

```
{
  "app_type": "AuthorizationCodePKCEFlow",
  "default_max_age": "620",
  "jwks": "{ 'keys': [{ 'kty': 'RSA', 'e': 'AQAB', 'use': 'sig', 'kid': 'OAS-
AKAN_wQRZ4g', 'alg': 'RS256', 'n': 'voIMw4NQfxl37AQPwi_mPlajjNY4SAbIDMYRvd88tPUo
fPcI6mcaCtBud6yjp60Tcjb8tLIC25321XX7FUhyFXeOKoqiLjlhpXhdPzeBO6Csz-
8TQRn6mYGBJgRt-
VoMZ0ujwfuMd4DzS1RoPPAiio2i8lmCRwyuB2DDAhrHh29BTCzzk6FZekqLET_Ycpsf22LG8HYg
wtaZUJW9v2bc824ImTiFvYknk-QMPiO52b1SXvzu-5L-
vsmrGWIlJjzCksykByPV_aLjiCy6gRP4DVVQIEskpMA2pNmv_n4s_D48Qn8pVE7CYMW_S3m_UlSR
YwIWWUoHBnqZsGnlL8SfW' }, { 'kty': 'RSA', 'e': 'AQAB', 'use': 'sig', 'kid': 'pkHDuE0BY
c9az7KGpX5AgZ-
UAGhsmTiaZJofs8gnc_A', 'alg': 'RS256', 'n': 'mEJJpgaEzG3hSumjkgfGK9o-
FSqi0sNJ9_V8MND_Dbwca0VfDtR2MtiSzNsJ2A7_MmNX4IgmYcPbgHdymWMqRDSQm4H-
6d9TvseG9ciJdxNzRbZ0ZVXdKhToY4CMK11XLVGiu1Tha_KOObOGQpggswNrgJ4lmLG-
FtUHEFj5ov1FCcKa1zUg3nbklArf5u13DvUBGoS9IZsarWE7gvEzq7suQrIYq1H5u9FGsZDvmO9a
NpVZZIUIJOhpJx7pgHXAo03OgKmZtlnNyKuKuq_9ew60KCC3CJqWoLW92oVS4J-
NZIZ5Vv9NJ_yPRkkX-LW0eXrgVmoIwqfPlnQ7CBfxj9kw' } ] }, 'version': 5 }",
  "logo_uri": "http://logo.com",
  "access_token_user_response_id": "OIDC_email",
  "pkce_client_auth_method": "private_key_jwt",
  "access_token_expiry": "620",
  "attr_claim_value_response_config": "OIDC_email",
  "refresh_token_expiry_in_mins": "1500",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "client_id": "b07HcFdnyCweXjrHwgY4mw",
  "token_endpoint_auth_method": "private_key_jwt",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "request_object_sign_algorithm": "RS256",
  "allow_all_users_access": "true",
  "client_secret": "ijner3SVa4zKvh7SAqgR8sj1",
  "client_name": "authzCodeClientWithAuthorizationCodePKCEFlow",
  "policy_uri": "http://localhost:8080/clp/register",
  "subject_value_response_config": "GroupMembership",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code"
  ],
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
}
```


Request Example

```
{
  "refresh_token_integration_date": "false",
  "jwe_enc_algorithm": "A256GCM",
  "subject_value": "Id",
  "is_session_token_embedded": "true",
  "client_secret_expires_at": "0",
  "jwks_uri": "http://www.test2.com/jwks",
  "tos_uri": "http://tos1.com",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Update - OAuth 2.1: Hybrid with PKCE

Request Example

JSON

```
{
  "client_id": "b07HcFdnyCweXjrHwgY4mw",
  "client_name": "authzCodeClientWithHybridWithPKCEFlow",
}
```

Response Example (Success)

JSON

```
{
  "app_type": "HybridWithPKCEFlow",
  "default_max_age": "620",
  "jwks": "{ 'keys': [{ 'kty': 'RSA', 'e': 'AQAB', 'use': 'sig', 'kid': 'OAS-  
AKAN_wQRZ4g', 'alg': 'RS256', 'n': 'voIMw4NQfxl37AQPwi_mPlajjNY4SAbIDMYRvd88tPUo  
fPcI6mcaCtBud6yjp60Tcjb8tLIC25321XX7FUhyFXeOKoqiLj1hpXhdPzeBO6Csz-  
8TQRn6mYGBJgRt-  
VoMZ0ujwfuMd4DzS1RoPPAiio2i81mCRwyuB2DDAhrHh29BTCzzk6FZekqLET_Ycpsf22LG8HYg  
wtaZUJW9v2bc824ImTiFvYknk-QMPiO52b1SXvzu-5L-  
vsmrGWI1JjzCksykByypV_aljiCy6gRP4DVVQIESkpMA2pNmV_n4s_D48Qn8pVE7CYMW_S3m_U1SR  
YwIWWUoHBnqZsGn1L8SFw' }, { 'kty': 'RSA', 'e': 'AQAB', 'use': 'sig', 'kid': 'pkHDuE0BY  
c9az7KGpX5AgZ-  
UAGhsmTIaZJofs8gnc_A', 'alg': 'RS256', 'n': 'mEJJpgaEzG3hSUmjkgfGK9o-  
FSqi0sNJ9_V8MND_Dbwca0VfDtr2MIiSzNsJ2A7_MmNX4IgmYcPbgHdymWMqRDSQm4H-  
6d9TvseG9ciJdxNzRbZ0ZVXdkhtoY4CMK11XLVGIU1Tha_KOObOGQpggswNrgJ41mLG-  
FtUHEFj5ov1FCcKA1zUg3nbklArf5ul3DvUBGoS9IZsarWE7gvEzq7suQrIYq1H5u9FGsZDvmO9a  
NpVZZIUIJOhpJx7pgHXAo03OgKmZtlnNyKuKuq_9ew60KCC3CJqWoLW92oVS4J-  
NZIZ5Vv9NJ_yPRKkX-LW0eXrgVmoIwqfPlnQ7CBfxj9kw' } ], 'version': 5 }",
}
```

Request Example

```
{
  "logo_uri": "http://logo.com",
  "access_token_user_response_id": "OIDC_email",
  "pkce_client_auth_method": "client_secret_basic",
  "access_token_expiry": "620",
  "attr_claim_value_response_config": "OIDC_email",
  "refresh_token_expiry_in_mins": "1500",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "client_id": "b07HcFdnyCweXjrHwgY4mw",
  "token_endpoint_auth_method": "client_secret_basic",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "request_object_sign_algorithm": "RS256",
  "allow_all_users_access": "true",
  "client_secret": "ijner3SVa4zKvh7SAggR8sj1",
  "client_name": "authzCodeClientWithHybridWithPKCEFlow",
  "policy_uri": "http:\\\\policy.com",
  "subject_value_response_config": "GroupMembership",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code"
  ],
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "refresh_token_integration_date": "false",
  "jwe_enc_algorithm": "A256GCM",
  "subject_value": "Id",
  "is_session_token_embedded": "true",
  "client_secret_expires_at": "0",
  "jwks_uri": "http://www.test2.com/jwks",
  "tos_uri": "http://tos1.com",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Reset OAuth Client Secret

Reset - OAuth 2.0: Authorization Code

Response Example (Success)

JSON

Response Example (Success)

```
{
  "grant_types": [
    "authorization_code",
    "refresh_token"
  ],
  "default_max_age": "600",
  "logo_uri": "http://logo1.com",
  "access_token_expiry": "630",
  "redirect_uris": [
    "http://localhost:8080/clp/register2",
    "http://localhost:8080/clp/register"
  ],
  "client_id": "tUY0vPHpBVVq8AvYjfrbjg",
  "client_secret_expires_at": "0",
  "scope": "phone",
  "client_secret": "jxbXB6qAEuwVz1Z1GFGJ_FTt",
  "tos_uri": "http://tos1.com",
  "client_name": "authzCodeClient1",
  "response_types": [
    "code"
  ],
  "policy_uri": "http://policy1.com",
  "status": "Disabled"
}
```

Reset - OAuth 2.0: Implicit

Response Example (Success)

JSON

```
{
  "grant_types": [
    "implicit"
  ],
  "default_max_age": "640",
  "logo_uri": "http://logo2.com",
  "access_token_expiry": "640",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "client_id": "qEYCONOviCJiSIoUx5Ibrw",
  "client_secret_expires_at": "0",
  "scope": "phone",
  "client_secret": "SxQ8QF_lAJjWLpckcAUy9ScR",
  "tos_uri": "http://tos1.com",
  "client_name": "implicitClient2",
  "response_types": [
    "token",
    "id_token",
    "token id_token"
  ],
}
```

Response Example (Success)

```
{
  "policy_uri": "http://policy1.com",
  "status": "Activated"
}
```

Reset - OAuth 2.0: Client Credentials

Response Example (Success)

JSON

```
{
  "grant_types": [
    "client_credentials"
  ],
  "allow_introspect_other_clients": "false",
  "client_secret_expires_at": "0",
  "scope": "email",
  "access_token_expiry": "3200",
  "client_secret": "F8xPPCVLVbF5hOzdwH1KA2Gb",
  "client_name": "clientCredentialClient2",
  "client_id": "DE3dJg5uxCrHhZ1bGnykmQ",
  "status": "Disabled"
}
```

Reset - OAuth 2.0: Authorization Code with Client Credentials

Response Example (Success)

JSON

```
{
  "app_type": "ServerSideApp",
  "access_token_expiry": "3600",
  "registration_client_uri":
"http://localhost:8080/am5/wpe/WebPolicyEditor.jsp/b07HcFdnyCweXjrHwgY4mw",
  "refresh_token_expiry_in_mins": "1440",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "registration_access_token": "KDDZtfEWCAA3PK-NbYa2jf1v",
  "client_id": "b07HcFdnyCweXjrHwgY4mw",
  "token_endpoint_auth_method": "client_secret_basic",
  "consent_rule": "NOT_TRUSTED",
}
```

Response Example (Success)

```
{
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "client_secret": "qSHpvoM8B5AQXigBxbWRja11",
  "client_name": "authzCodeClientWithClientCredential1",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code",
    "client_credentials"
  ],
  "refresh_token_integration_date": "false",
  "subject_value": "Id",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Reset - OAuth 2.1: Authorization Code with PKCE

Response Example (Success)

JSON

```
{
  "grant_types": [
    "authorization_code",
    "refresh_token"
  ],
  "client_secret_expires_at": "0",
  "access_token_expiry": "3600",
  "client_secret": "4P9fSuZXyEeOuhp2kPvCsZRe",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "client_name": "authzCodeClient",
  "client_id": "ZE12qw7uU0-B_1h4cpQulw",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Reset - OAuth 2.1: Hybrid with PKCE

Response Example (Success)
JSON
<pre>{ "grant_types": ["authorization_code"], "client_secret_expires_at": "0", "access_token_expiry": "3600", "client_secret": "KBuxuVa7T9OvQopv2uBNHaEa", "client_name": "authzCodeClient", "client_id": "lUqttU57pLAej0MEDFyVpQ", "response_types": ["code id_token"], "status": "Activated", "client_secret_validity_period_in_mins": "0" }</pre>

Read OAuth Client Metadata

Read - OAuth 2.0: Authorization Code

Response Example (Success)
JSON
<pre>{ "grant_types": ["authorization_code", "refresh_token"], "default_max_age": "600", "logo_uri": "http://logol.com", "access_token_expiry": "630", "redirect_uris": ["http://localhost:8080/clp/register2", "http://localhost:8080/clp/register"], "client_id": "tUY0vPHpBVVq8AvYjfrbjg", "client_secret_expires_at": "0", "scope": "phone", "client_secret": "jxbXB6qAEuwVz1Z1GFGJ_FTt", "tos_uri": "http://tos1.com", "client_name": "authzCodeClient1", "response_types": ["code"], }</pre>

Response Example (Success)
<pre>"policy_uri": "http://policy1.com", "status": "Disabled" }</pre>

Read - OAuth 2.0: Implicit

Response Example (Success)		
<table border="1"><thead><tr><th>JSON</th></tr></thead><tbody><tr><td><pre>{ "grant_types": ["implicit"], "default_max_age": "640", "logo_uri": "http://logo2.com", "access_token_expiry": "640", "redirect_uris": ["http://localhost:8080/clp/register"], "client_id": "qEYCONOviCJiSIoUx5Ibrw", "client_secret_expires_at": "0", "scope": "phone", "client_secret": "SxQ8QF_lAJjWLpckcAUy9ScR", "tos_uri": "http://tos1.com", "client_name": "implicitClient2", "response_types": ["token", " id_token", " token id_token"], "policy_uri": "http://policy1.com", "status": "Activated" }</pre></td></tr></tbody></table>	JSON	<pre>{ "grant_types": ["implicit"], "default_max_age": "640", "logo_uri": "http://logo2.com", "access_token_expiry": "640", "redirect_uris": ["http://localhost:8080/clp/register"], "client_id": "qEYCONOviCJiSIoUx5Ibrw", "client_secret_expires_at": "0", "scope": "phone", "client_secret": "SxQ8QF_lAJjWLpckcAUy9ScR", "tos_uri": "http://tos1.com", "client_name": "implicitClient2", "response_types": ["token", " id_token", " token id_token"], "policy_uri": "http://policy1.com", "status": "Activated" }</pre>
JSON		
<pre>{ "grant_types": ["implicit"], "default_max_age": "640", "logo_uri": "http://logo2.com", "access_token_expiry": "640", "redirect_uris": ["http://localhost:8080/clp/register"], "client_id": "qEYCONOviCJiSIoUx5Ibrw", "client_secret_expires_at": "0", "scope": "phone", "client_secret": "SxQ8QF_lAJjWLpckcAUy9ScR", "tos_uri": "http://tos1.com", "client_name": "implicitClient2", "response_types": ["token", " id_token", " token id_token"], "policy_uri": "http://policy1.com", "status": "Activated" }</pre>		

Read - OAuth 2.0: Client Credentials

Response Example (Success)		
<table border="1"><thead><tr><th>JSON</th></tr></thead><tbody><tr><td><pre>{ "grant_types": ["client_credentials"], "allow_introspect_other_clients": "false", "client_secret_expires_at": "0", "scope": "email", </pre></td></tr></tbody></table>	JSON	<pre>{ "grant_types": ["client_credentials"], "allow_introspect_other_clients": "false", "client_secret_expires_at": "0", "scope": "email", </pre>
JSON		
<pre>{ "grant_types": ["client_credentials"], "allow_introspect_other_clients": "false", "client_secret_expires_at": "0", "scope": "email", </pre>		

Response Example (Success)

```
{
  "access_token_expiry": "3200",
  "client_secret": "F8xPPCVLVbF5hOzdwH1KA2Gb",
  "client_name": "clientCredentialClient2",
  "client_id": "DE3dJg5uxCrHhZ1bGnykmQ",
  "status": "Disabled"
}
```

Read - OAuth 2.0: Authorization Code with Client Credentials

Response Example (Success)

JSON

```
{
  "grant_types": [
    "authorization_code",
    "client_credentials"
  ],
  "client_secret_expires_at": "0",
  "access_token_expiry": "3600",
  "client_secret": "qSHpvoM8B5AQXigBxbWRja11",
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "client_name": "authzCodeClientWithClientCredential1",
  "client_id": "b07HcFdnyCweXjrHwgY4mw",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Read - OAuth 2.0: Hybrid with PKCE

Response Example (Success)

JSON

```
{
  "app_type": "MobileNativeApp",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code",
    "refresh_token"
  ],
  "pkce_client_auth_method": "none",
  "access token expiry": "3600",
}
```


Response Example (Success)

```
{
  "refresh_token_expiry_in_mins": "1440",
  "refresh_token_integration_date": "false",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "client_id": "w0ePYZg6ot-2nOuGdeknsW",
  "token_endpoint_auth_method": "none",
  "consent_rule": "NOT_TRUSTED",
  "subject_value": "Id",
  "refresh_token_lifetime_in_hours": "168",
  "is_session_token_embedded": "false",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "oAuth_session_concurrency": "false",
  "client_name": "w0ePYZg6ot-2nOuGdeknsW",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated"
}
```

Read - OAuth 2.1: Authorization Code with PKCE

Response Example (Success)

JSON

```
{
  "app_type": "AuthorizationCodePKCEFlow",
  "jwks": "{ 'kty': 'RSA', 'e': 'AQAB', 'n': 'meSpPUen6UhWOhiJ47qWs63l-g6_d1_XywBj_r0M2bqrFZ6UIHZeHISCxx8_TwnmVc8o4B1IRtDZDbom1Tf_B7vz6T9jjs8hUyk9i sMZlQyyAKO6U6Q-H0Uawhpb5N00-6WB5hXP4IGnzMfXQTUyMMthewaUjehEpvYeX4x7K2-cFwmcAuQmHNsDupJG34ZiIG7YXviiVTaQVQAI0mRxr3cAfufwy3_OAhqTZarVBa-udbHElA3tQuSmI88aDbZrxtu4sKpoZGU1sSAyJjujF20-9RxcdpK1fyhGt17qZwO6pGFhgVHWyWbVTTpSHhao-I2tnCoapuremrcIG56g-w' }",
  "pkce_client_auth_method": "private_key_jwt",
  "access_token_expiry": "3600",
  "refresh_token_expiry_in_mins": "1440",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "client_id": "ZE12qw7uU0-B_1h4cpQulw",
  "token_endpoint_auth_method": "private_key_jwt",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "client_secret": "4P9fSuZXyEeOuhp2kPvCsZRe",
  "client_name": "authzCodeClient",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code",
  ]
}
```

Response Example (Success)

```
    "refresh_token"
  ],
  "redirect_uris": [
    "http://localhost:8080/clp/register"
  ],
  "refresh_token_integration_date": "false",
  "subject_value": "Id",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
  "required_authn_level": "1",
  "response_types": [
    "code"
  ],
  "status": "Activated",
  "client_secret_validity_period_in_mins": "0"
}
```

Read - OAuth 2.1: Hybrid with PKCE

Response Example (Success)

JSON

```
{
  "app_type": "HybridWithPKCEFlow",
  "pkce_client_auth_method": "client_secret_basic",
  "access_token_expiry": "3600",
  "refresh_token_expiry_in_mins": "1440",
  "access_token_format": "Opaque",
  "id_token_encrypt": "false",
  "client_id": "lUqttU57pLAej0MEDFyVpQ",
  "token_endpoint_auth_method": "client_secret_basic",
  "consent_rule": "NOT_TRUSTED",
  "refresh_token_lifetime_in_hours": "168",
  "revoke_token_invalid_session": "false",
  "allow_all_users_access": "true",
  "client_secret": "Tz_KJh2eJwovve5KPZ38SiNw",
  "client_name": "lUqttU57pLAej0MEDFyVpQ",
  "allow_multiple_profiles": "false",
  "grant_types": [
    "authorization_code"
  ],
  "refresh_token_integration_date": "false",
  "subject_value": "Id",
  "is_session_token_embedded": "false",
  "client_secret_expires_at": "0",
  "oAuth_session_concurrency": "false",
  "remove_scope_upon_token_revoke": "Keep",
}
```

Response Example (Success)
<pre> "required_authn_level": "1", "response_types": ["code id_token"], "status": "Activated", "client_secret_validity_period_in_mins": "0" } </pre>

Sample Error Response

Create OAuth Client

Create - Status Code 400: Bad Request

The following error is returned as a JSON object in the response body.

Response Example (invalid_client_metadata)
JSON <pre> Response Status code: 400 { "error_description": "grant_types: wrongGrantType are not supported.", "error": "invalid_client_metadata" } </pre>

Create - Status Code 401: Unauthorized

The following error is returned through the header.

Response Example (invalid_token)
Code <pre> Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token" </pre>

Status Code 500: Internal Server Error

Update OAuth Client

Update - Status Code 400: Bad Request:

The following error is returned as a JSON object in the response body.

Response Example (invalid_client_metadata)	
JSON	
<pre>Response Status code: 400 { "error_description": "wrongScope scope is not allowed. Scope values need to be subset of OAuth Authz Server supported scopes.", "error": "invalid_client_metadata" }</pre>	

Update - Status Code 401: Unauthorized

The following errors are returned through the header.

Response Example (unauthorized_client)	
Code	
<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="Unauthorized client_id",error="unauthorized_client"</pre>	

Response Example (invalid_token)	
Code	
<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre>	

Status Code 500: Internal Server Error

Reset OAuth Client Secret

Reset - Status Code 401: Unauthorized

The following errors are returned through the header.

Response Example (unauthorized_client)	
Code	<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="Unauthorized client_id",error="unauthorized_client"</pre>

Response Example (invalid_token)	
Code	<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre>

Status Code 500: Internal Server Error

Read OAuth Client Metadata

Read - Status Code 401: Unauthorized

The following errors are returned through the header.

Response Example (unauthorized_client)	
Code	<pre>Response Status Code: 401</pre>

Response Example (unauthorized_client)
<pre>WWW-Authenticate: Bearer error_description="Unauthorized client_id",error="unauthorized_client"</pre>

Response Example (invalid_token)		
<table> <tr> <th>Code</th></tr> <tr> <td> <pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre> </td></tr> </table>	Code	<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre>
Code		
<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre>		

Status Code 500: Internal Server Error

Delete OAuth Client

Delete - Status Code 401: Unauthorized

The following errors are returned through the header.

Response Example (unauthorized_client)		
<table> <tr> <th>Code</th></tr> <tr> <td> <pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="Unauthorized client_id",error="unauthorized_client"</pre> </td></tr> </table>	Code	<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="Unauthorized client_id",error="unauthorized_client"</pre>
Code		
<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="Unauthorized client_id",error="unauthorized_client"</pre>		

Response Example (invalid_token)		
<table> <tr> <th>Code</th></tr> <tr> <td> <pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre> </td></tr> </table>	Code	<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre>
Code		
<pre>Response Status Code: 401 WWW-Authenticate: Bearer error_description="invalid initial access token",error="invalid_token"</pre>		

Status Code 500: Internal Server Error

Secondary Account Parameters

The parameters to be used for identity mapping via secondary account is tabled below.

Field	Data Type	Description
id	String	The OAuth client's user identity
appld	String	The OAuth client's application ID
appObjectClass	String	Specify "OAuthClient".
profileName	String	Specify a unique profile name if multiple profiles are expected and used by the same end user for this OAuth client. If not specified, default value "profile1" is used.
